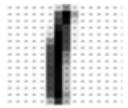
PHENIKAA UNIVERSITY

Faculty of Computer Science

COMPUTER VISION
Convolutional Neural Networks

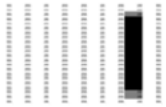
Dang Thi Thuy An

Building an Intuition for CNNs



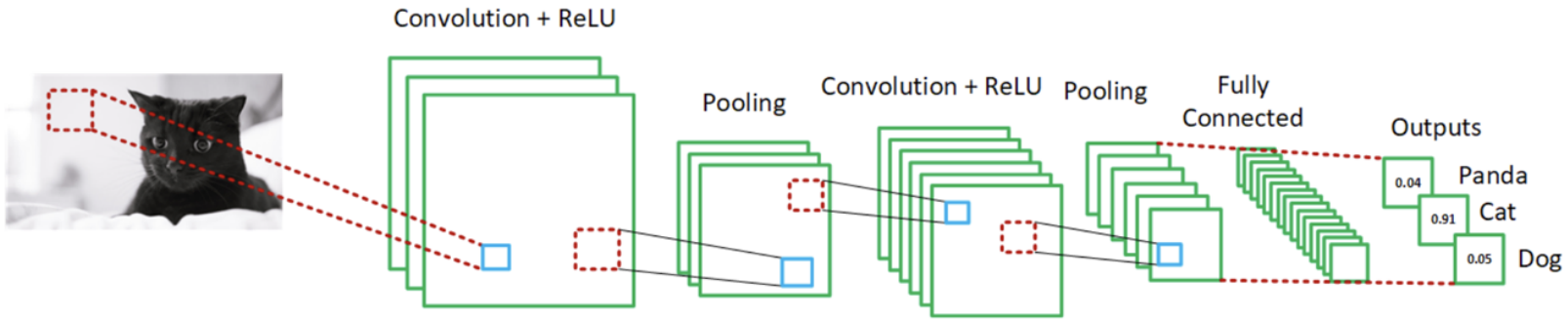
- What digit is this?
- How would we program a computer to know this?
  - Overall shape?

  - Maybe if something lies in this region only it's a one?



- But what if it's shifted?

What If We Had Filters that Could Scan All Parts of An Image?



Overview of our CNN Chapter

- Convolutions
- Feature Detectors
- 3D Conv Filters - Convolution on Color Images
- Kernel Size and Depth
- Padding
- Stride
- Activation Layer - ReLU
- Pooling
- Fully Connected Layer
- Softmax
- Building a CNN
- Parameter Counts in CNNs
- Why CNNs Work so Well For Images
- The Training Process Part 1 - Loss Functions
- The Training Process Part 2 - Back Propagation
- The Training Process Part 3 - Gradient Descent
- Optimisers and Learning Rate Schedules
- Summary of CNNs

Feature Detectors

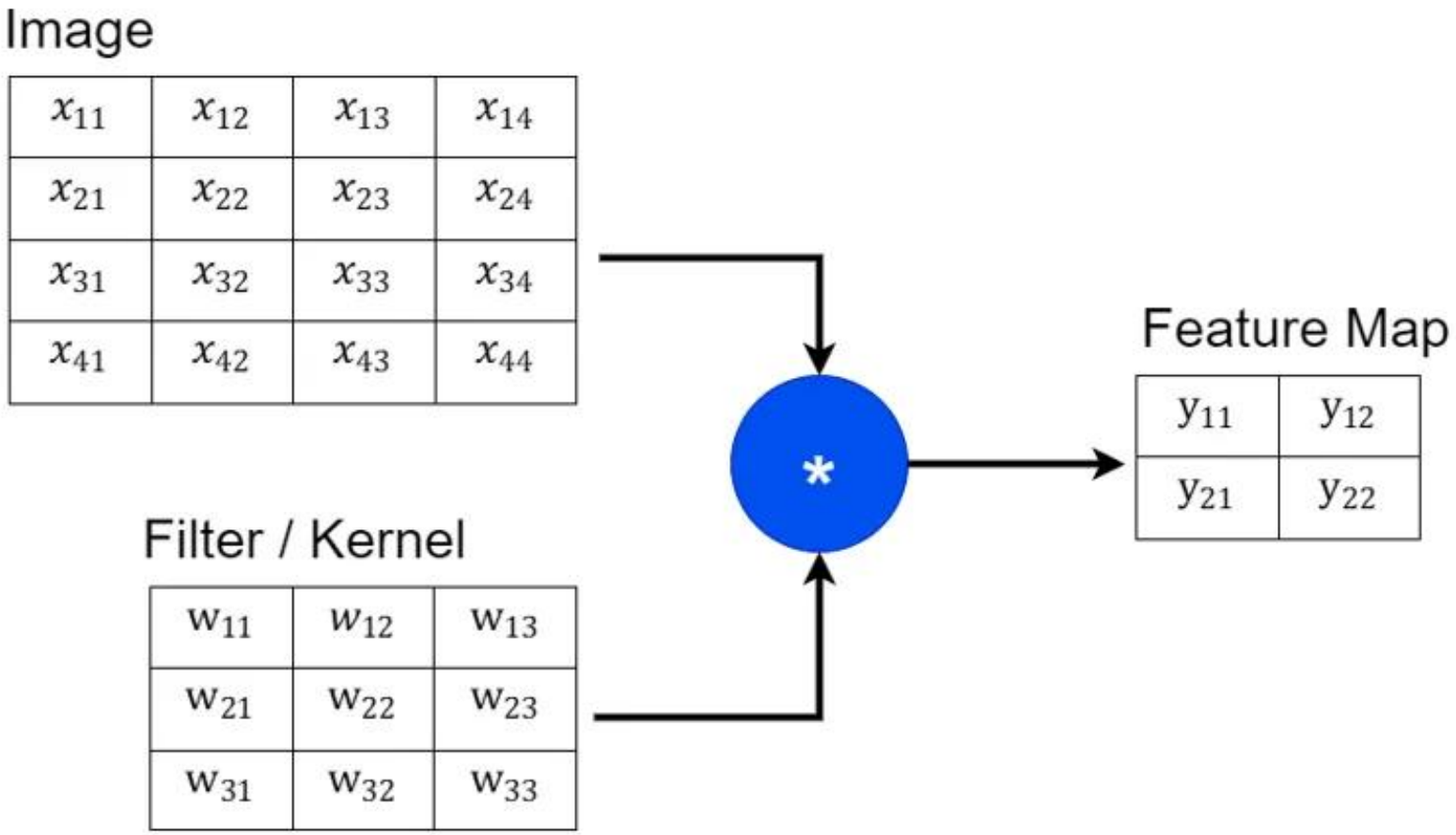
How Feature Detectors Help Classify Images

The Convolution Operation

- Mathematical term to describe the process of combining **two** functions to produce a **third**
- In our situation, the output is called a Feature Map
- We use a matrix, called a Filter or Kernel that is applied to our Image
- So the first 'Function' is the image that is combined with the Kernel or Filter which produces a Feature Map

$$\text{Image} \times \text{Kernel} = \text{Feature Map}$$

The Convolution Operation



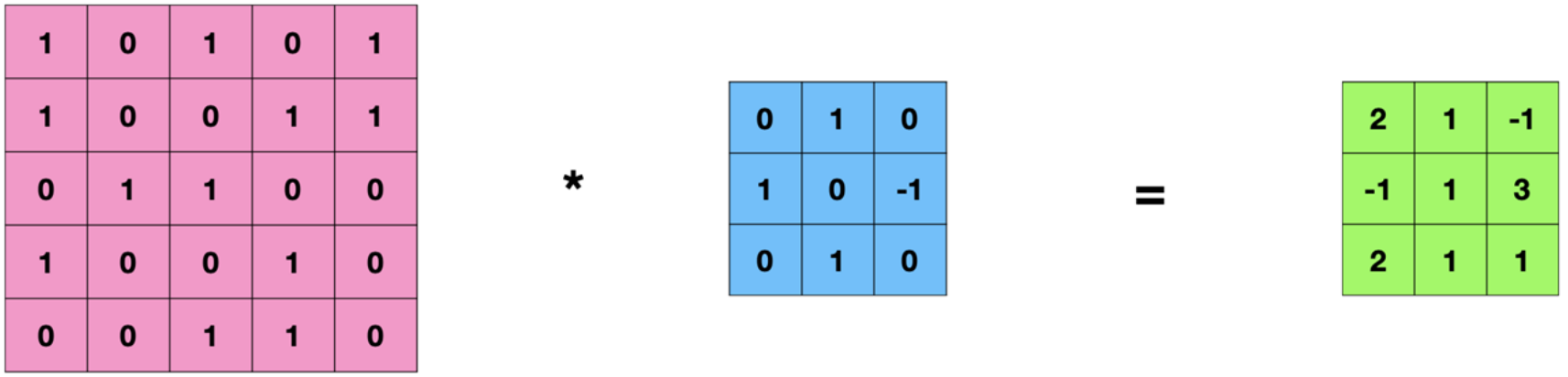
https://medium.com/@lovelyndavid/understanding-the-convolution-operation-in-cnn-with-an-example-636ef8335240

Computation of filter responses using convolution operation

Feature responses	Receptive field	Computation
y_{11}	x_{11} x_{12} x_{13} x_{14} x_{21} x_{22} x_{23} x_{24} x_{31} x_{32} x_{33} x_{34} x_{41} x_{42} x_{43} x_{44}	$x_{11}w_{11} + x_{12}w_{12} + x_{13}w_{13} + x_{21}w_{21} + x_{22}w_{22} + x_{23}w_{23} + x_{31}w_{31} + x_{32}w_{32} + x_{33}w_{33}$
y_{12}	x_{11} x_{12} x_{13} x_{14} x_{21} x_{22} x_{23} x_{24} x_{31} x_{32} x_{33} x_{34} x_{41} x_{42} x_{43} x_{44}	$x_{12}w_{11} + x_{13}w_{12} + x_{14}w_{13} + x_{22}w_{21} + x_{23}w_{22} + x_{24}w_{23} + x_{32}w_{31} + x_{33}w_{32} + x_{34}w_{33}$
y_{21}	x_{11} x_{12} x_{13} x_{14} x_{21} x_{22} x_{23} x_{24} x_{31} x_{32} x_{33} x_{34} x_{41} x_{42} x_{43} x_{44}	$x_{21}w_{11} + x_{22}w_{12} + x_{23}w_{13} + x_{31}w_{21} + x_{32}w_{22} + x_{33}w_{23} + x_{41}w_{31} + x_{42}w_{32} + x_{43}w_{33}$
y_{22}	x_{11} x_{12} x_{13} x_{14} x_{21} x_{22} x_{23} x_{24} x_{31} x_{32} x_{33} x_{34} x_{41} x_{42} x_{43} x_{44}	$x_{22}w_{11} + x_{23}w_{12} + x_{24}w_{13} + x_{32}w_{21} + x_{33}w_{22} + x_{34}w_{23} + x_{42}w_{31} + x_{43}w_{32} + x_{44}w_{33}$

https://medium.com/@lovelyndavid/understanding-the-convolution-operation-in-cnn-with-an-example-636ef8335240

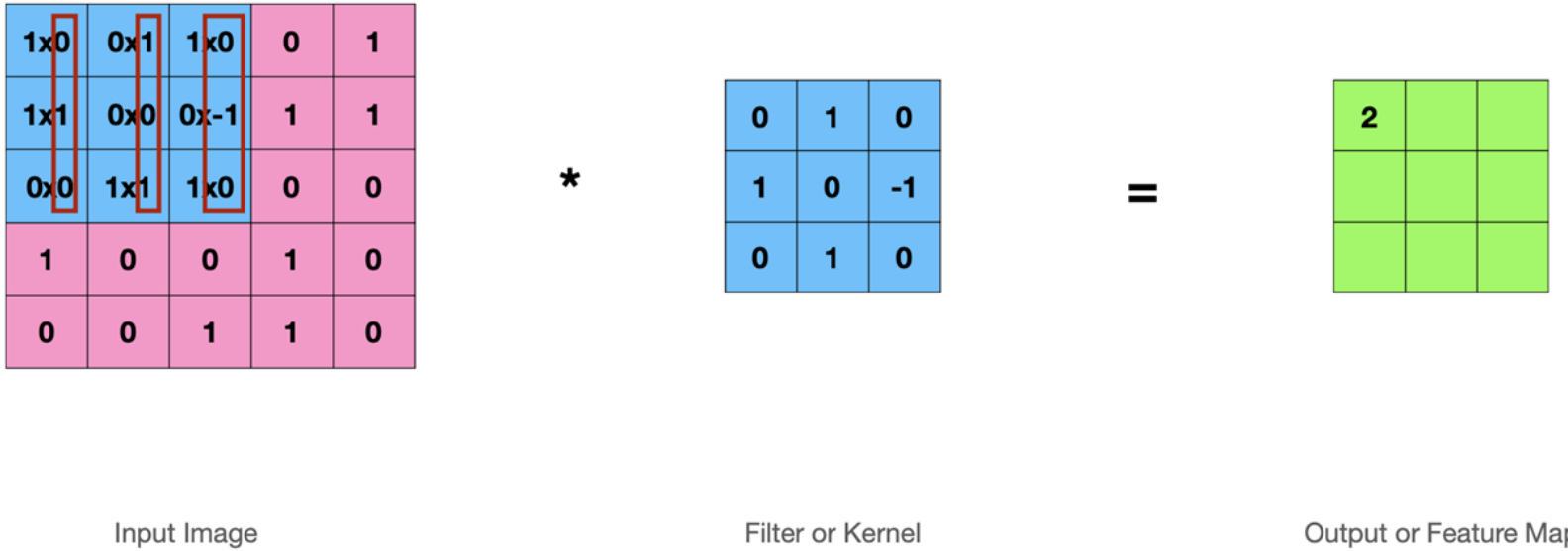
Example of a Convolution Operation



Input Image | Filter or Kernel | Output or Feature Map

Example of a Convolution Operation

$$(1 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times -1) + (0 \times 0) + (1 \times 1) + (1 \times 0) = 2$$



Input Image | Filter or Kernel | Output or Feature Map

Example of a Convolution Operation

$$(0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times -1) + (1 \times 0) + (1 \times 1) + (0 \times 0) = 1$$

1	0x0	1x1	0x0	1
1	0x1	0x0	1x-1	1
0	1x0	1x1	0x0	0
1	0	0	1	0
0	0	1	1	0

Input Image

*

0	1	0
1	0	-1
0	1	0

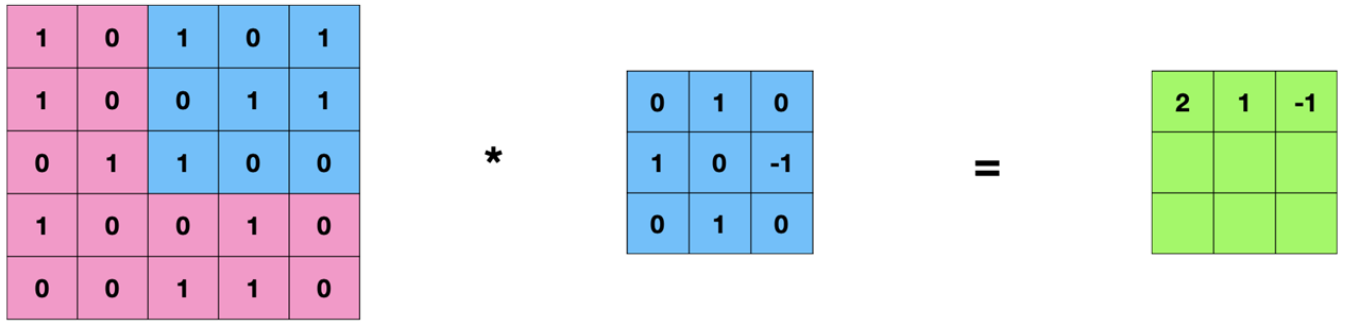
Filter or Kernel

=

2	1	

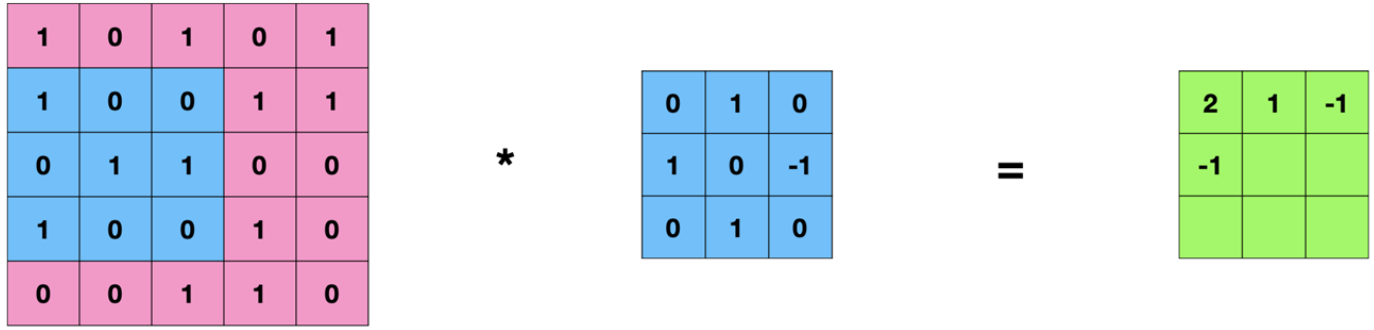
Output or Feature Map

Example of a Convolution Operation

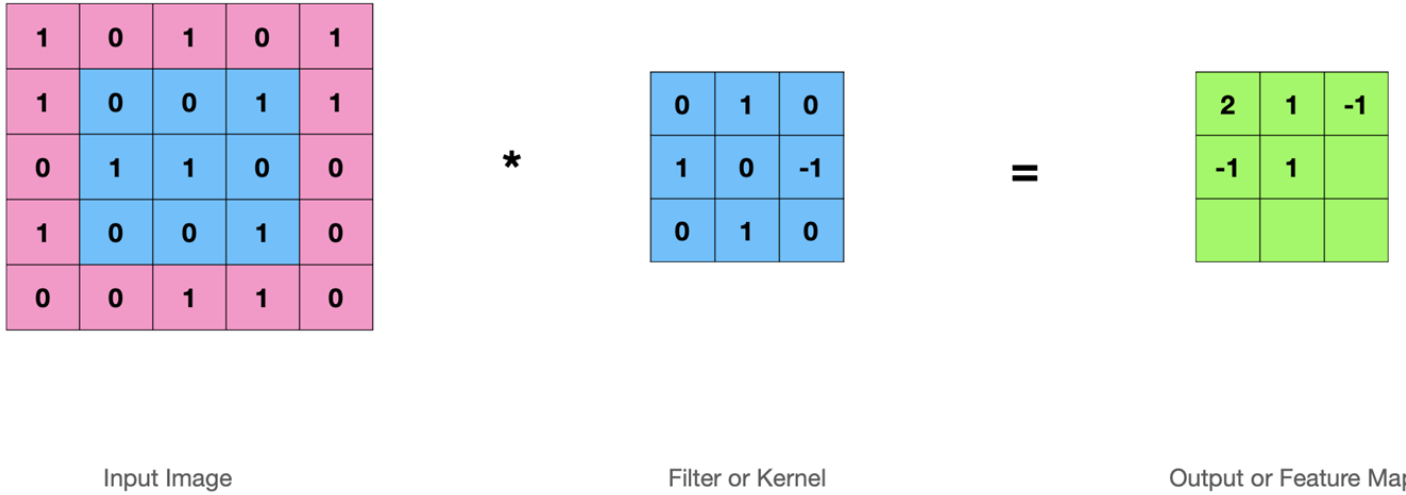


Input Image | Filter or Kernel | Output or Feature Map

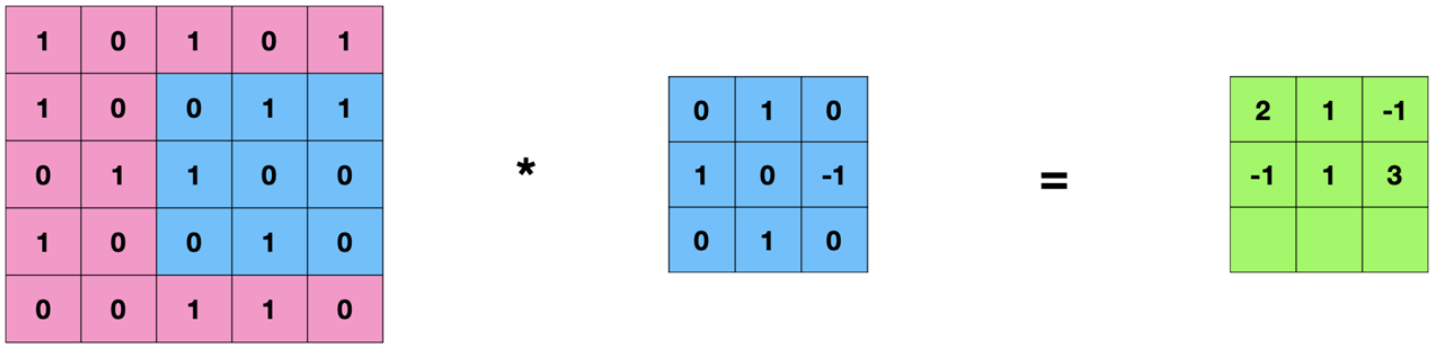
Example of a Convolution Operation



Example of a Convolution Operation

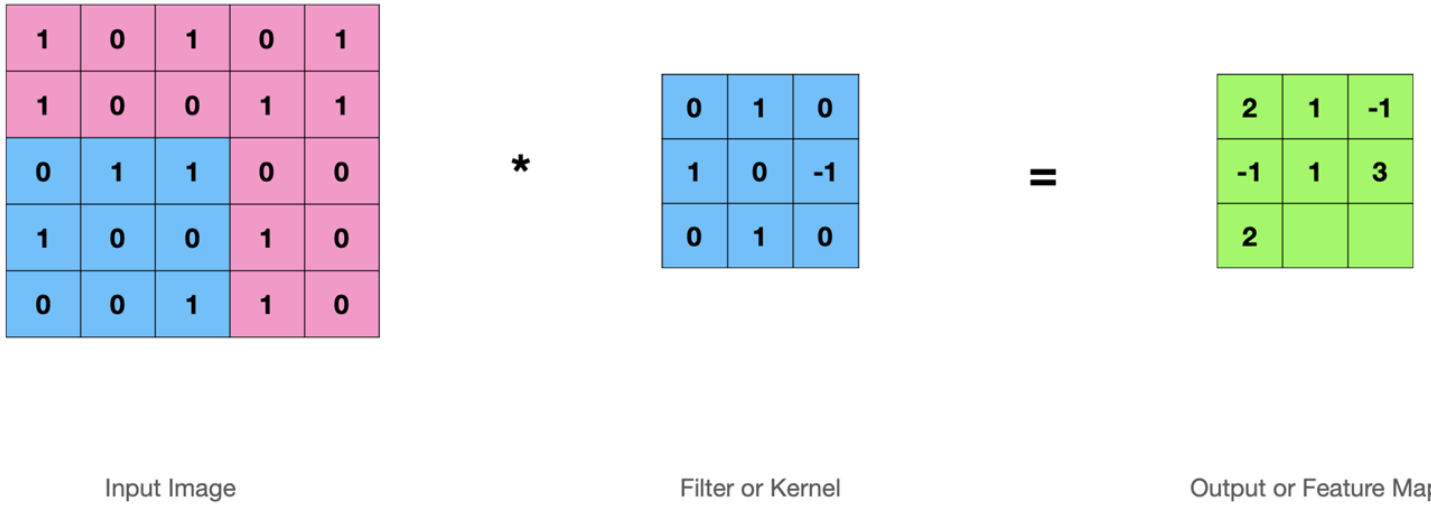


Example of a Convolution Operation

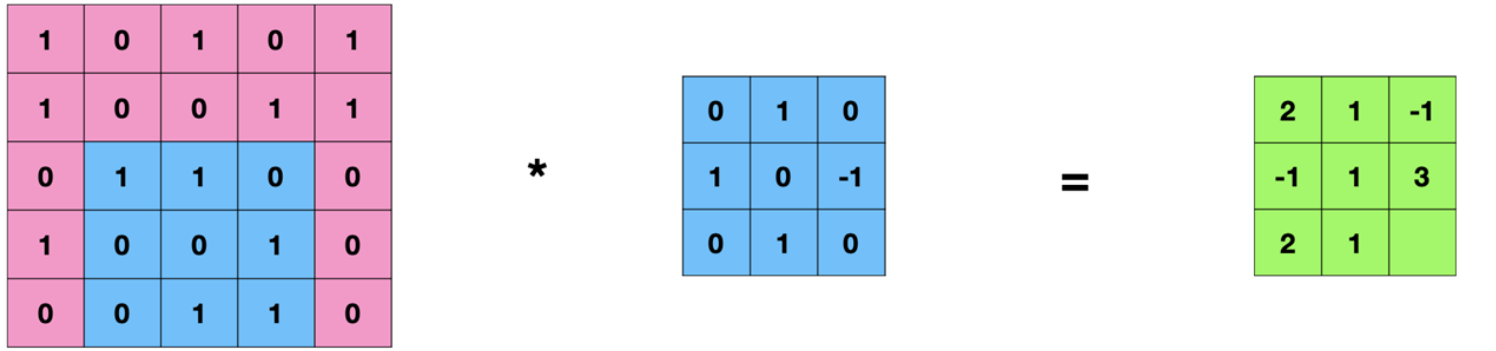


Input Image | Filter or Kernel | Output or Feature Map

Example of a Convolution Operation



Example of a Convolution Operation

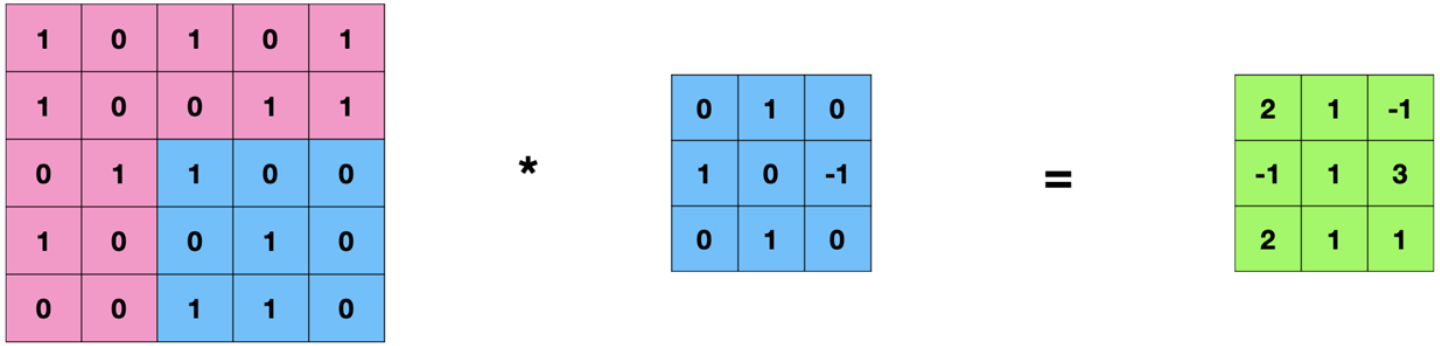


Input Image

Filter or Kernel

Output or Feature Map

Example of a Convolution Operation



Input Image | Filter or Kernel | Output or Feature Map

Image Features

- Our feature maps are actually **Feature Detectors**
- **Why did we do this?**
- Because Convolution Filters or Kernels **detect features** in images

Our Convolution Filter as an Edge Detector

1	1	0	0	0
1	1	0	0	0
1	1	0	0	0
1	1	0	0	0
1	1	0	0	0

Input Image

*

1	0	-1
1	0	-1
1	0	-1

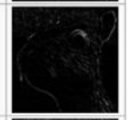
Filter or Kernel

=

3	3	0
3	3	0
3	3	0

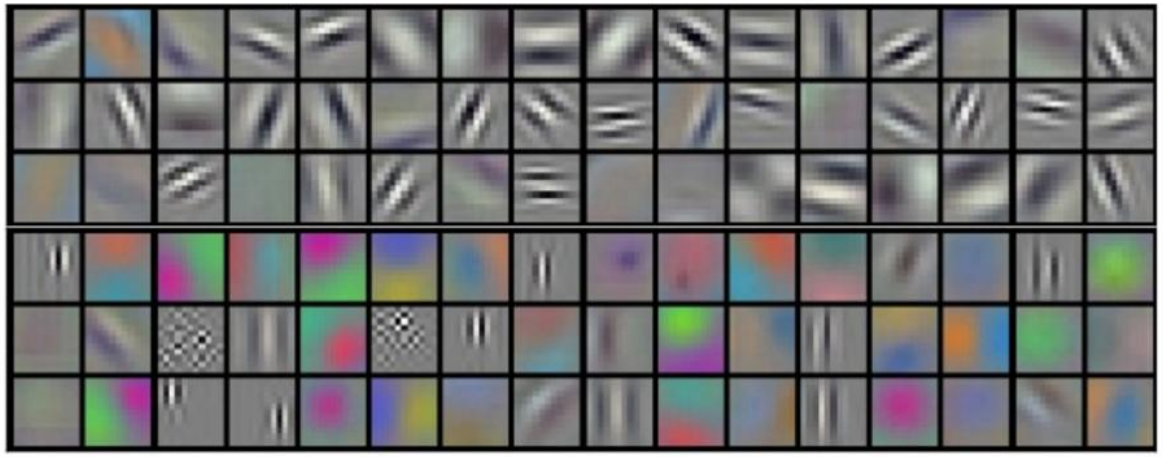
Output or Feature Map

Convolution Filters Detect Many Features

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

https://en.wikipedia.org/wiki/Kernel_(image_processing)

Convolution Filters Detect Many Features



Example filters learned by Krizhevsky et al.

Calculating Feature Map Size

Feature Map Size = $n - f + 1 = m$

Feature Map Size = $5 - 3 + 1 = 3$

1	0	1	0	1
1	0	0	1	1
0	1	1	0	0
1	0	0	1	0
0	0	1	1	0

*

0	1	0
1	0	-1
0	1	0

=

2	1	-1
-1	1	3
2	1	1

5 x 5

$n \times n$

3 x 3

$f \times f$

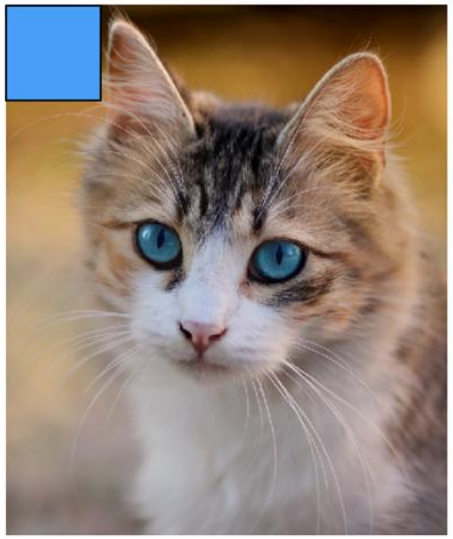
3 x 3

$m \times m$

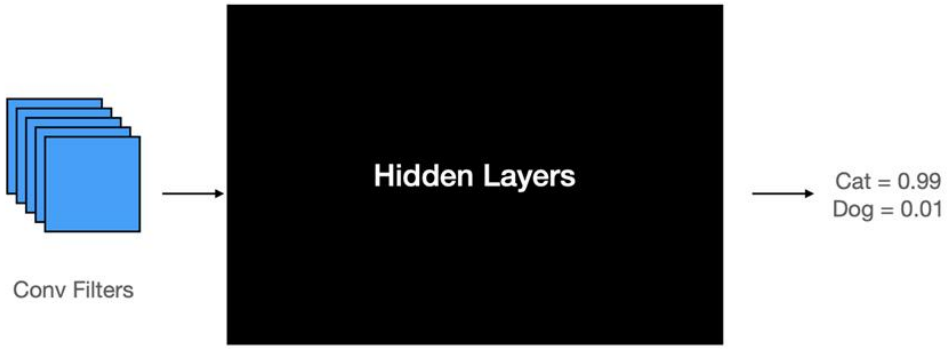
Convolutions

How We Detect Features in Images

How Classifiers Use Feature Detectors

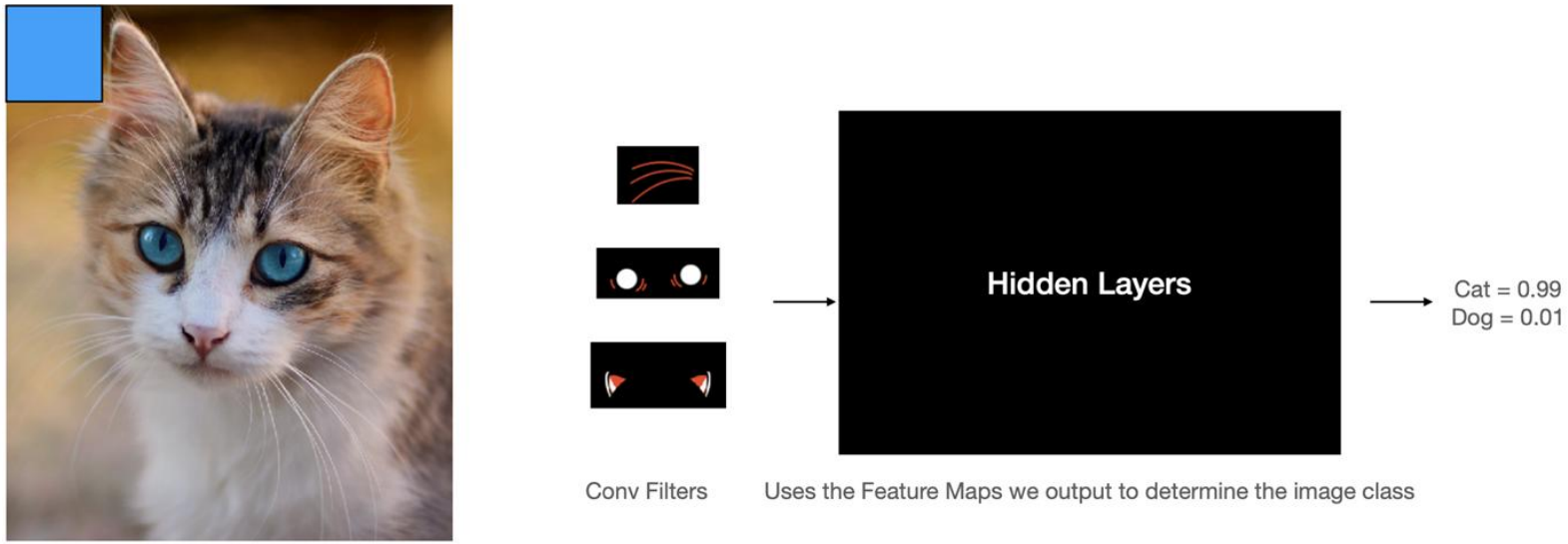


We perform Convolution operations with the input image and our filters



Uses the Feature Maps we output to determine the image class

How Classifiers Use Feature Detectors



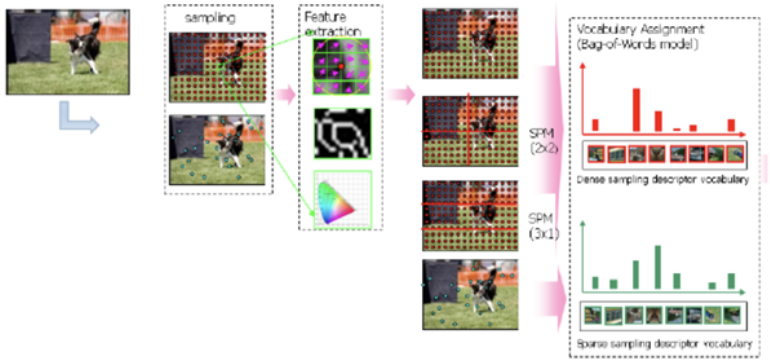
We perform Convolution operations with the input image and our filters

Conv Filters

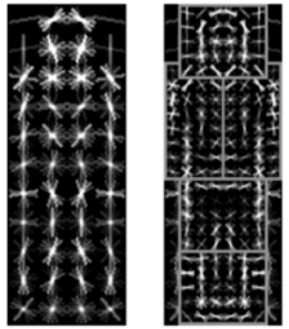
Uses the Feature Maps we output to determine the image class

In the Past Hand Engineered Features Were Often Used

HoGs, SIFT, SURF, ORB, PHOG, Haar etc.



Yan & Huang
(Winner of PASCAL 2010 classification competition)

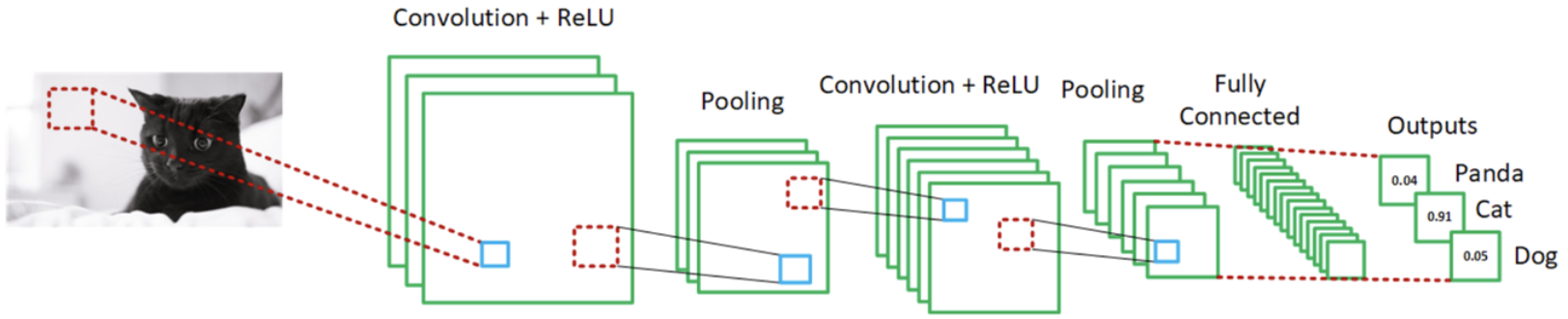


Felzenszwalb, Girshick,
McAllester and Ramanan, PAMI 2007

Hand crafting Features is VERY hard, messy and leads to often poor results…

CNNs solved this by having the ability to Learn Features

The Layers of a CNN



- Convolution
- ReLU
- Pooling Layers
- Fully Connected or Dense Layer
- Output

Convolution on Color Images

How we use 3D Volumes of Convolution Filters

Convolution on our Grey Scale Image

1	1	0	0	0
1	1	0	0	0
1	1	0	0	0
1	1	0	0	0
1	1	0	0	0

Input Image

*

1	0	-1
1	0	-1
1	0	-1

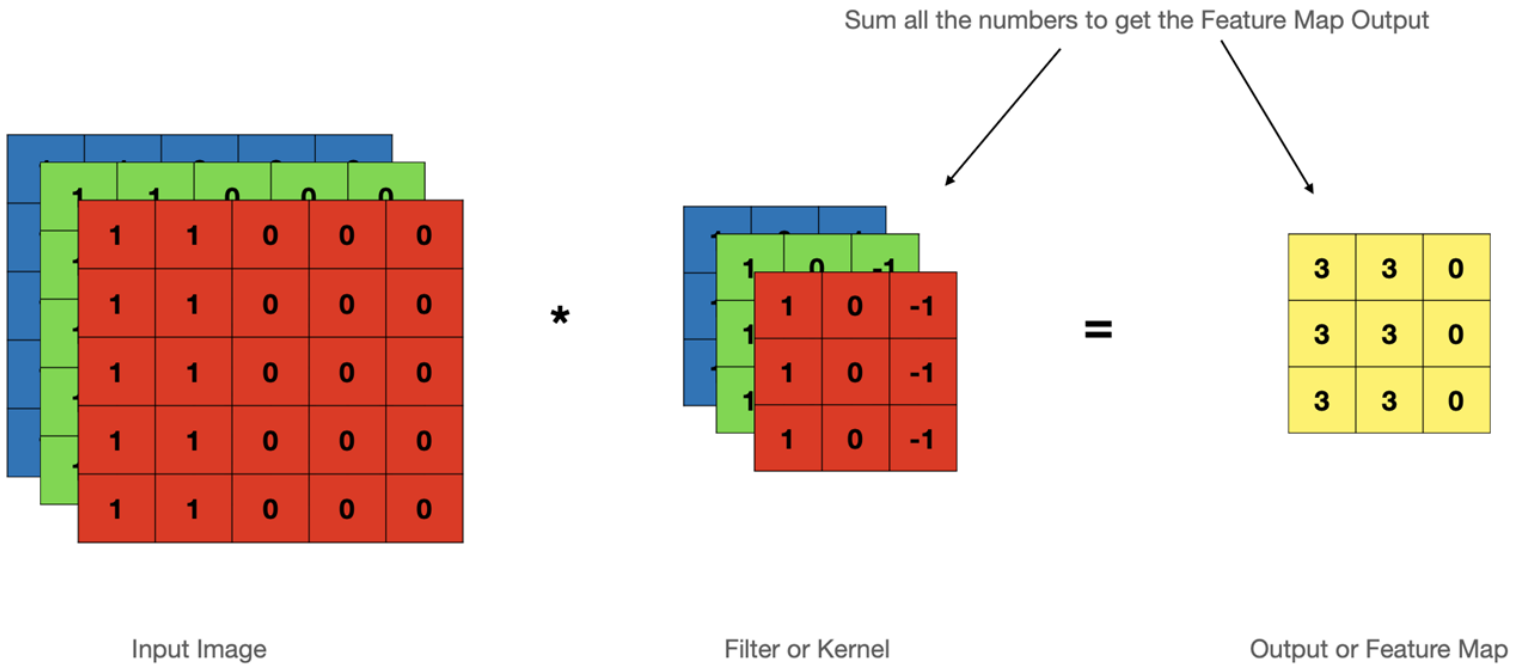
Filter or Kernel

=

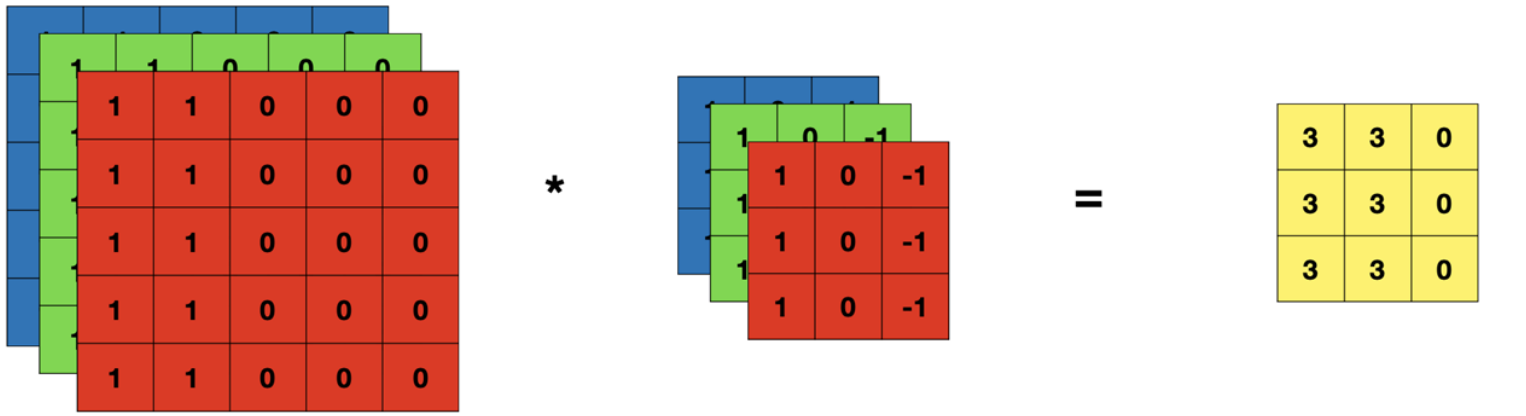
3	3	0
3	3	0
3	3	0

Output or Feature Map

Convolution Operations on Color Images



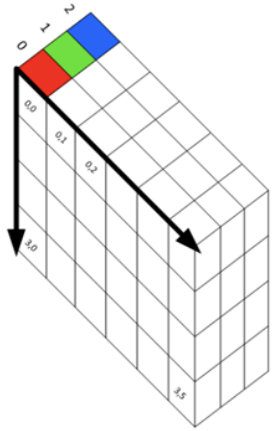
Advantages of Having a Filter For Each Colour



Input Image | Filter or Kernel | Output or Feature Map

- We can detect features that are specific to a colour

Considered 3D Volumes



Input Image
5 x 5 x **3**

*

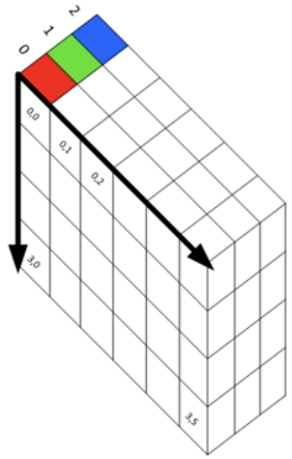
Filter or Kernel
3 x 3 x **3**

=

3	3	0
3	3	0
3	3	0

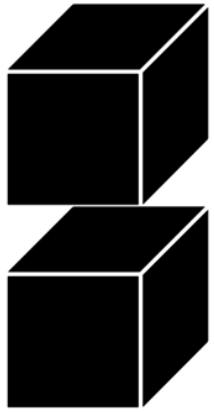
Output or Feature Map
3 x 3

How Multiple Filters Affect Our Output



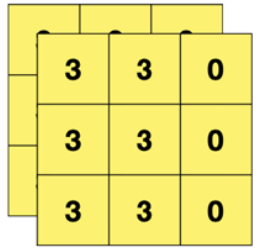
Input Image
5 x 5 x 3

*



2 Filters or Kernels
3 x 3 x 3 x 2

=

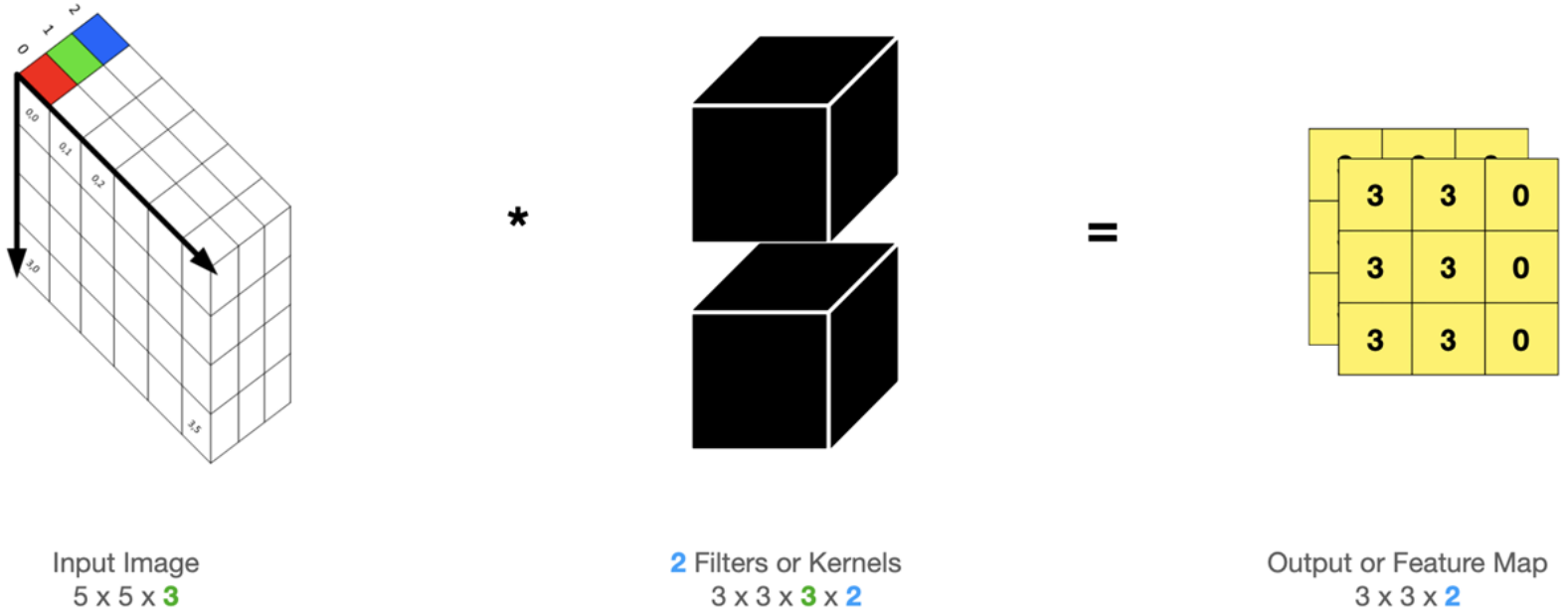


Output or Feature Map
3 x 3 x 2

Calculating Output Size for 3D Conv Volumes

$$(n \times n \times n_c) * (f \times f \times n_c) = (n - f + 1) \times (n - f + 1) \times n_f$$

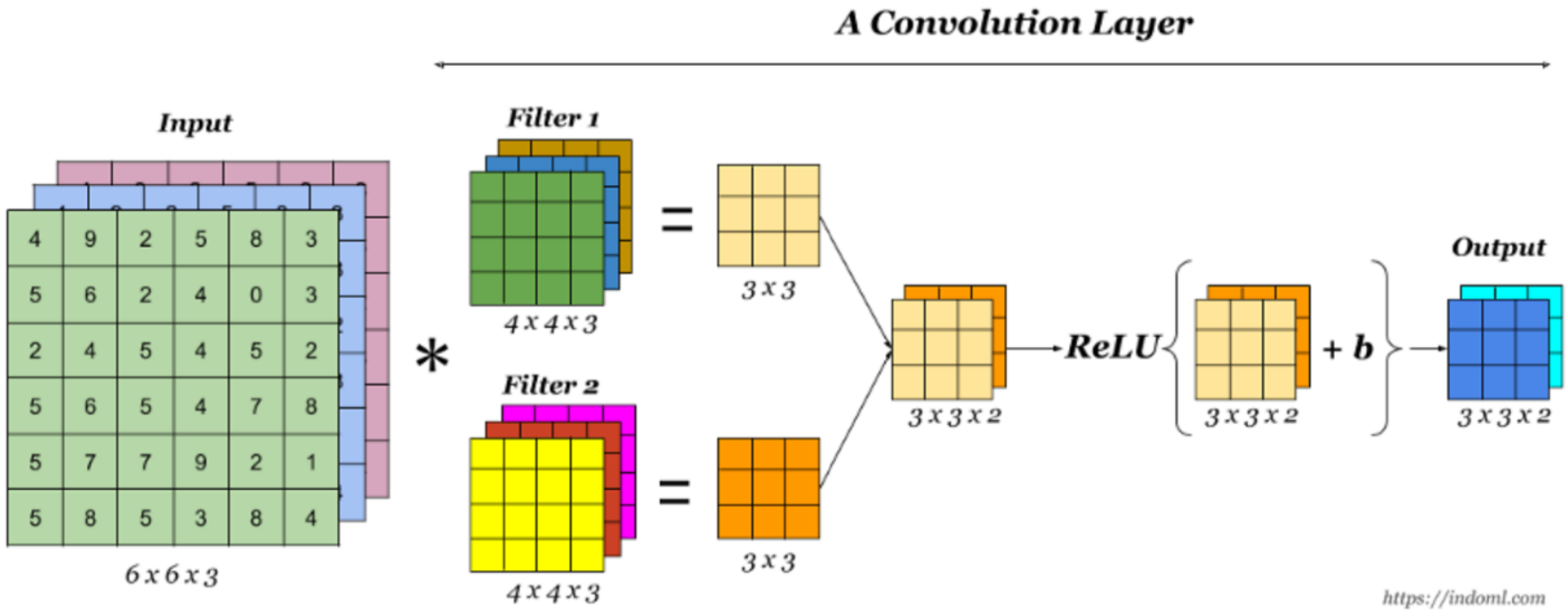
$$(5 \times 5 \times 3) * (3 \times 3 \times 3) = 3 \times 3 \times 2$$



Input Image
5 x 5 x 3

2 Filters or Kernels
3 x 3 x 3 x 2

Output or Feature Map
3 x 3 x 2


A Convolution Layer
Input
4 9 2 5 8 3
5 6 2 4 0 3
2 4 5 4 5 2
5 6 5 4 7 8
5 7 7 9 2 1
5 8 5 3 8 4
6 x 6 x 3
*
Filter 1
4 x 4 x 3
=
3 x 3
Filter 2
4 x 4 x 3
=
3 x 3
3 x 3 x 2
ReLU
3 x 3 x 2
+ b
Output
3 x 3 x 2
https://indoml.com

Kernel Size and Depth

We look at some of parameters that define our Conv Filter

Parameters that Control the Conv Filter

- Kernel Size (k x k)
- Depth (1 for grayscale or 3 for RGB)
- Stride
- Padding

Sizing Convolution Filters

- In the previous example we used a 3 x 3 Filter or Kernel
- Can we use other sizes?

1	1	0	0	0
1	1	0	0	0
1	1	0	0	0
1	1	0	0	0
1	1	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

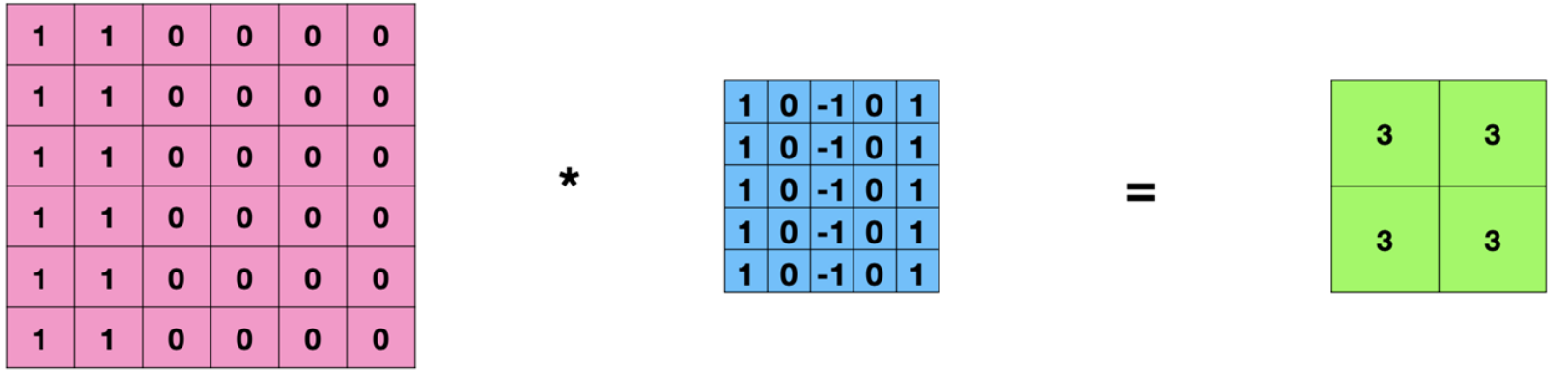
3	3	0
3	3	0
3	3	0

Sizing Convolution Filters

- Yes we can use larger filters/kernels

Feature Map Size = $n - f + 1 = m$

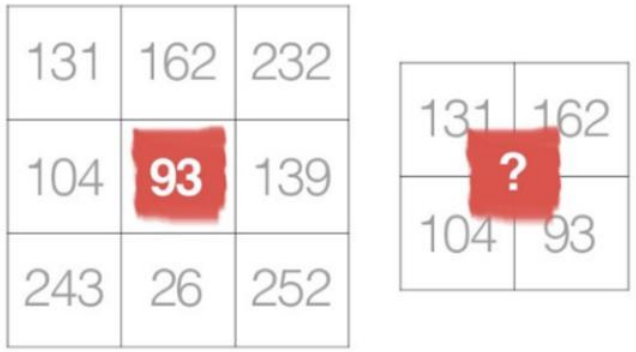
Feature Map Size = $6 - 5 + 1 = 2$



6 x 6 | 5 x 5 | Output or Feature Map

Odd Number vs Even Number for Filter Dimensions

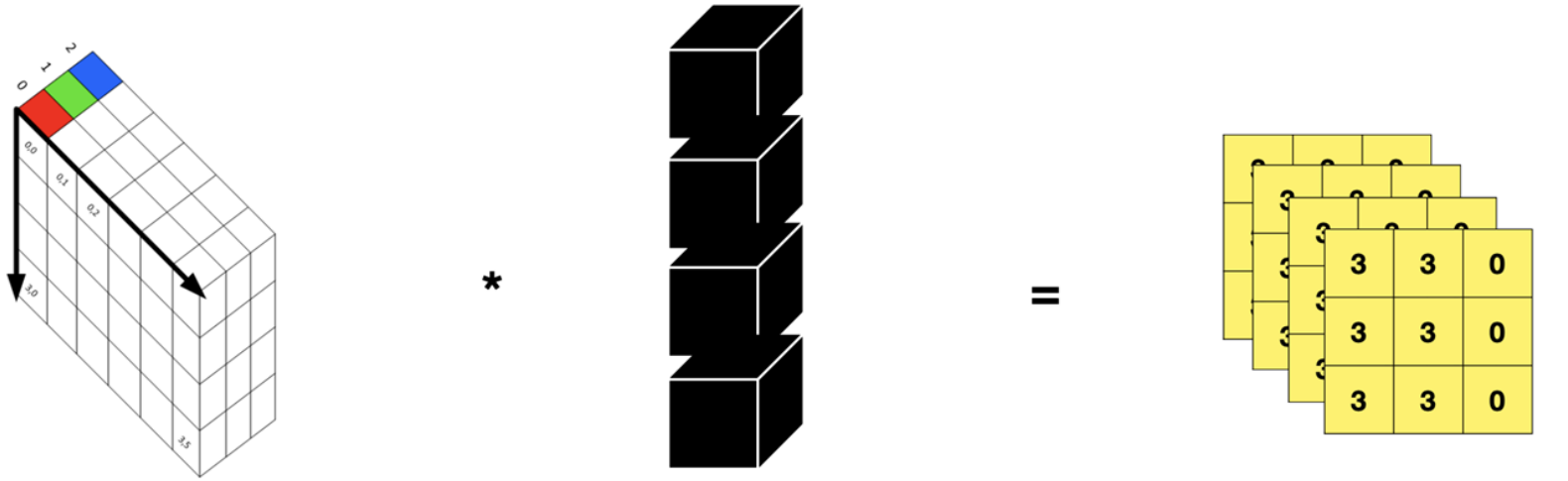
- Odd sized filters are symmetrical around the centre pixel or anchor point
- A lack of symmetry here results in distortions across layers



source:https://towardsdatascience.com/deciding-optimal-filter-size-for-cnns-d6f7b56f9363

Depth

- Depth typically refers to the **colour channels**
- However, in some nomenclature it can refer to the 3rd dimension of any layer in our CNN e.g. our Feature Map has depth of 4



Input Image
5 x 5 x 3

4 Filters or Kernels
3 x 3 x 3 x 4

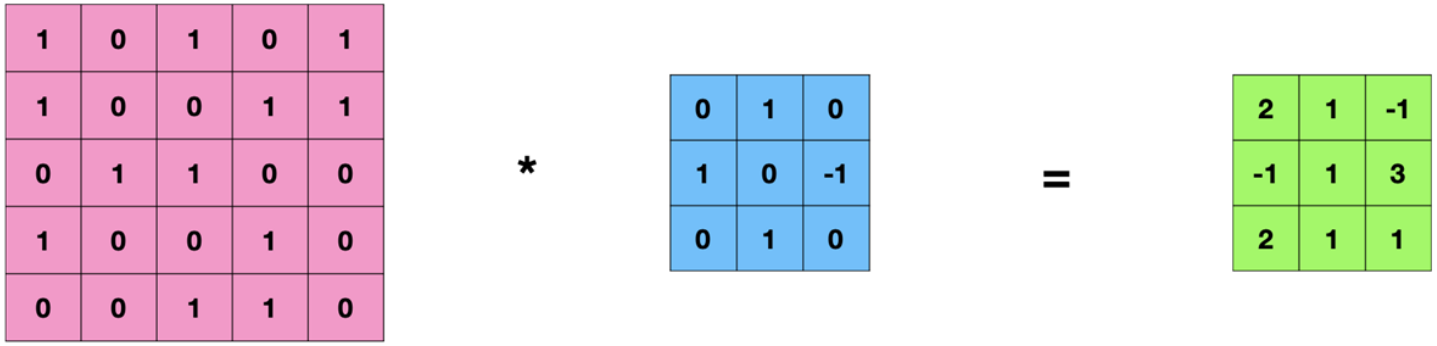
Output or Feature Map
3 x 3 x 4

Padding

Manipulating the input size

Padding

Notice How Conv Filters Produce an Output Smaller than the Input

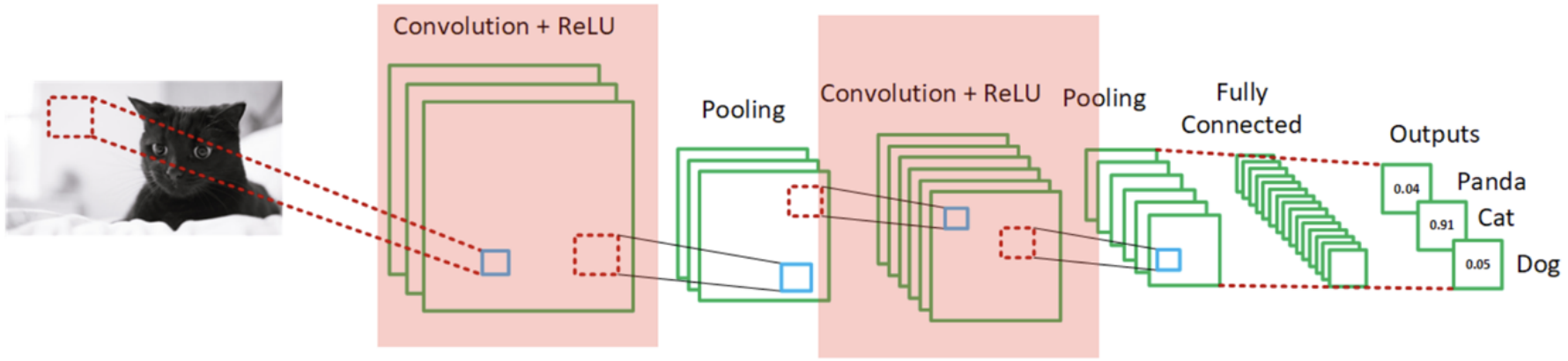


5 x 5
$n \times n$

3 x 3
$f \times f$

3 x 3
$m \times m$

CNNs can have several sequences of Convolution layers



Consecutive Conv layers would keep shrinking the output

Can we preserve our image size?

We've added a 1 pixel pad of zeros (zero padding) around our input

0	0	0	0	0	0	0
0	1	0	1	0	1	0
0	1	0	0	1	1	0
0	0	1	1	0	0	0
0	1	0	0	1	0	0
0	0	0	1	1	0	0
0	0	0	0	0	0	0

*

0	1	0
1	0	-1
0	1	0

=

7 x 7

$n \times n$

3 x 3

$f \times f$

Padding

Let's Perform our Convolution with the Padding

Feature Map Size = $n - f + 1 = m$

Feature Map Size = $7 - 3 + 1 = 5$

0	0	0	0	0	0	0
0	1	0	1	0	1	0
0	1	0	0	1	1	0
0	0	1	1	0	0	0
0	1	0	0	1	0	0
0	0	0	1	1	0	0
0	0	0	0	0	0	0

7 x 7

$n \times n$

*

0	1	0
1	0	-1
0	1	0

3 x 3

$f \times f$

=

2	1	-1	2	2
-1	1	3	2	1
2	1	1	1	2
1	1	1	0	2
2	0	2	3	1

5 x 5

$m \times m$

Why Use Padding?

- For very deep networks we don't want to keep reducing the size
- Pixels at the edges contribute less to the output Feature Maps, thus we're throwing away information from them

1	0	1	0	1
1	0	0	1	1
0	1	1	0	0
1	0	0	1	0
0	0	1	1	0

Without padding, our top left pixel is only touched by the Conv Filter once

Whereas, our centre pixel is passed over numerous times

Stride

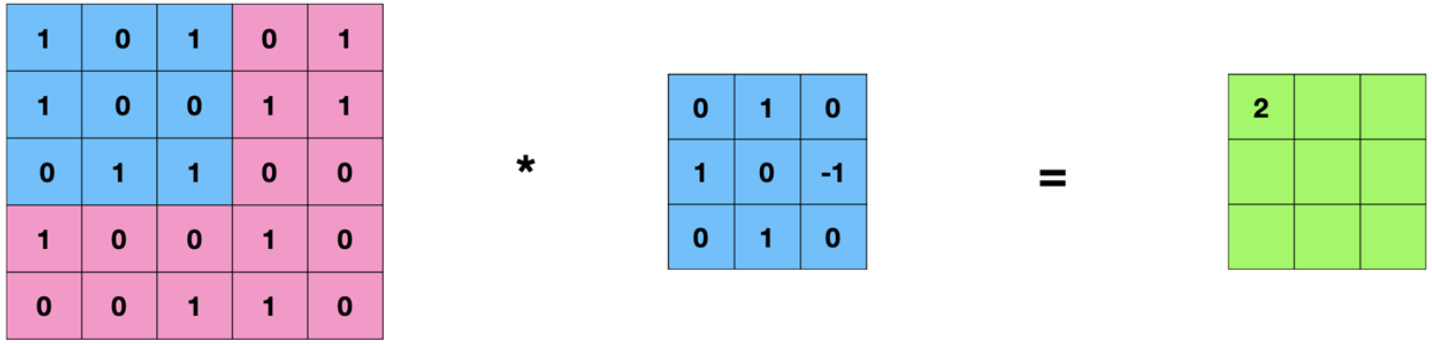
We look at a parameter that defines how we move our Conv Filter

Stride

Our Step Size

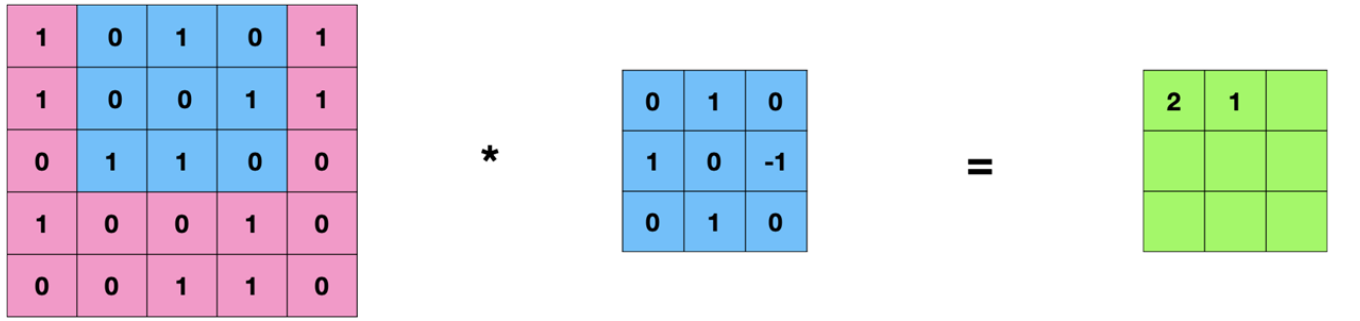
- Stride defines how many steps we take when sliding our Convolution Window across the input image

What a Stride of 1 Looks Like

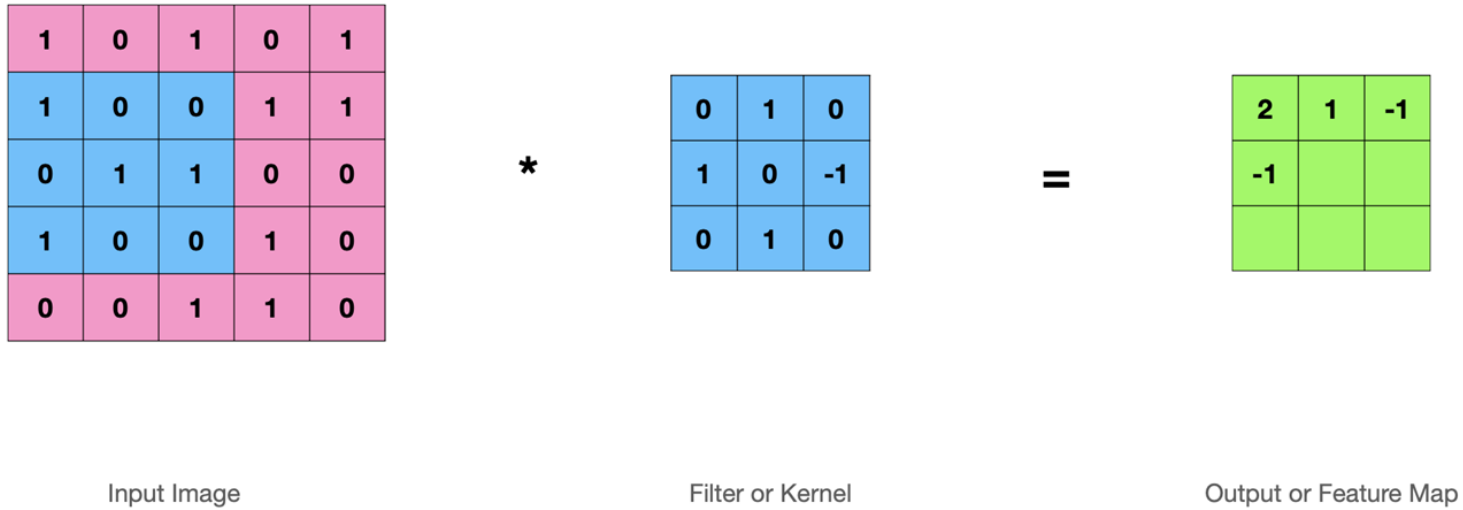


Input Image | Filter or Kernel | Output or Feature Map

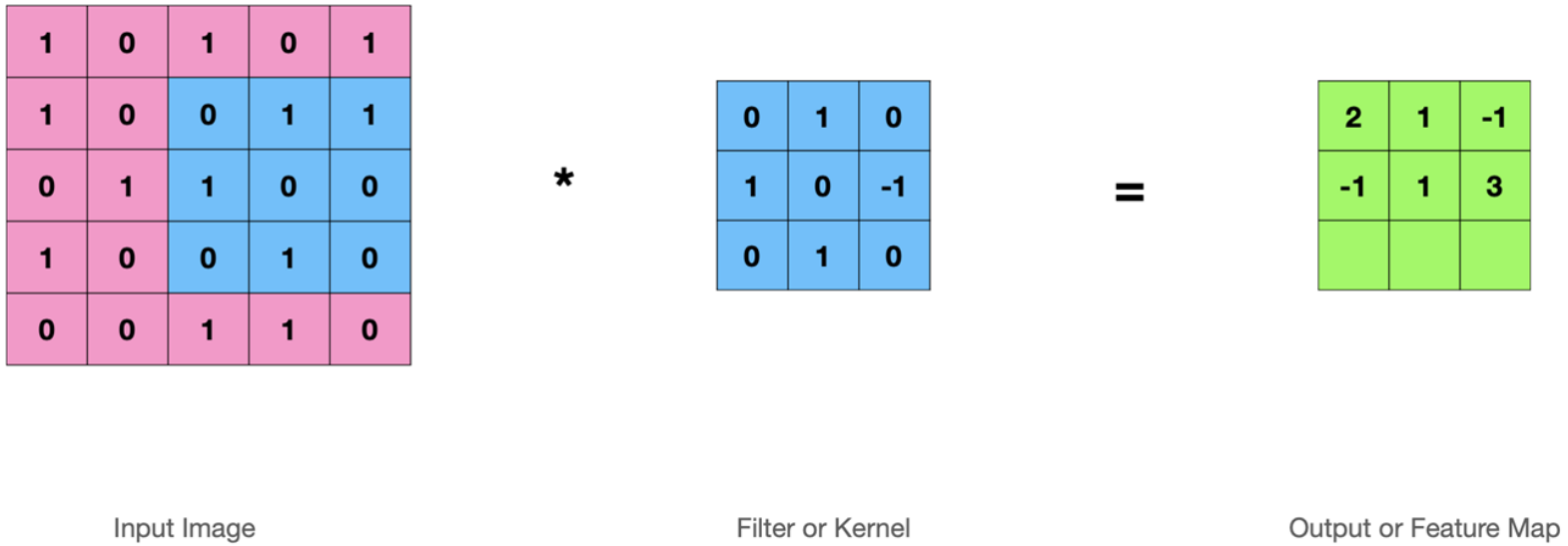
What a Stride of 1 Looks Like



What a Stride of 1 Looks Like

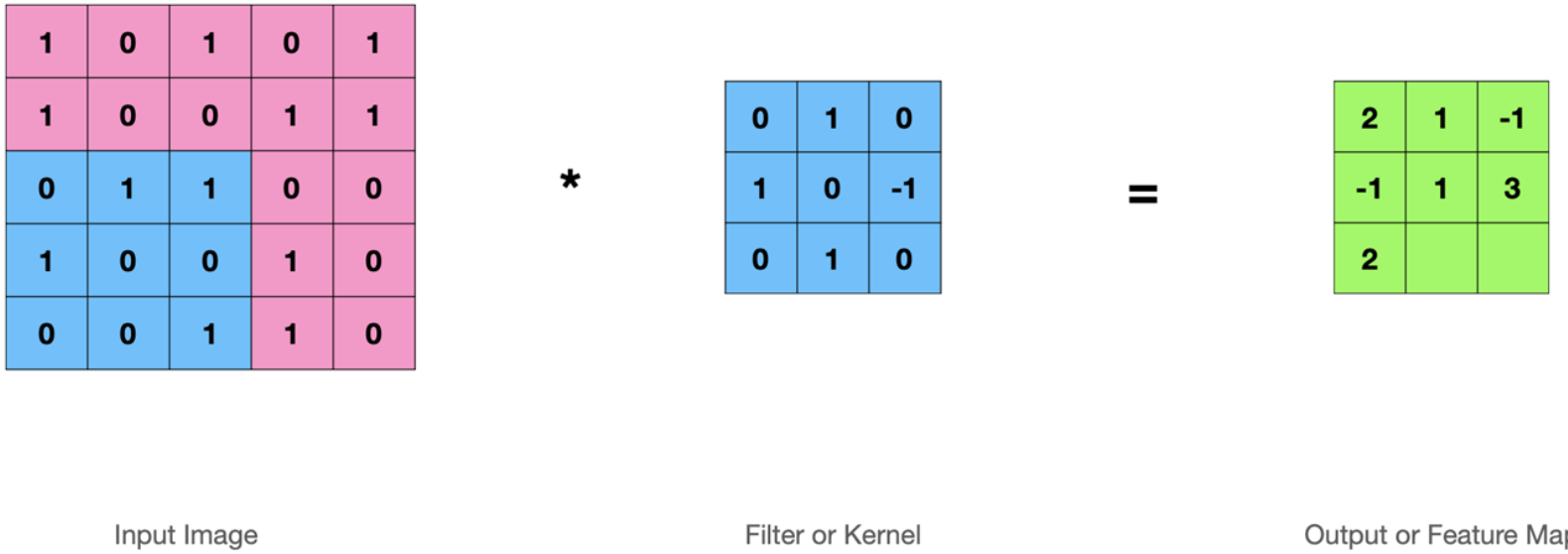


What a Stride of 1 Looks Like



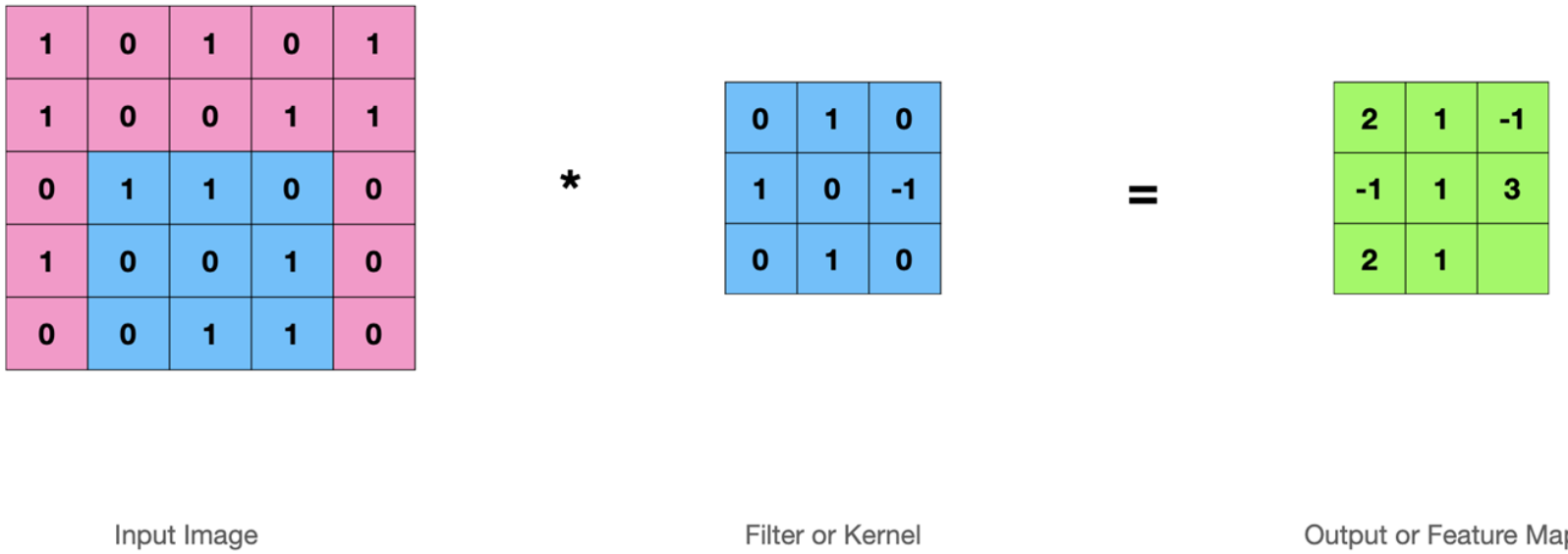
Input Image | Filter or Kernel | Output or Feature Map

What a Stride of 1 Looks Like

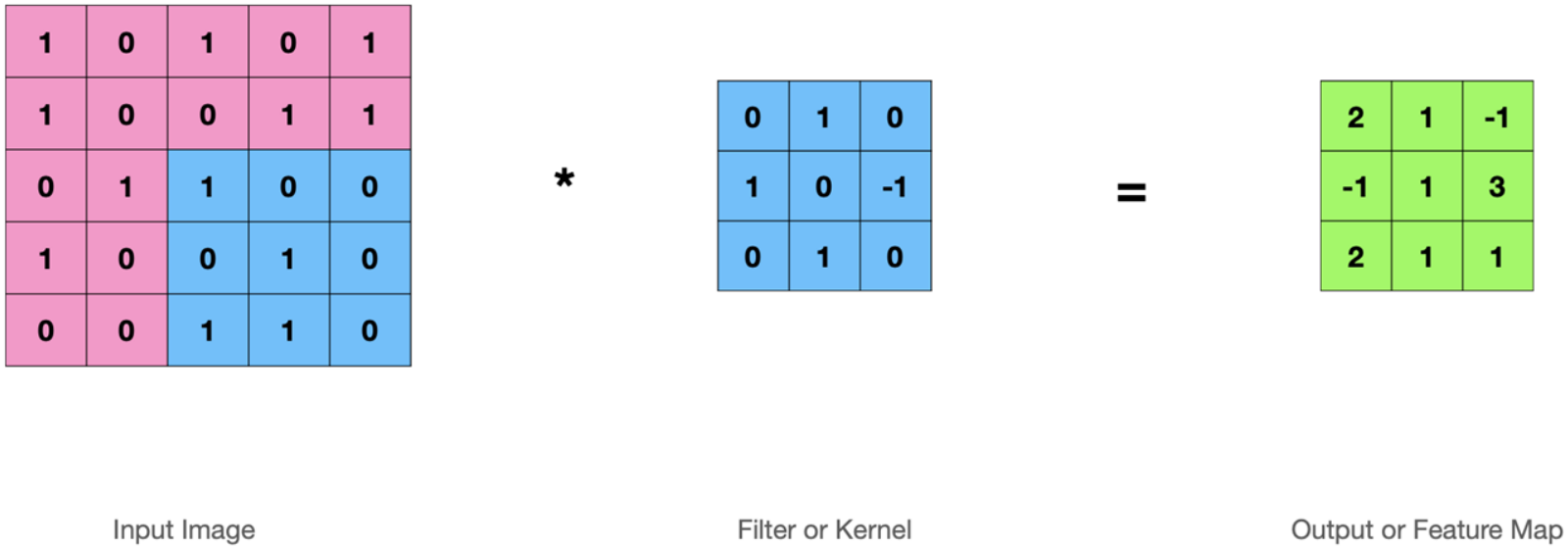


Input Image | Filter or Kernel | Output or Feature Map

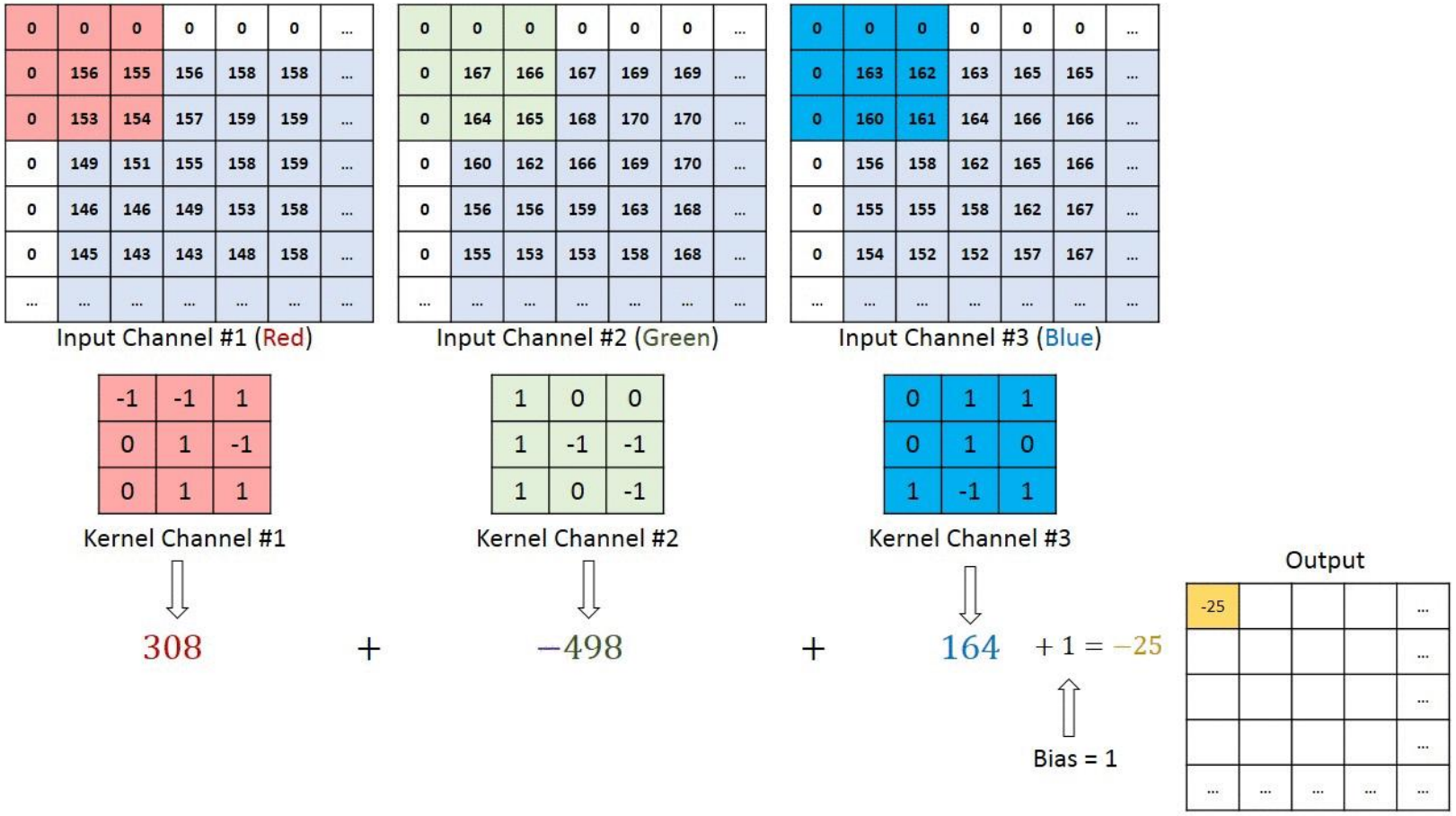
What a Stride of 1 Looks Like



What a Stride of 1 Looks Like



Convolution operation on a MxNx3 image matrix with a 3x3x3 Kernel, padding of 1, and stride of 1

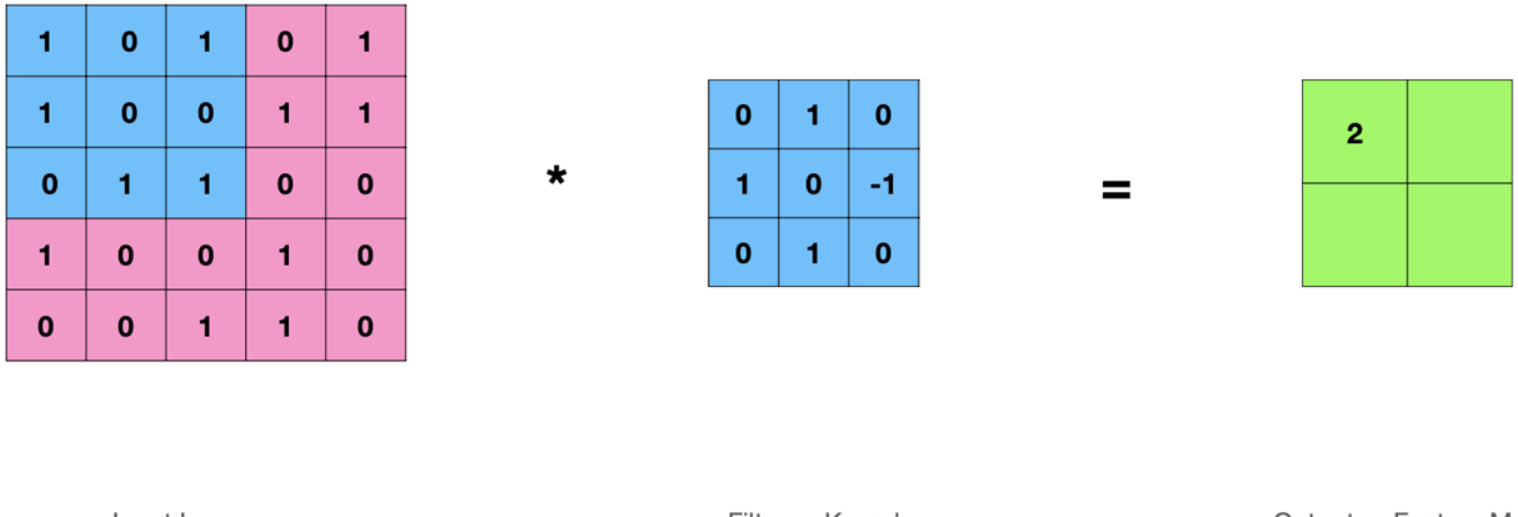


https://medium.com/towards-data-science/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

What about a Stride of 2?

What a Stride of 2 Looks Like

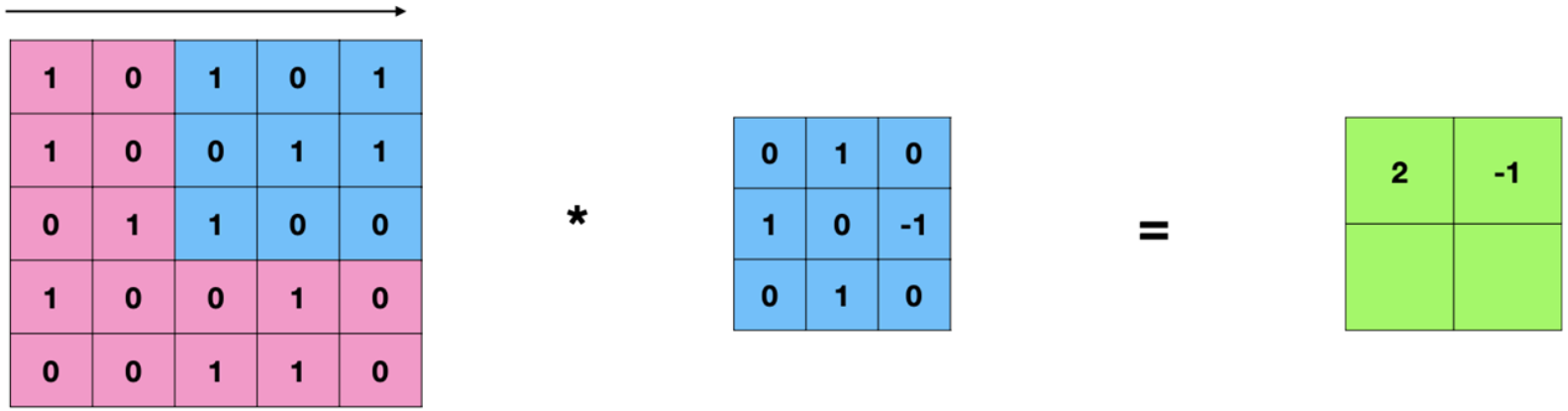
We start off in the same position



Input Image | Filter or Kernel | Output or Feature Map

What a Stride of 2 Looks Like

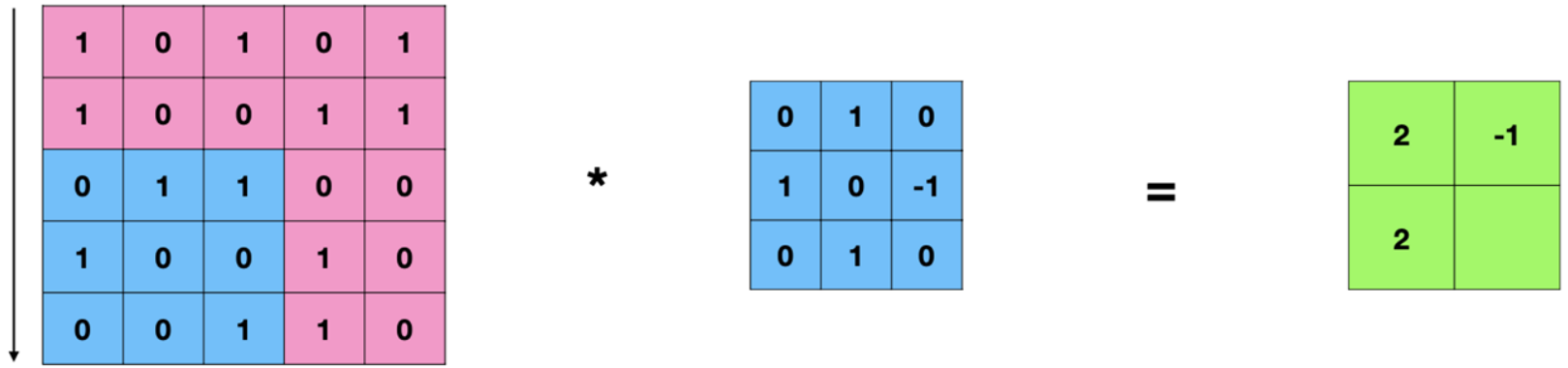
Now we jump two spots to the left



Input Image | Filter or Kernel | Output or Feature Map

What a Stride of 2 Looks Like

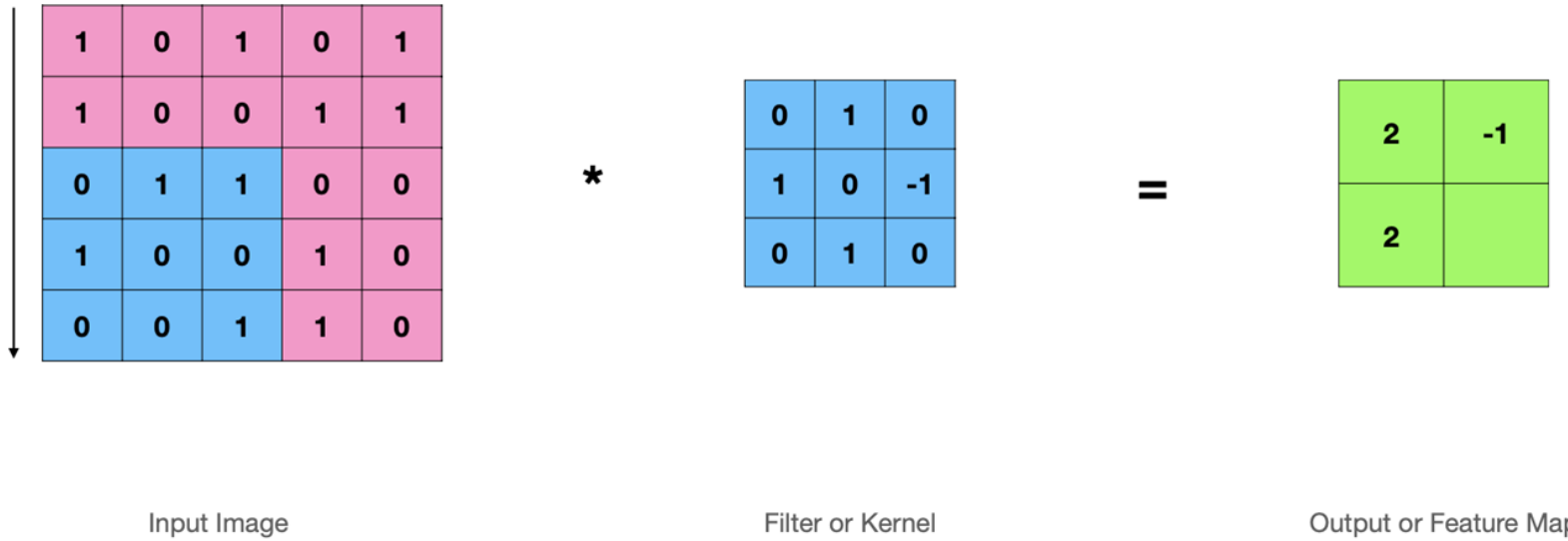
Now we go down, but by also 2 spots



Input Image | Filter or Kernel | Output or Feature Map

What a Stride of 2 Looks Like

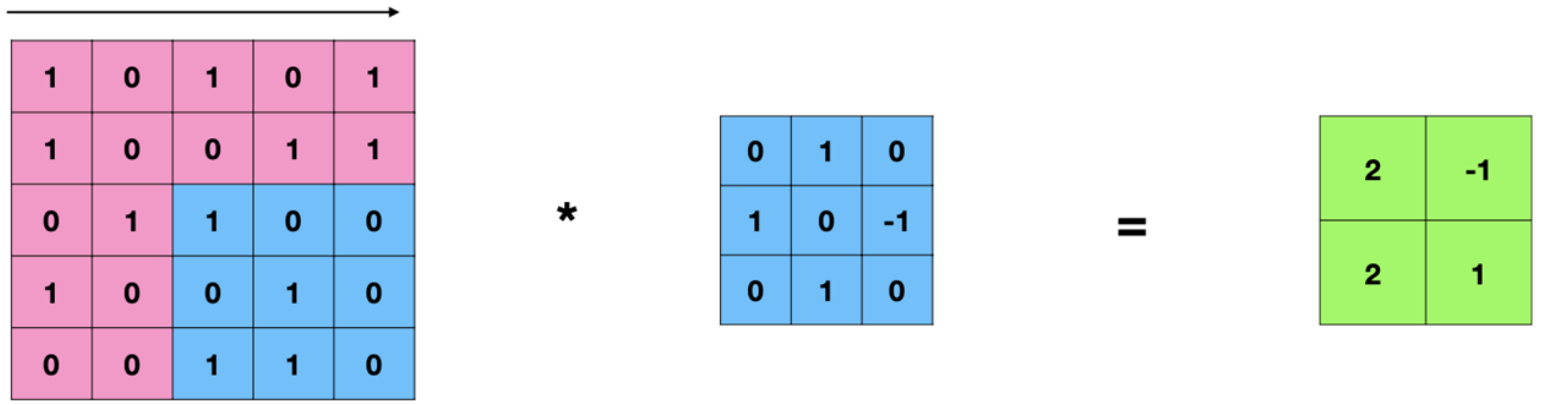
Now we go down, but by also 2 spots



Input Image | Filter or Kernel | Output or Feature Map

What a Stride of 2 Looks Like

Now we jump two spots to the left



Input Image | Filter or Kernel | Output or Feature Map

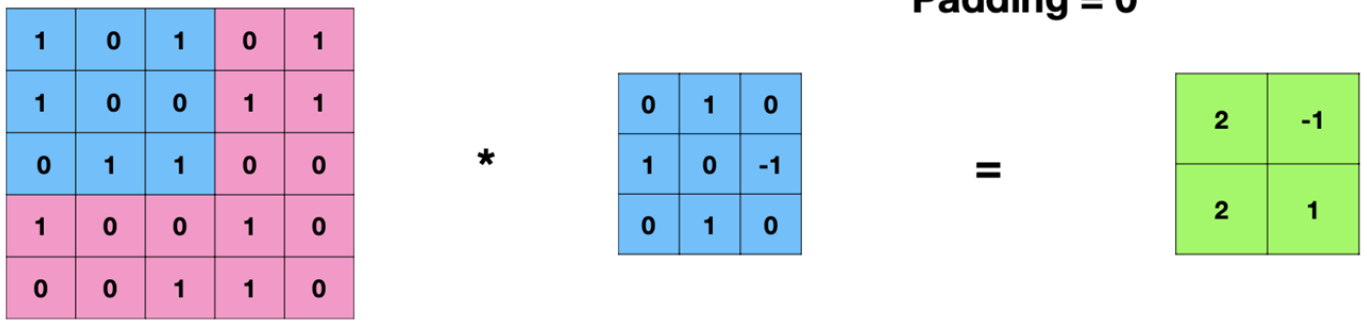
Stride Observations

- A larger Stride produced a **smaller** Feature Map output
- Larger Stride has **less overlap**
- We can use stride to **control the size of the Feature Map output**

Calculating Output Size

Using Stride and Padding

Stride = 2
Padding = 0



Input Image
5 x 5

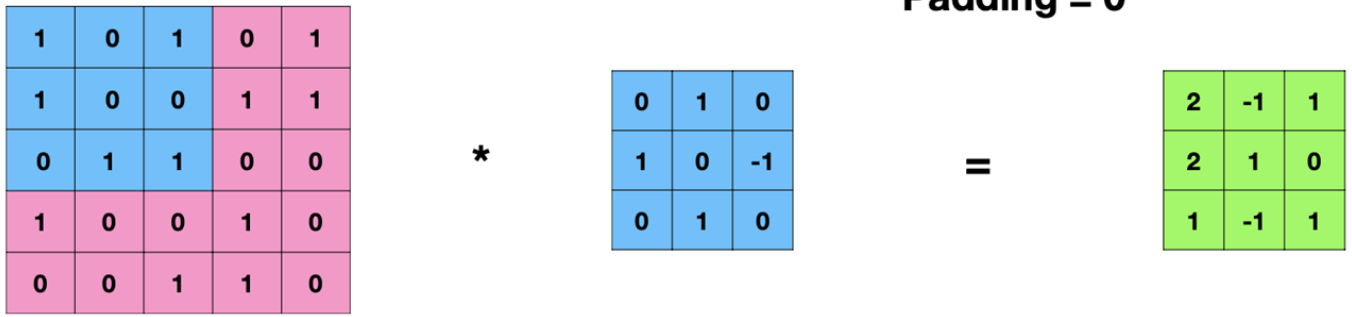
Filter or Kernel
3 x 3

$$(n \times n) * (f \times f) = (\frac{n+2p-f}{s}+1) \times (\frac{n+2p-f}{s}+1) = (\frac{5+(2\times 0)-3}{2}+1) \times (\frac{5+(2\times 0)-3}{2}+1) = 2 \times 2$$

Calculating Output Size

Using Stride and Padding

Stride = 1
Padding = 0



Input Image
5 x 5

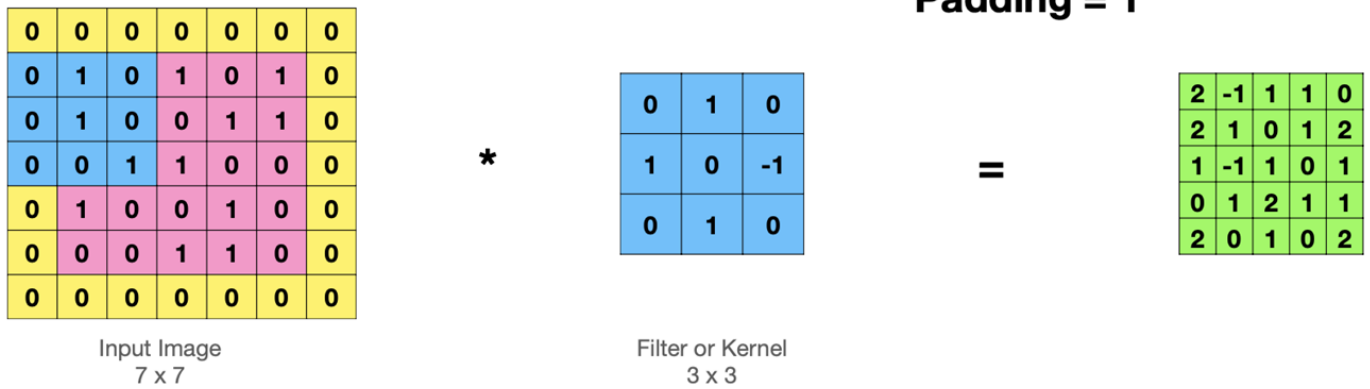
Filter or Kernel
3 x 3

$$(n \times n) * (f \times f) = (\frac{n+2p-f}{s}+1) \times (\frac{n+2p-f}{s}+1) = (\frac{5+(2\times 0)-3}{1}+1) \times (\frac{5+(2\times 0)-3}{1}+1) = 3 \times 3$$

Calculating Output Size

When using Stride & Padding

Stride = 1
Padding = 1



Input Image
7 x 7

Filter or Kernel
3 x 3

$$(n \times n) * (f \times f) = (\frac{n+2p-f}{s}+1) \times (\frac{n+2p-f}{s}+1) = (\frac{5+(2\times 1)-3}{1}+1) \times (\frac{5+(2\times 1)-3}{1}+1) = 5 \times 5$$

Note if we get non integer value for our output size, we found it down to the nearest integer

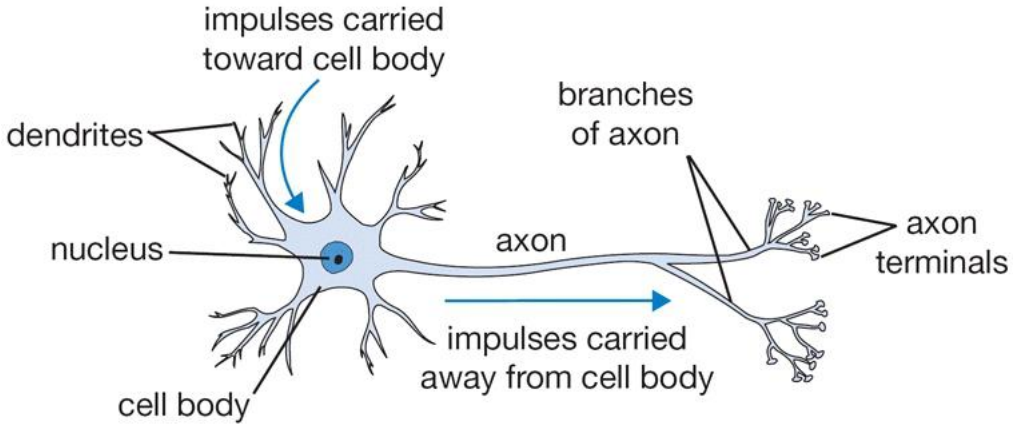
Activation Layer ReLU

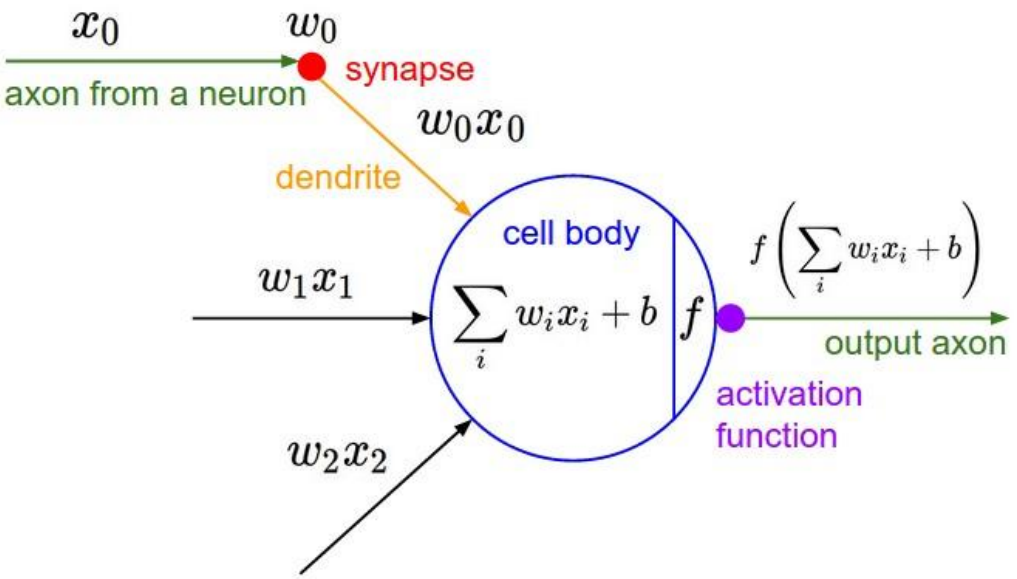
What are Activation Functions and their importance

Purpose of Activation Functions

To enable the learning of **complex patterns** in our data

- Biological neurons fire (activate) on certain inputs, these are then fed into other neurons
- Introduces **non-linearity** to our network
- This allows a non-linear decision boundary via non-linear combinations of the weight and inputs





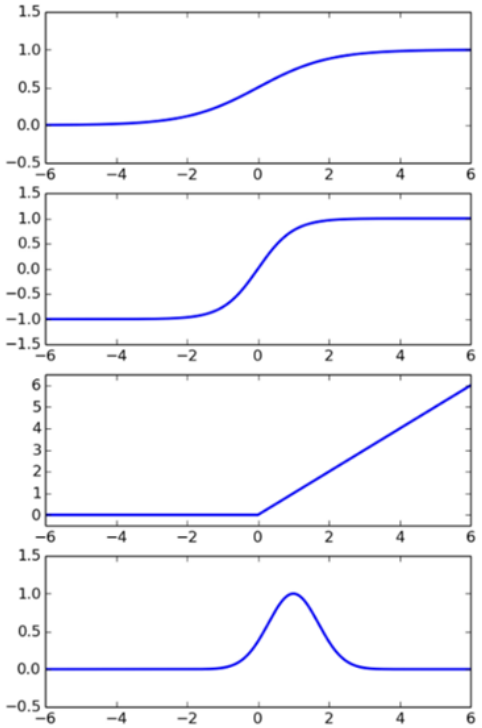
Stanford's CS231 Course

Types of Activation Functions

There are several activation functions we can use in our CNN. However, Rectified Linear Units (ReLU) have become the activation function of choice for CNNs.

ReLU is advantageous in CNN Training:

- Simple Computation (fast to train)
- Does not saturate



Sigmoid

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

Hyperbolic Tangent

$$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Rectified Linear

$$\phi(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

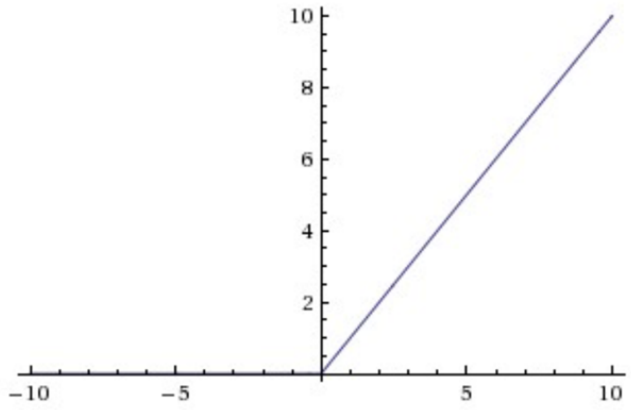
Radial Basis Function

$$\phi(z, c) = e^{-(\epsilon \|z - c\|)^2}$$

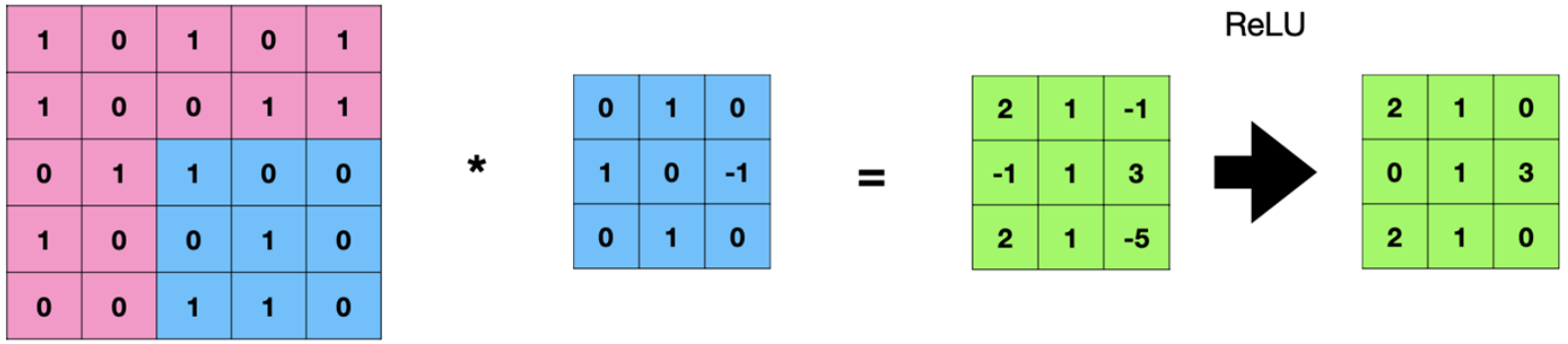
The ReLU Operation

- Change all negative values to 0
- Leave all positive Values alone

$$f(x) = max(0,x)$$

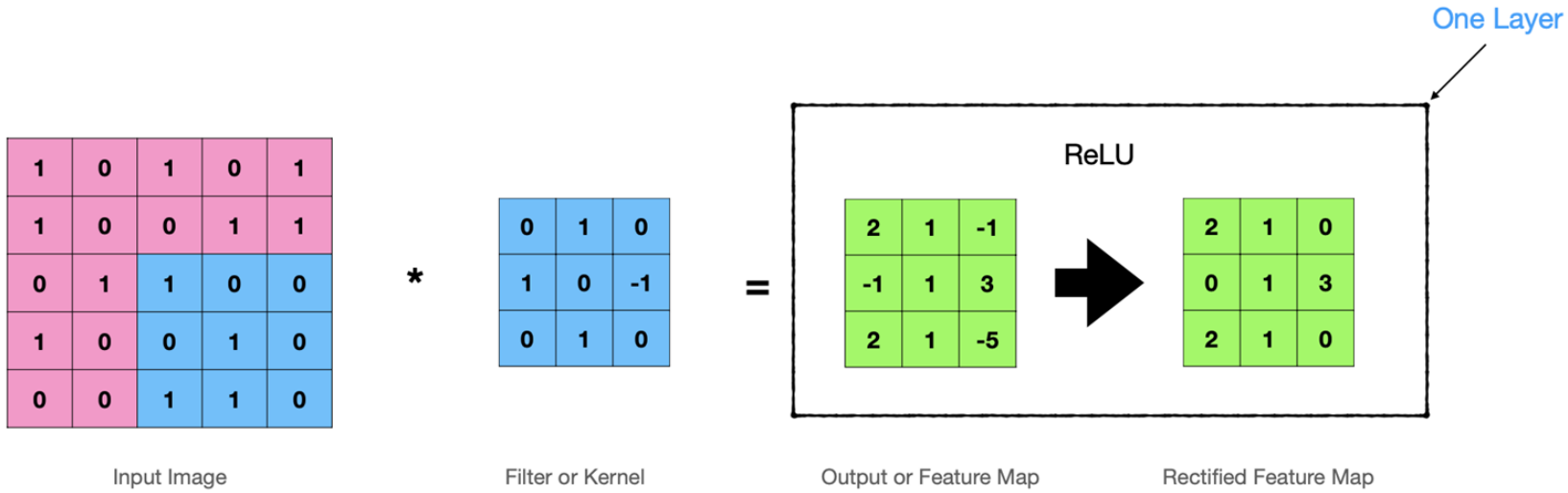


Applying the ReLU Activation

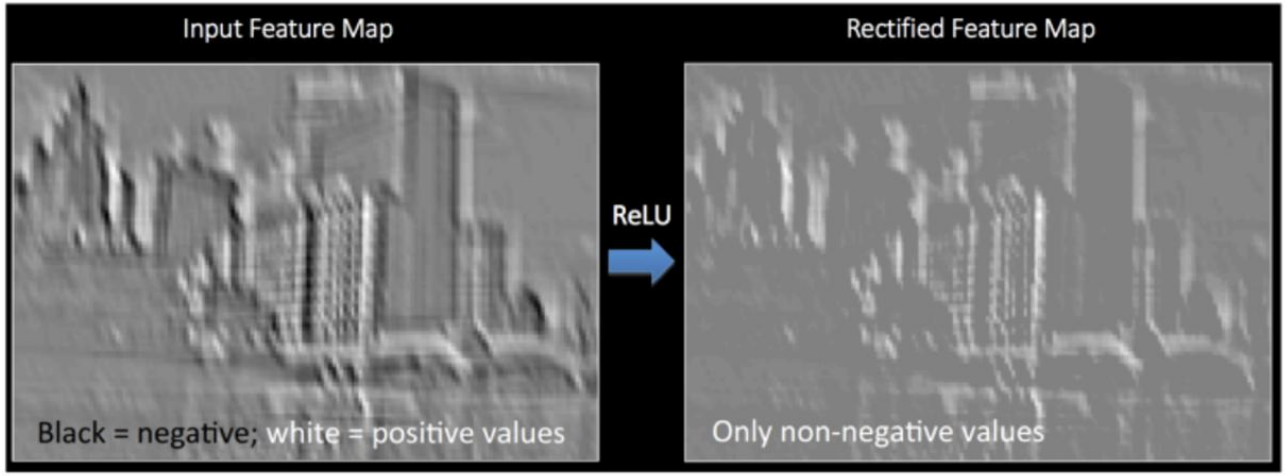


Input Image | Filter or Kernel | Output or Feature Map

Applying the ReLU Activation



Example of a Rectified Linear Map



Source - http://mlss.tuebingen.mpg.de/2015/slides/fergus/Fergus_1.pdf

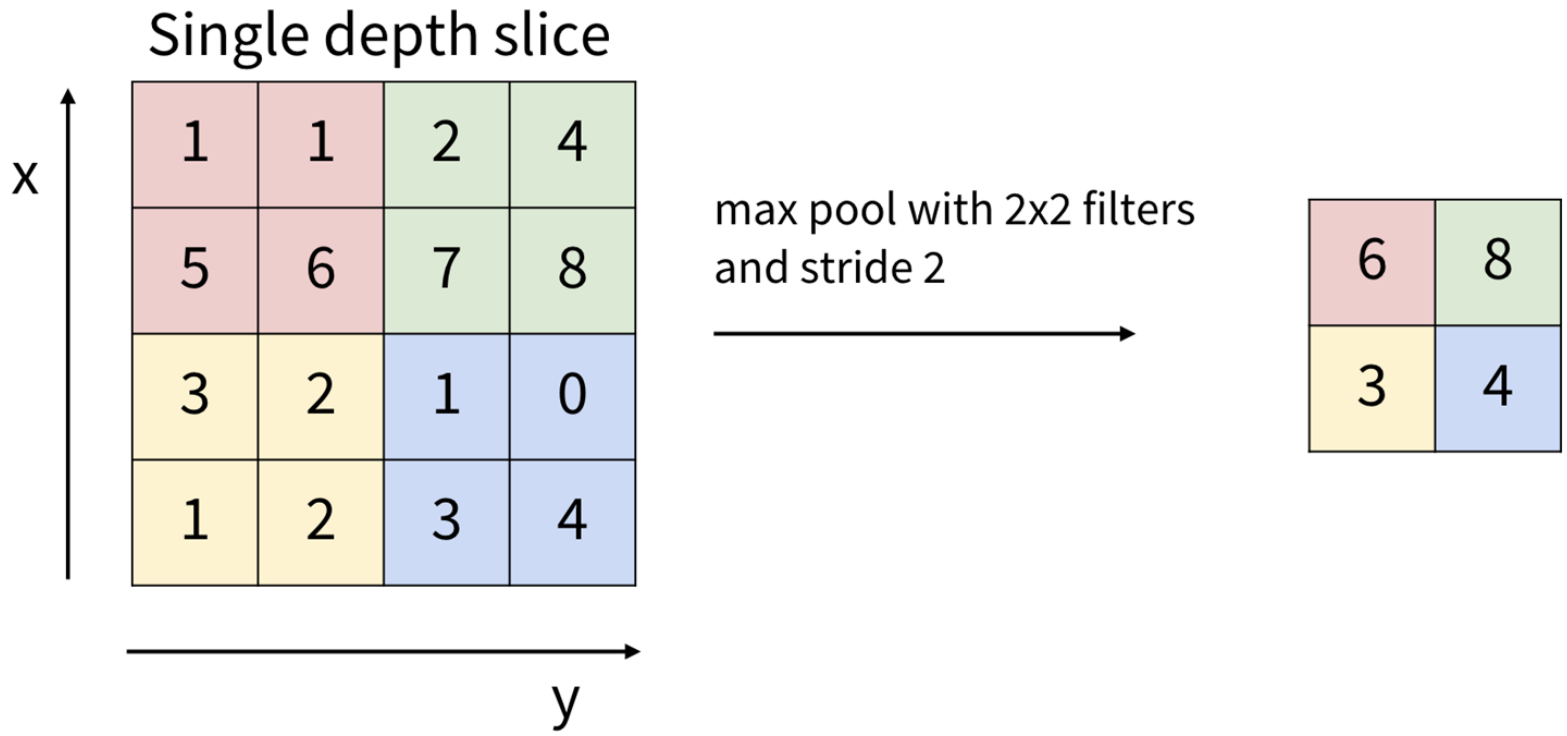
Pooling

We explore the Max Pool Layer and it's purpose

Pooling

- Pooling is the process whereby we reduce the size or dimensionality of the Feature Map
- This allows us to reduce the number of Parameters in our Network whilst retaining important features
- Also called Subsampling or Downsampling

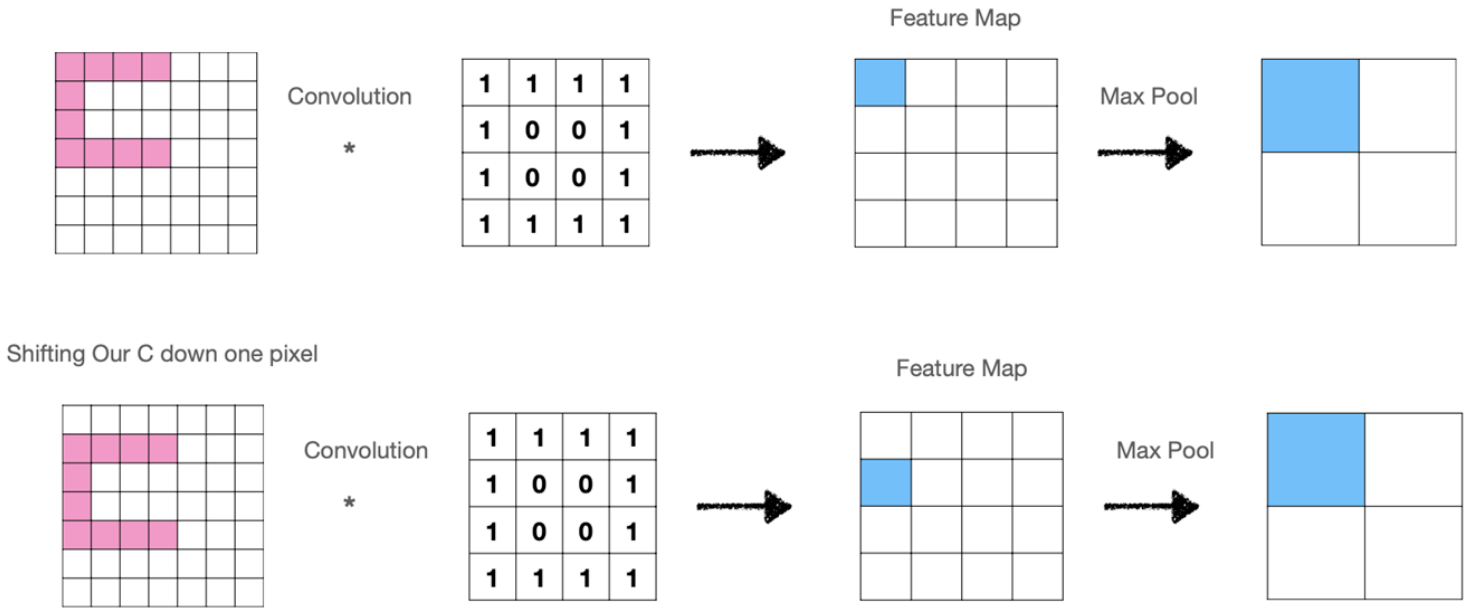
Example of Max Pooling



More on Pooling

- Typically we use 2x2 kernels and a Stride of 2 with no padding.
- With the above setting, pooling reduces the dimensionality by a factor of 2 (width and height).
- Pooling makes our model more invariant to minor transformations and distortions in our image.
- We can also use Average Pooling or Sum Pooling

How Max Pooling Achieves Translation Invariance



Why Pooling Works

Pooling reduces our feature map size by half in most cases, is that ok?

- Neighbouring pixels are **strongly correlated**, especially in lowest layers
- Remember further apart two pixels are from each other, the less correlated
- Therefore, we can **reduce the size** of the output by subsampling (**pooling**) the filter response **without losing information**
- A big stride in the pooling layer leads to high information loss
- In practice, a stride of 2 and a kernel size 2x2 for the pooling layer was found to be effective in practice

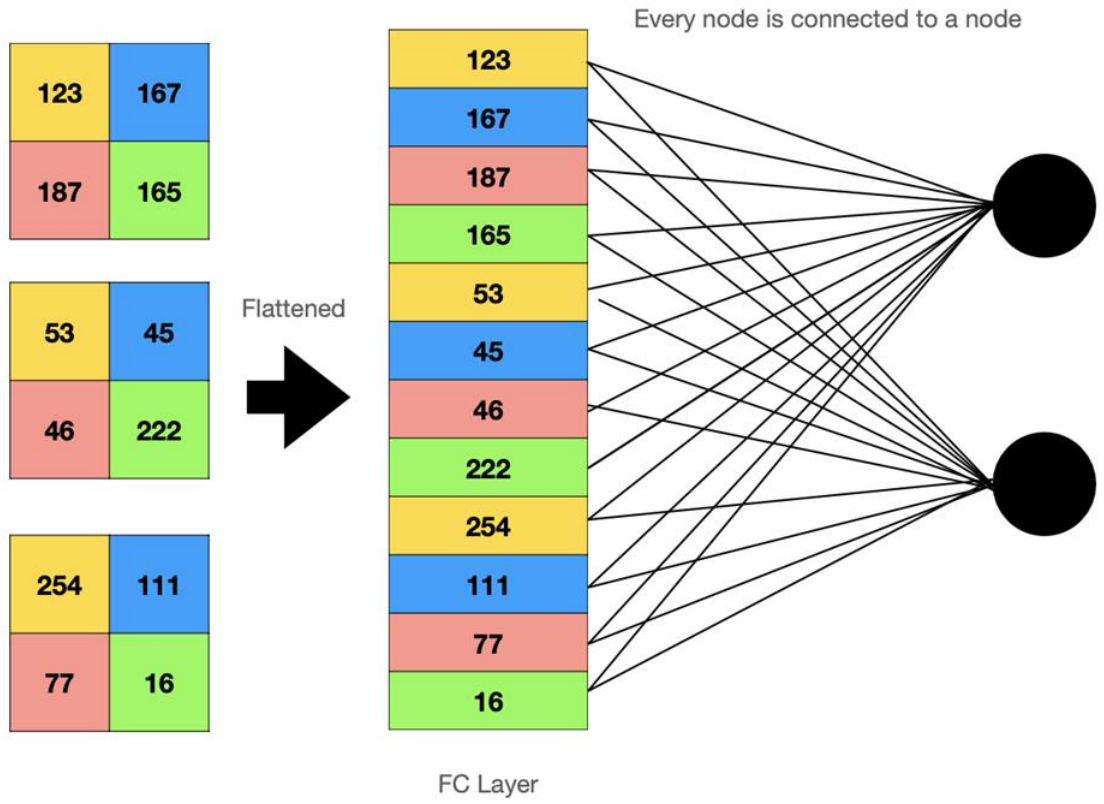
Fully Connected Layer

We explore the Fully Connected Layer and it's purpose

What does Fully Connected Mean?

- FC is a type of layer used in artificial neural networks where each neuron or node from the previous layer is connected to each neuron of the current layer.
- It takes the 3D Volume output of the previous layer and flattens it into a single vector that is used for input in the next layer.
- It's sometimes called the Dense Layer

FC Layer - The Max Pool Layer is Flattened



Purpose of the Fully Connected Layer

- It compiles the data/outputs extracted from previous layers to form the final output
- It's an easy way of learning non-linear combinations of these features

Softmax Layer

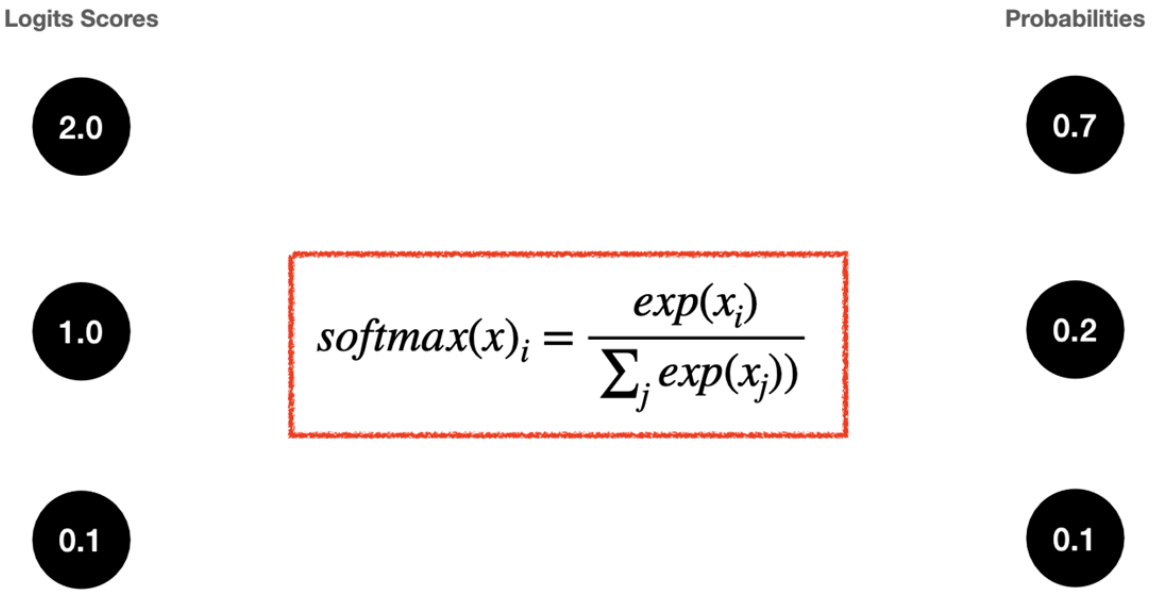
Softmax Layer to Probabilities

Softmax Layer

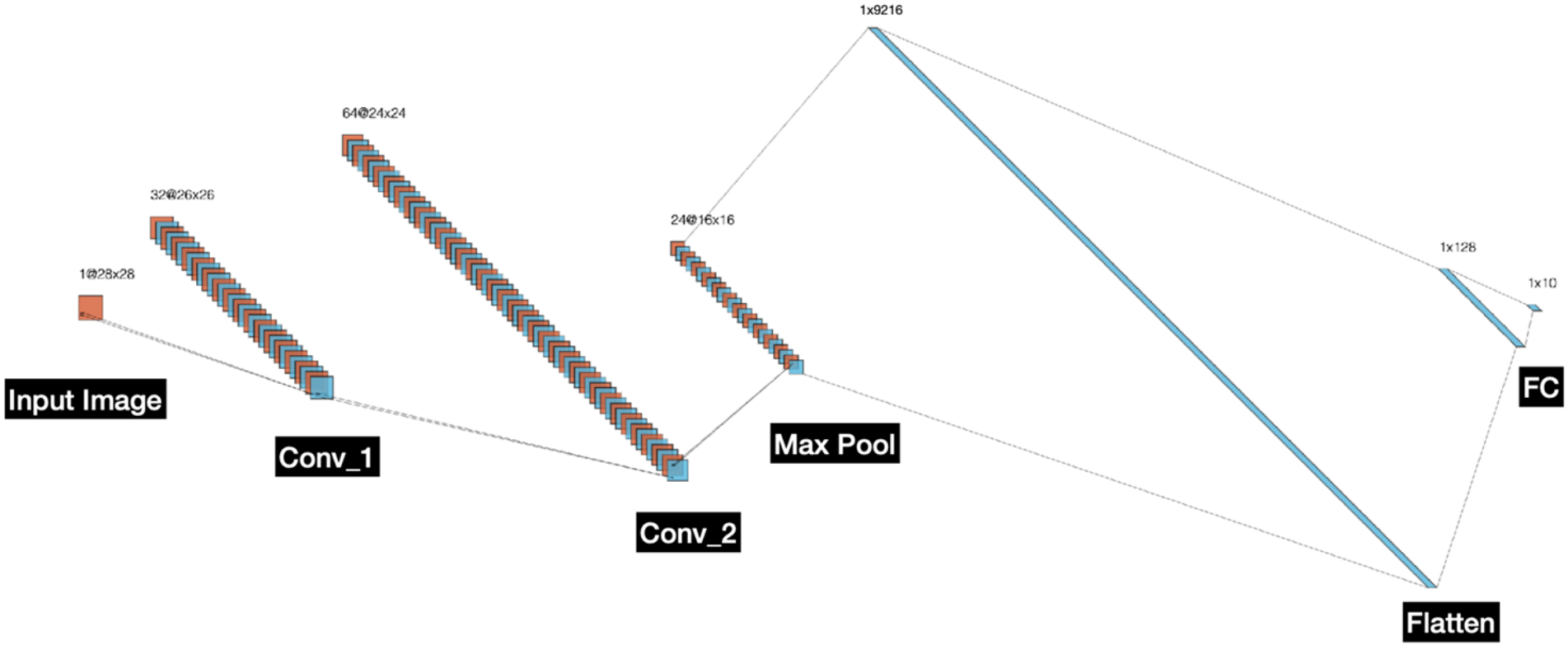
- We now need to produce probability outcomes for each class in Network.
- Softmax converts the network's final layer Logits into Probability distribution of classes
- It takes the exponents of every output and the normalises each output by the sum of the exponents.
- It guarantees a well behaved distribution i.e. all sum to 1 and no values are zero.

$$softmax(x)_i = \frac{exp(x_i)}{\sum_j exp(x_j))}$$

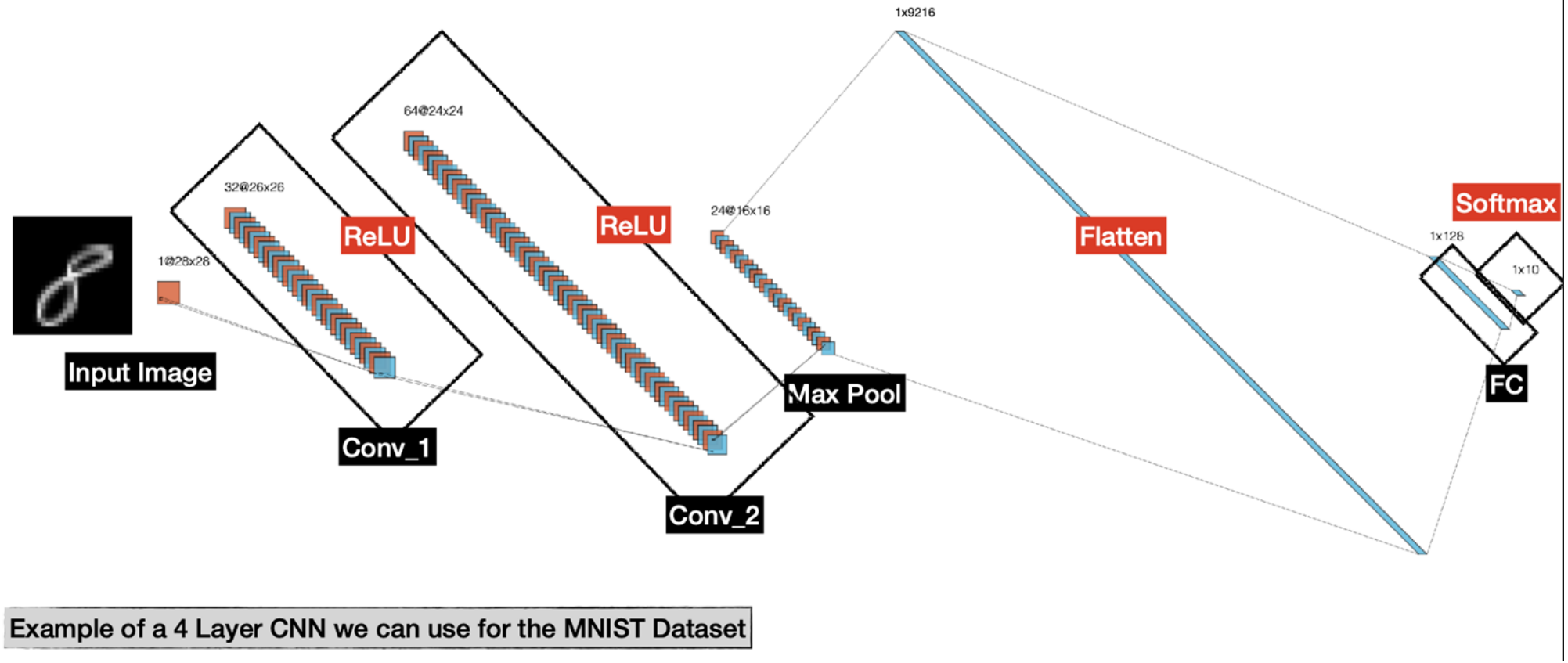
Softmax Layer



A Simple CNN

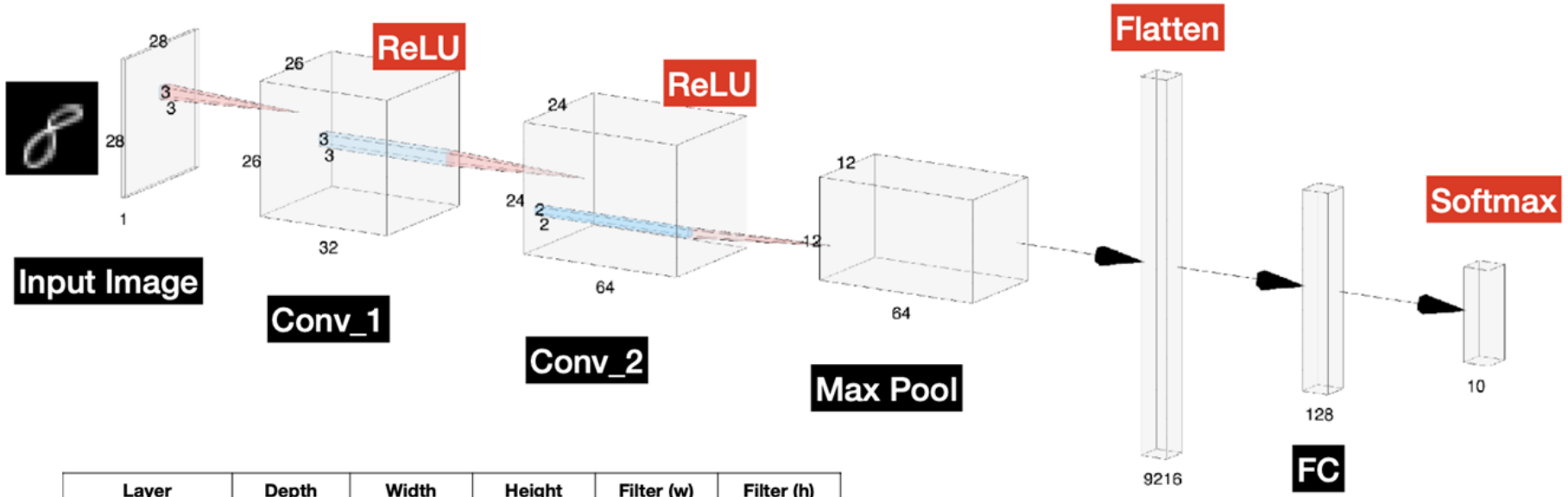

1x9216
64@24x24
32@26x26
24@16x16
1x128
1x10
1@28x28
Input Image
Conv_1
Conv_2
Max Pool
Flatten
FC

A 4 Layer Deep CNN for MNIST



Example of a 4 Layer CNN we can use for the MNIST Dataset

Another Representation

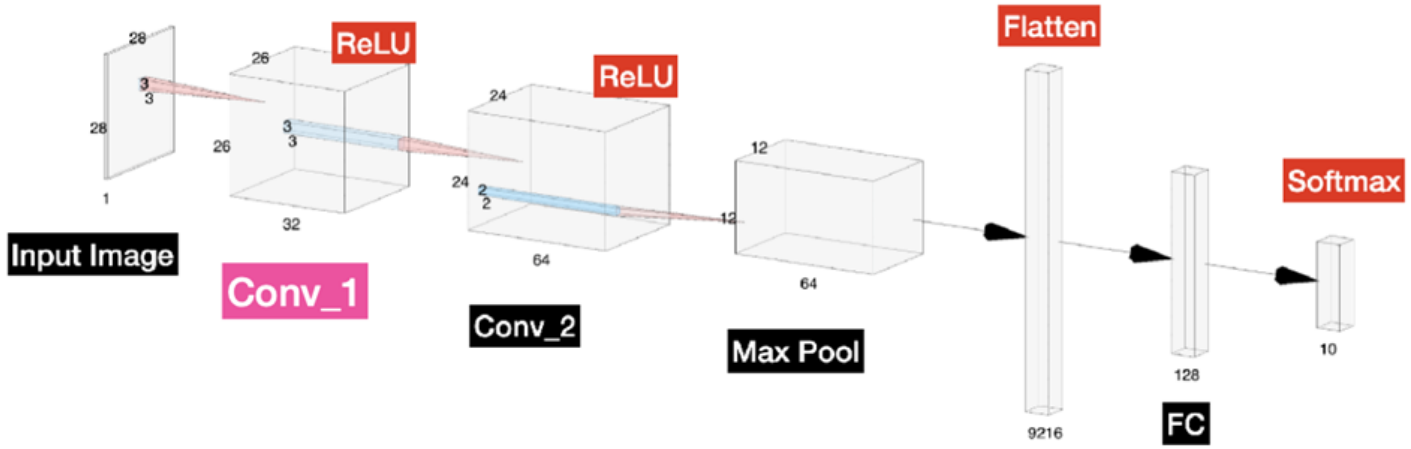


Layer	Depth	Width	Height	Filter (w)	Filter (h)
Input	1	28	28		
Conv_1	32	26	26	3	3
Conv_2	64	24	24	3	3
Max Pool	64	12	12	2	2
Flatten	9216	1	1		
Fully Connected	128	1	1		
Output	10	1	1		

Notes:

- We choose 32 & 64 Filters or Kernels for Conv_1 & Conv_2
- We choose to
- The Feature Maps are shown as Conv_1 and Conv_2
- Stride is 1
- Padding is 0 (not used)
- Max Pool Stride is 2

Calculating the Output Size of Conv_1



Notes:

- We choose 32 Filters or Kernels for Conv_1
- The Feature Maps are shown as Conv_1 & Conv_2
- Stride is 1
- Padding is 0 (not used)
- Max Pool Stride is 2

$$(n \times n) * (f \times f) = (\frac{n + 2p - f}{s} + 1) \times (\frac{n + 2p - f}{s} + 1) = (\frac{28 + (2 \times 0) - 3}{1} + 1) \times (\frac{28 + (2 \times 0) - 3}{1} + 1) = \text{26 x 26}$$

Where

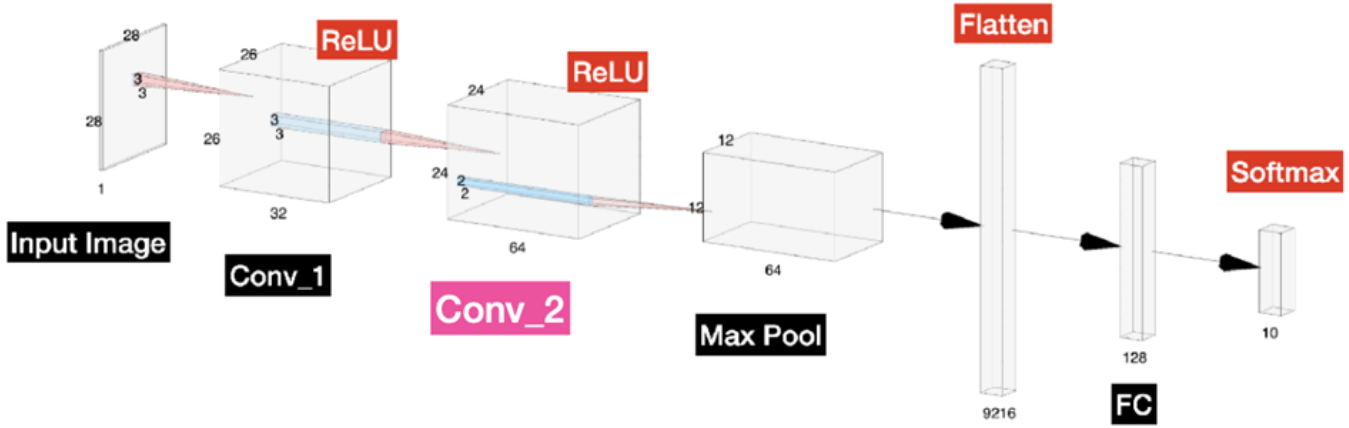
$n = 28$

$f = 3$

$s = 1$

$p = 0$

Calculating the Output Size of Conv_2



Notes:

- We choose 64 Filters or Kernels for Conv_2
- The Feature Maps are shown as Conv_1 & Conv_2
- Stride is 1
- Padding is 0 (not used)
- Max Pool Stride is 2

$$(n \times n) * (f \times f) = (\frac{n + 2p - f}{s} + 1) \times (\frac{n + 2p - f}{s} + 1) = (\frac{26 + (2 \times 0) - 3}{1} + 1) \times (\frac{26 + (2 \times 0) - 3}{1} + 1) = \text{24 x 24}$$

Where

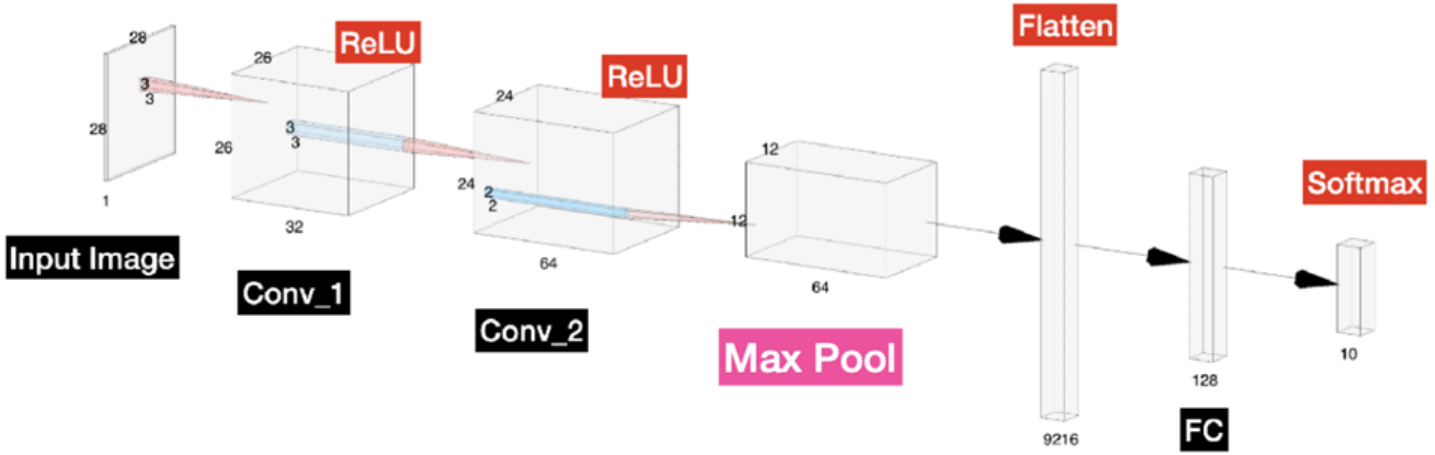
$n = 26$

$f = 3$

$s = 1$

$p = 0$

Calculating the Output Size of the Max Pool Layer



Notes:

- We choose 64 Filters or Kernels for Conv_2
- The Feature Maps are shown as Conv_1 & Conv
- Stride is 1
- Padding is 0 (not used)
- Max Pool Stride is 2

$$(n \times n) * (f \times f) = (\frac{n+2p-f}{s}+1) \times (\frac{n+2p-f}{s}+1) = (\frac{24+(2\times 0)-2}{2}+1) \times (\frac{24+(2\times 0)-2}{2}+1) = 12 \times 12$$

12 x 12

Where

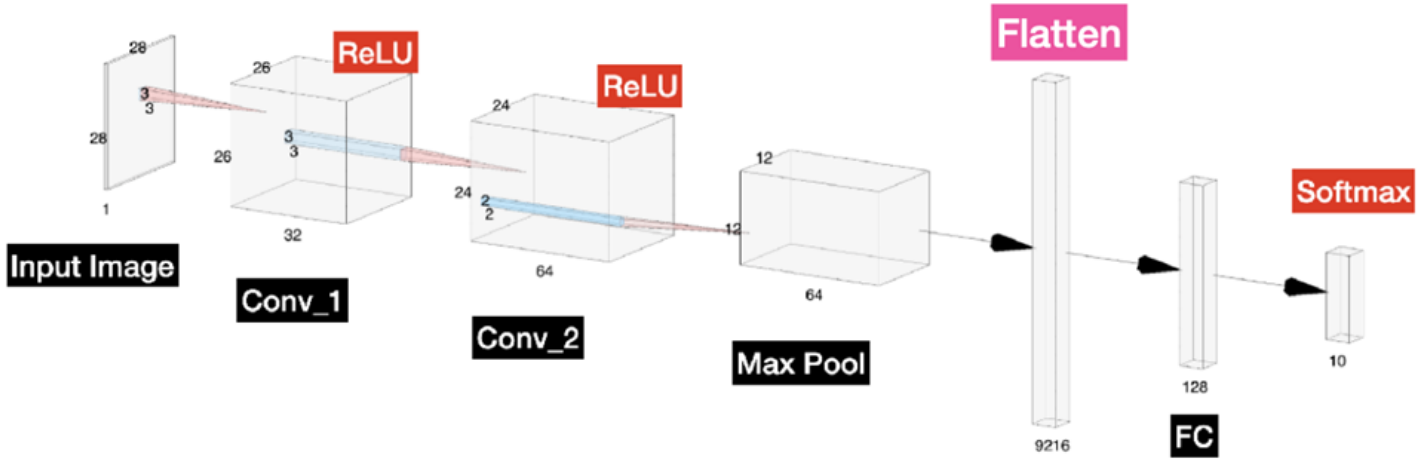
$n = 24$

$f = 2$

$s = 2$

$p = 0$

Calculating the Output Size of Flattened Layer



Notes:

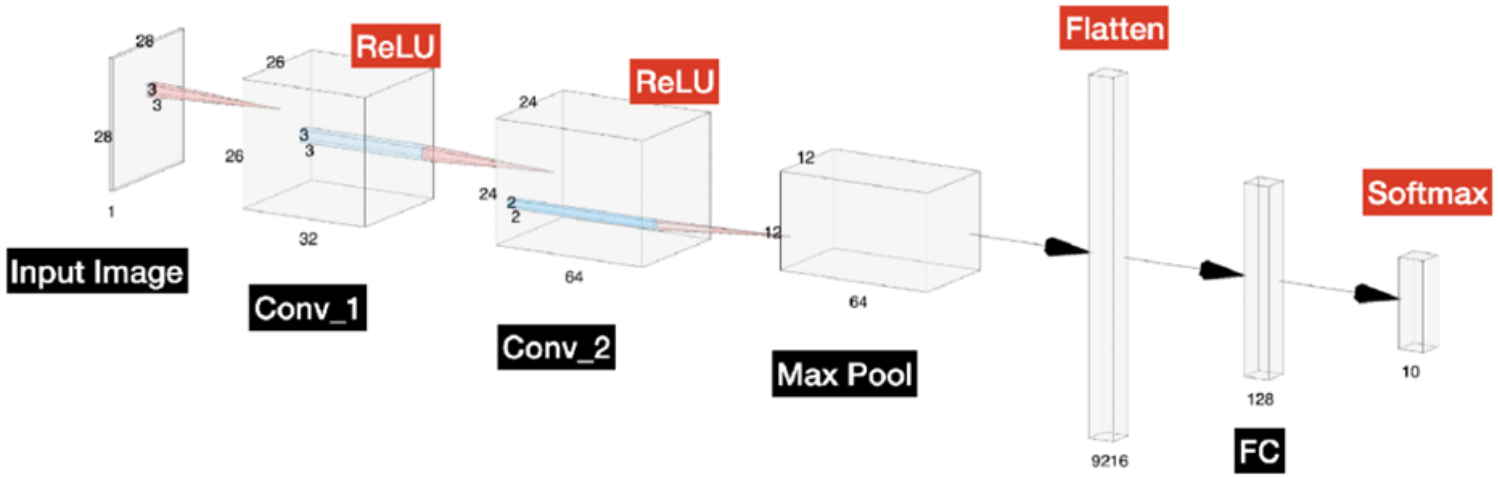
- We choose 64 Filters or Kernels for Conv_2
- The Feature Maps are shown as Conv_1 & Conv_2
- Stride is 1
- Padding is 0 (not used)
- Max Pool Stride is 2

$$12 \times 12 \times 64 = 9216$$

Where

$$n = 12$$

The Rest of the Our CNN



Notes:

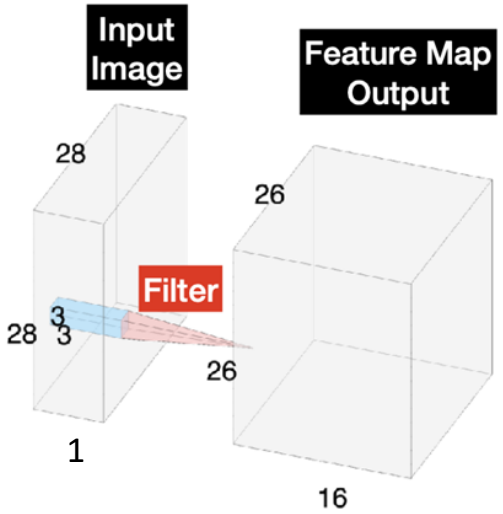
- We choose 64 Filters or Kernels for Conv_2
- The Feature Maps are shown as Conv_1 & Conv_2
- Stride is 1
- Padding is 0 (not used)
- Max Pool Stride is 2
- **We choose 128 nodes in our FC Layer**
- **Our Dataset has 10 classes, hence why the final layer has 10 nodes**

Parameters

What are Parameters or weights?

- They are the variables that need to be learnt when training a Model
- Often called learnable parameters or weights
- Our hidden layers like the Convolution or Fully Connected Layers have weights

Calculating Learnable Parameters in a Conv Filter



- How many parameters are in 16 Convolution Filters with 3x3 kernels?
- Does the input image dimension matter? No, only it's depth
- All that matters is that we have 16 Conv Filters of 3x3 kernel size

Parameters for One Filter

$$(Height \times Width \times Depth) + bias$$

$$(3 \times 3 \times 1) + 1 = \boxed{10}$$

Parameters for 16 Filters

$$((Height \times Width \times Depth) + bias) \times 16$$

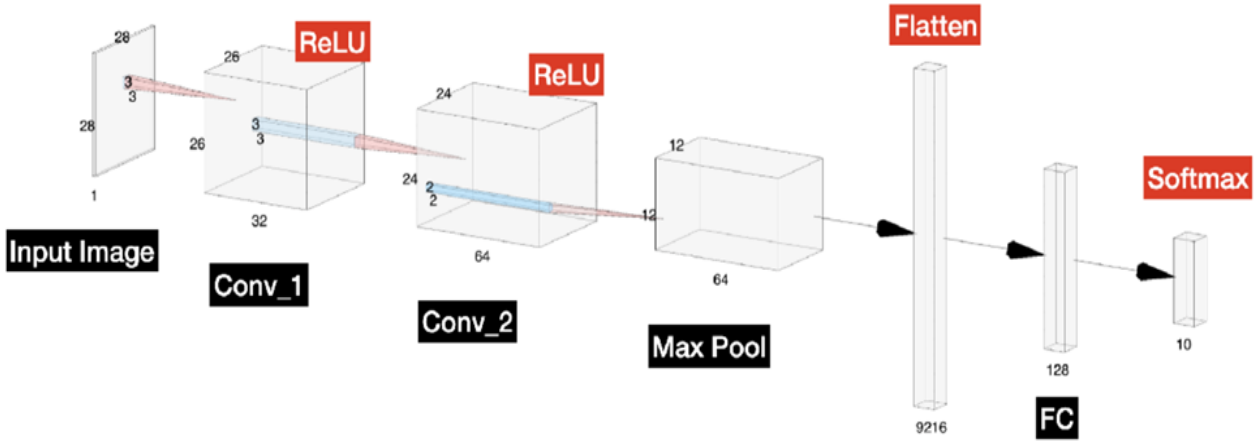
$$((3 \times 3 \times 1) + 1) \times 16 = \boxed{160}$$

Note: Height, Width = the size of filters/kernels (k x k/ f x f)

Biases

- Biases allow us to shift our activation function (left or right) by adding a constant value
- Biases are per '**neuron**', so per **filter** in our case (shared in the case with colour RGB images as the input)
- These shared biases per 'neuron' allow the filter to detect the same feature
- They are a learnable parameter

Let's Calculate the Number of Parameters in Our Simple CNN



Layer	Parameters
Conv_1 + ReLU	**320**
Conv_2 + ReLU	**18494**
Max Pool	**0**
Flatten	**0**
FC_1	**1,179,776**
FC_2 (Output)	**1,290**
Total	**1,199,882**

Conv_1

$$((Height \times Width \times Depth) + bias) \times N_f$$

$$((3 \times 3 \times 1) + 1) \times 32 = 320$$

Conv_2

$$((Height \times Width \times Depth) + bias) \times N_f$$

$$((3 \times 3 \times 32) + 1) \times 64 = 18{,}494$$

No Trainable Parameters

- Max Pool
- Flatten
- ReLU

Fully Connected/Dense

$$(Length + bias) \times N_{nodes}$$

$$(9216 + 1) \times 128 = 1{,}179{,}776$$

Final Output (FC/Dense)

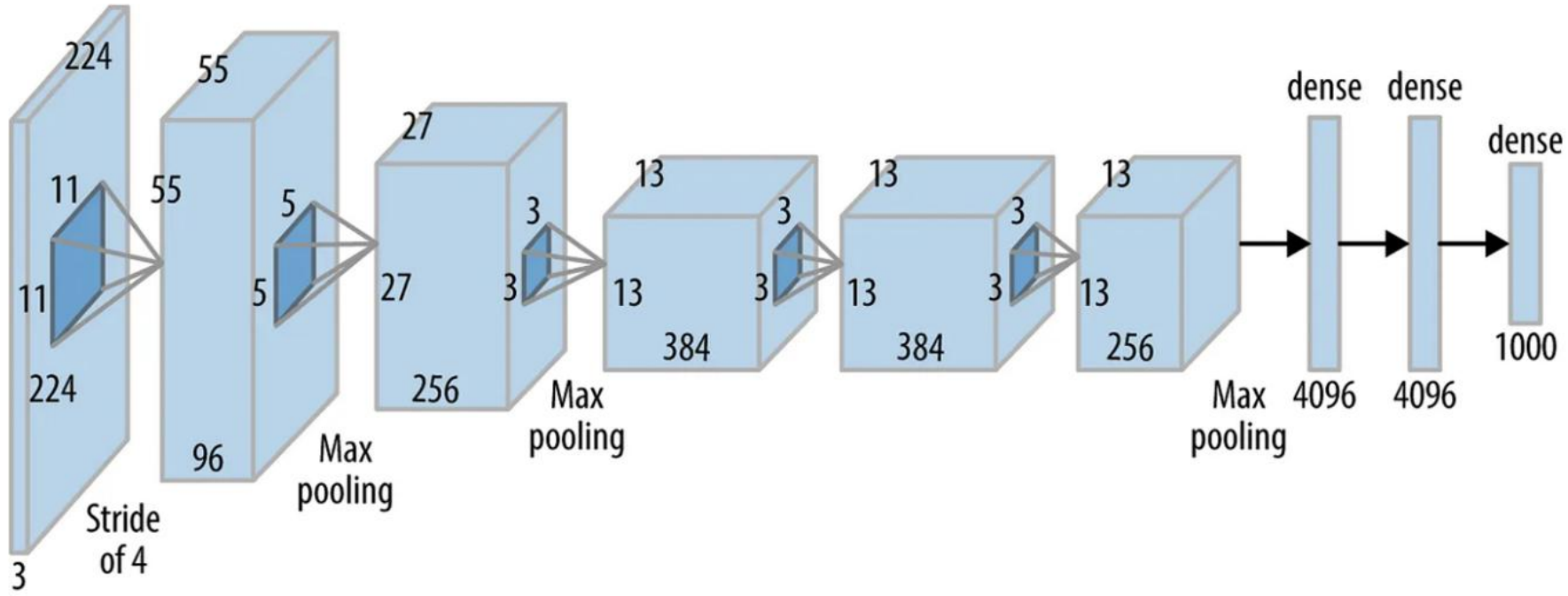
$$(Length + bias) \times N_{nodes}$$

$$(128 + 1) \times 10 = 1{,}290$$

Note:

- Height, Width = the size of filters/kernels (k x k/ f x f)

Let's Calculate the number of paramaters in Alexnet Networks



Alexnet Block Diagram (source:oreilly.com)

Let's Calculate the number of paramaters in Alexnet Networks

AlexNet Network - Structural Details													
Input			Output			Layer	Stride	Pad	Kernel size		in	out	# of Param
227	227	3	55	55	96	conv1	4	0	11	11	3	96	34944
55	55	96	27	27	96	maxpool1	2	0	3	3	96	96	0
27	27	96	27	27	256	conv2	1	2	5	5	96	256	614656
27	27	256	13	13	256	maxpool2	2	0	3	3	256	256	0
13	13	256	13	13	384	conv3	1	1	3	3	256	384	885120
13	13	384	13	13	384	conv4	1	1	3	3	384	384	1327488
13	13	384	13	13	256	conv5	1	1	3	3	384	256	884992
13	13	256	6	6	256	maxpool5	2	0	3	3	256	256	0
						fc6			1	1	9216	4096	37752832
						fc7			1	1	4096	4096	16781312
						fc8			1	1	4096	1000	4097000
						Total							**62,378,344**

Conv1: $((11\ x\ 11\ x\ 3) + 1)\ x\ 96\ =\ 34{,}944$

The network has a total of 62 million trainable variables

Why CNNs Work So Well For Images

We explore design choices in CNNs and how it relates to their performance in the real world

Standard Neural Networks

A thought experiment…

- Standard Neural Networks don't have Convolution Filter Inputs
- For Images every pixel will be it's own input
- Therefore, a small image that's 28 x 28 would have 784 input nodes for our first layer

Standard Neural Networks

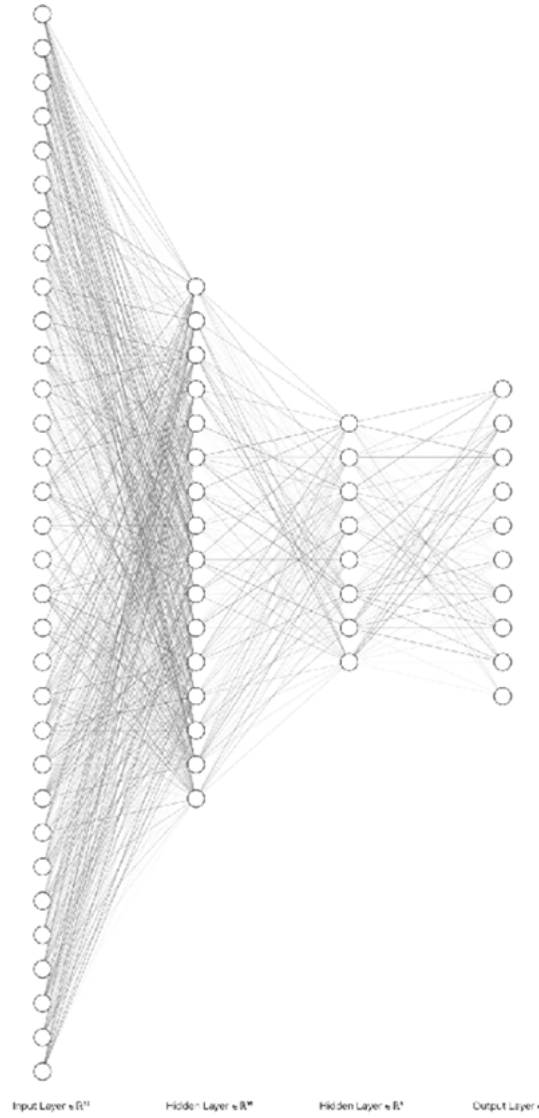
A thought experiment…

- Our second layer, if we breakdown our previous CNN model would be:
  - 32 Filters for 26 x 26 Feature Maps
  - 13,312
- If they were fully connected, our weight matrix would be:
  - 784 x 13,312 = **10,436,608**
- For just one hidden layer!

Standard Neural Networks

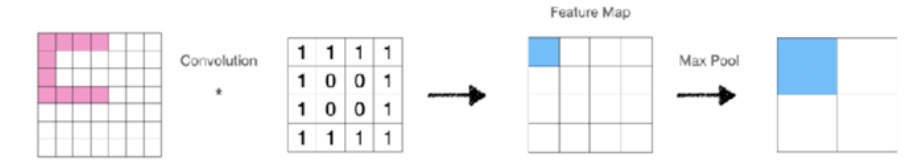
Not Feasible for Image Classification!

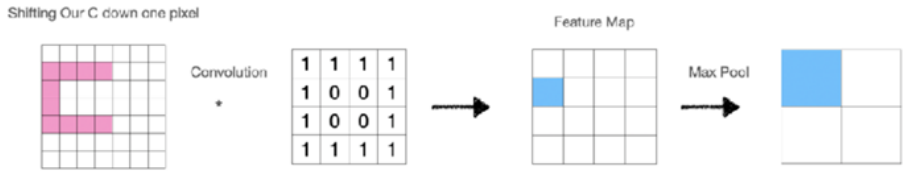
- Not scalable to large data inputs
- Overfitting

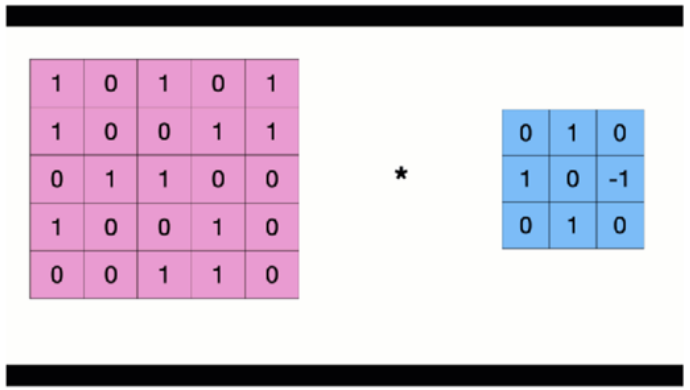


Advantages of Convolution Neural Networks

- **Parameter sharing** - where a single filter can be used all parts of an image
- **Sparsity of connections** - As we saw, fully connected layers in a typical Neural Network result in a weight matrix with large number of parameters.
- **Invariance** - Remember our Max Pool Example

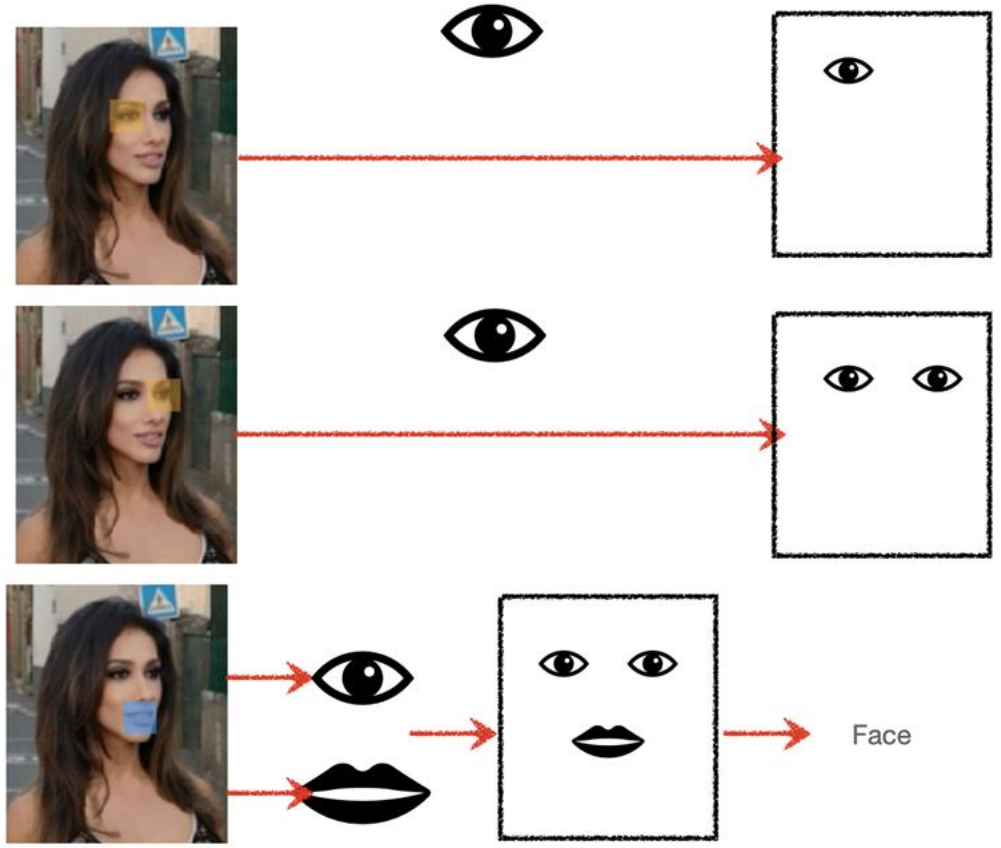






Convolution Neural Networks Assumptions

- Low-level features are local
- Features are translational invariant
- High-level features are made up of low-level features

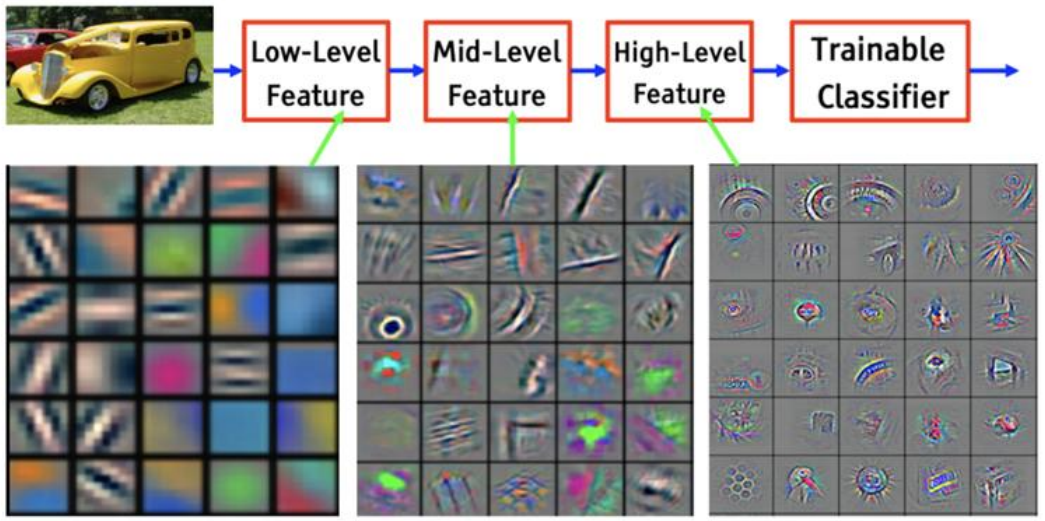


How to Train a CNN

We now explore a high level view of how the training process works

What Conv Filters Learn

- Typically early layers of our CNN learn **low level** features (like edges, or lines)
- Mid-level layers learn **simple patterns**
- High-level layers learn more **structured complex patterns**
- **How is this done?**

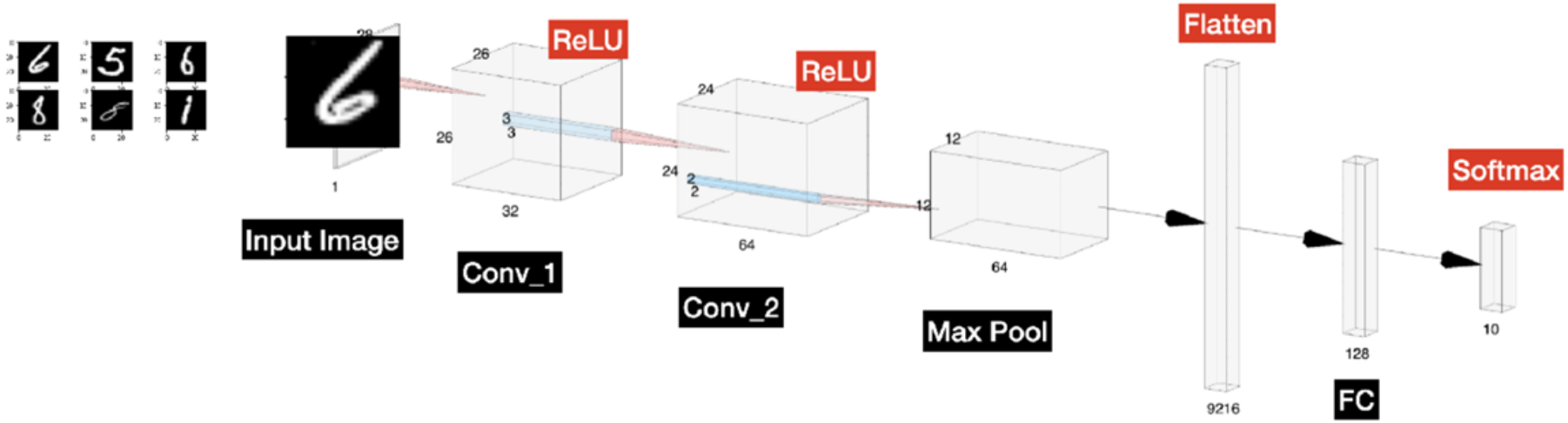


http://www.iro.umontreal.ca/~bengioy/talks/DL-Tutorial-NIPS2015.pdf

What Happens During Training?

- **Initialise random weights** values for our trainable parameters
- **Forward propagate** an image or batch of images through our network
- Calculate the **total error**
- Use **Back Propagation** to update our gradients (weights) via Gradient Descent
- **Propagate more images** (or batch) and update weights, until all images have been propagated (one epoch)
- **Repeat a few more epochs** (i.e. passing all image batches through our Network) until our loss reaches satisfactory values

The Training Process



	6	5	6	8	8	1
0	0.06	0.13	0.13	0.07	0.06	0.1
1	0.12	0.12	0.14	0.12	0.13	0.06
2	0.12	0.13	0.07	0.09	0.06	0.11
3	0.08	0.09	0.06	0.11	0.1	0.13
4	0.1	0.07	0.12	0.15	0.09	0.09
5	0.13	0.1	0.1	0.11	0.14	0.06
6	0.11	0.05	0.09	0.1	0.13	0.09
7	0.11	0.1	0.06	0.11	0.12	0.13
8	0.09	0.13	0.1	0.06	0.08	0.1
9	0.07	0.09	0.12	0.07	0.1	0.12

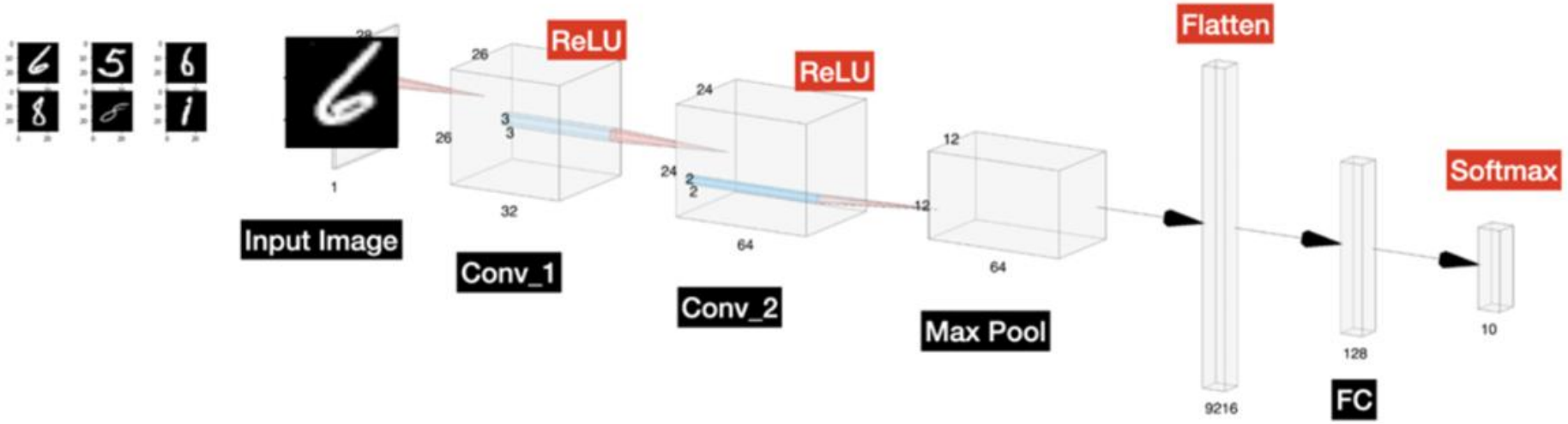
We need to Learn from our Results

- How correct are our results?
- We need a way tell the model it needs to do better

Loss Functions

Loss Functions are essential to training

Quantifying Loss



	6	5	6	8	8	1
0	0.06	0.13	0.13	0.07	0.06	0.1
1	0.12	0.12	0.14	0.12	0.13	0.06
2	0.12	0.13	0.07	0.09	0.06	0.11
3	0.08	0.09	0.06	0.11	0.1	0.13
4	0.1	0.07	0.12	0.15	0.09	0.09
5	0.13	0.1	0.1	0.11	0.14	0.06
6	0.11	0.05	0.09	0.1	0.13	0.09
7	0.11	0.1	0.06	0.11	0.12	0.13
8	0.09	0.13	0.1	0.06	0.08	0.1
9	0.07	0.09	0.12	0.07	0.1	0.12

- How bad are the probabilities we predicted?
- How do we quantify the degree our prediction is off by?

Cross Entropy Loss or Categorical Cross Entropy Loss

Class	Predicted Probabilities	Ground Truth
0	0.1	0
1	0.2	0
2	0.1	0
3	0.05	0
4	0.05	0
5	0.05	0
6	0.05	0
7	0.3	1
8	0.05	0
9	0.05	0

- Cross Entropy Loss uses two distributions, our ground truth distribution $p(x)$ and $q(x)$ our predicted distribution.
- $L = -\,y\,.\,log(\hat{y})$
- Where y is the ground truth vector, $\hat{y}$ is the predicted distribution and ' . ' is the inner product.

Cross Entropy Loss a Simpler Example

Class	Predicted Probabilities	Ground Truth
0	0.3	0
1	0.6	1
2	0.1	0

- $L = -\,y\,.\,log(\hat{y})$
- $L = -\,(0 \times log(0.3) + 1 \times log(0.6) + 0 \times log(0.1)$
- $L = -\,(0 + 1 \times -0.222 + 0) = 0.222$
- **NOTE:**
  - Multi-class log loss rewards/penalises the correct classes only

$$L = \frac{1}{N} \sum_{i-1}^{N} L_i$$

Other Loss Functions

- Loss Functions are sometimes called **Cost Functions**
- For Binary Classification problems we use **Binary Cross-Entropy Loss** (same as categorical cross-entropy loss except it uses just one output node)
- For Regressions we often use the Mean Square Error (MSE)
  - Mean Square Error (MSE) = (Target - Predicted)^2
    - $MSE = \frac{1}{n}\sum (Y_i - \hat{Y}_i)^2$
- Other loss functions that are sometimes used:
  - L1, L2
  - Hinge Loss
  - Mean Absolute Error (MAE)

What do we do with our Quantified Loss?

- Updating all the weights of our model is not trivial
- How do we correctly update our weights to minimise loss?
- We use Back Propagation
- And we use the loss value for this!

- During backpropagation, this loss value is used to calculate the gradient (slope) of the loss function with respect to each weight in the neural network.

- The gradient indicates the direction and magnitude of the weight adjustment needed to minimize the loss.

Backpropagation

Backpropagation makes Neural Networks Trainable

Backpropagation

- Backpropagation is a popular algorithm used in artificial neural networks (ANNs) for training deep learning models.
- It is a supervised learning technique used to adjust the weights of the neurons in the network to minimize the error between the predicted output and the actual output.
- Using the loss, it tells us how much to change/update the gradients by so that we reduce the overall loss

Backpropagation Example

Input	Desired Output
0	0
1	2
2	4

The output of the model when ‘W” value is 3:

Input	Desired Output	Model output (W=3)
0	0	0
1	2	3
2	4	6

Backpropagation Example

- Notice the difference between the actual output and the desired output:

Input	Desired Output	Model output (W=3)	Absolute Error	Square Error
0	0	0	0	0
1	2	3	1	1
2	4	6	2	4

- Let's change the value of 'W'. Notice the error when 'W' = '4'

Input	Desired Output	Model output (W=3)	Absolute Error	Square Error	Model output (W=4)	Square Error
0	0	0	0	0	0	0
1	2	3	1	1	4	4
2	4	6	2	4	8	16

- When we increase the value of 'W' the error has increased. So, obviously there is no point in increasing the value of 'W' further

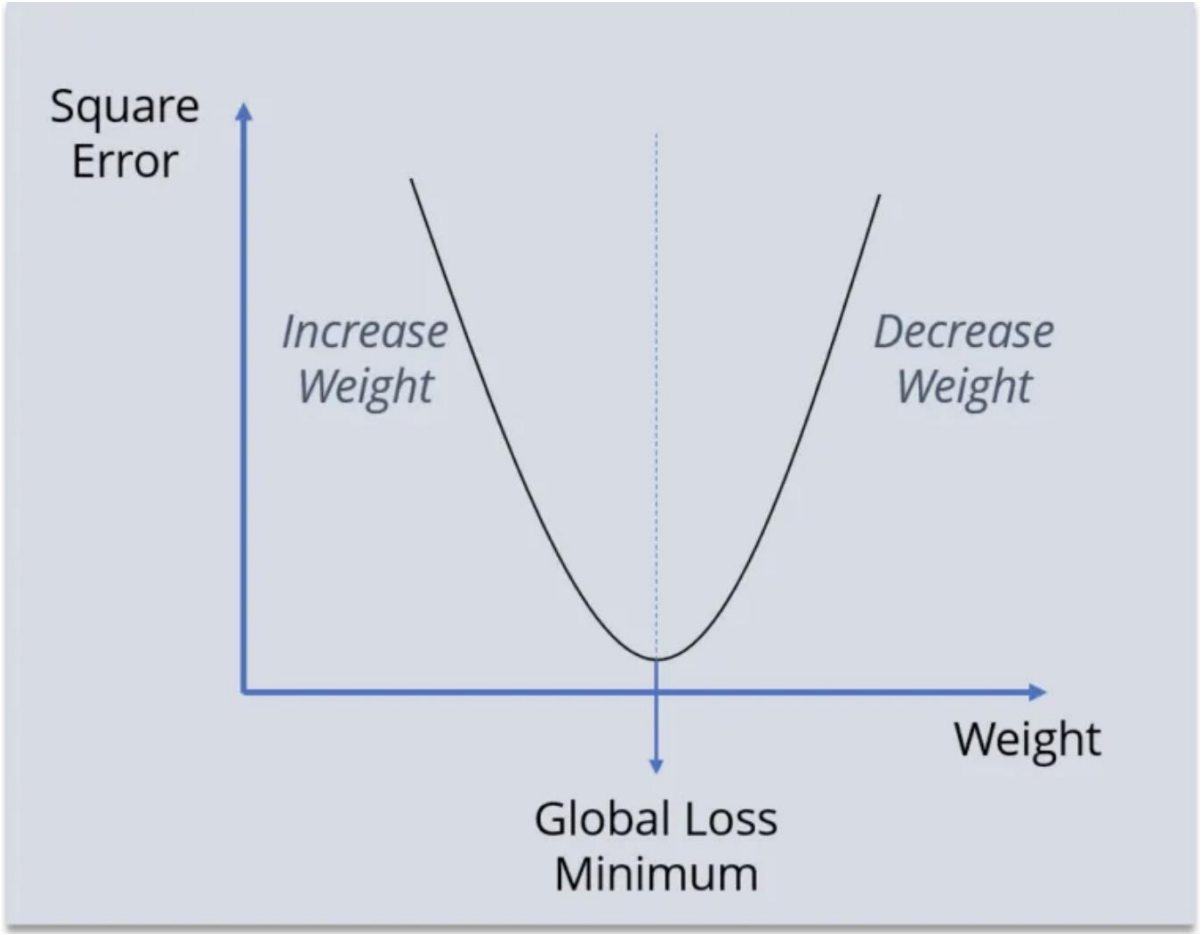
Backpropagation Example

- What happens if I decrease the value of ‘W’?

Input	Desired Output	Model output (W=3)	Absolute Error	Square Error	Model output (W=2)	Square Error
0	0	0	0	0	0	0
1	2	3	2	4	2	0
2	4	6	2	4	4	0

Backpropagation

- We need to reach the 'Global Loss Minimum'.



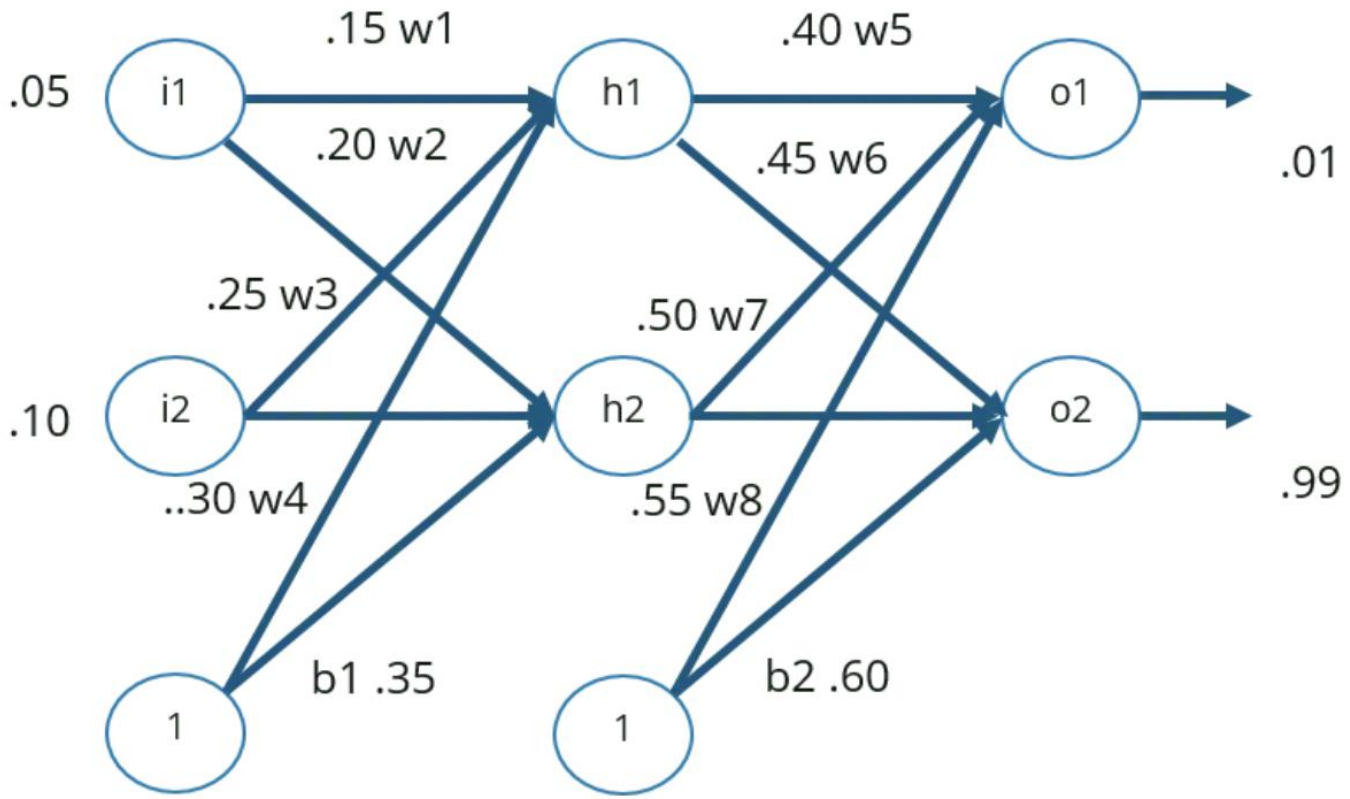
https://medium.com/edureka/backpropagation-bd2cf8fdde81

Backpropagation Process

- Step — 1: Forward Propagation
- Step — 2: Backward Propagation
- Step — 3: Putting all the values together and calculating the updated weight value
- By continuously changing the weights for each data input (or batch of images) we are lowering the overall loss for our training data.

Backpropagation Example

Explained Using Neural Networks

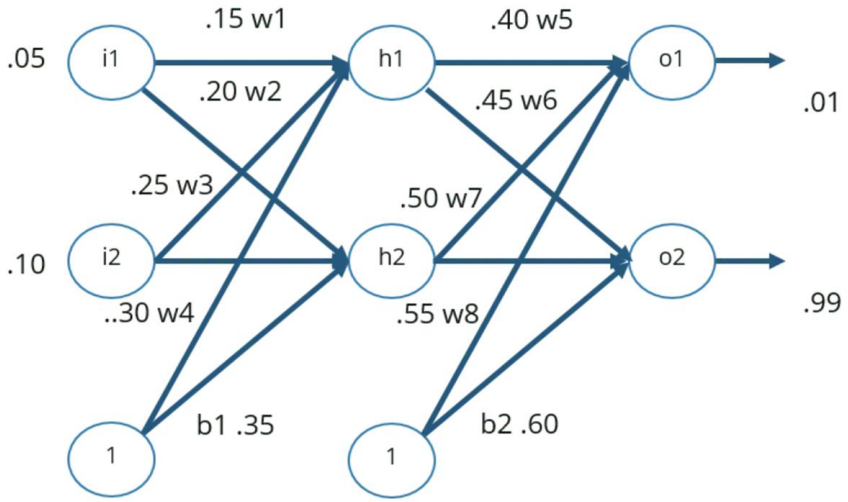


Backpropagation Example

Explained Using Neural Networks

Step — 1: Forward Propagation

We will start by propagating forward.



Net Input For h1:

net h1 = w1*i1 + w2*i2 + b1*1 → net h1 = 0.15*0.05 + 0.2*0.1 + 0.35*1 = 0.3775

Output Of h1:

out h1 = $1/1+e^{-\text{net h1}}$ → $1/1+e^{.3775}$ = 0.593269992

Output Of h2:

out h2 = 0.596884378

$$S(x) = \frac{1}{1 + e^{-x}}$$

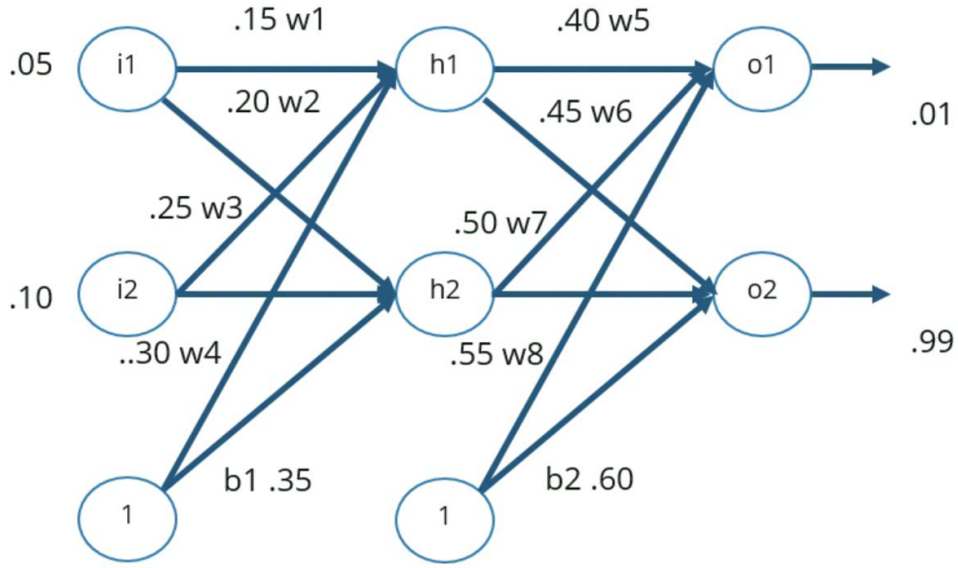
$S(x)$ = sigmoid function

e = Euler's number

Backpropagation Example

Explained Using Neural Networks

- We will repeat this process for the output layer neurons, using the output from the hidden layer neurons as inputs.



Output For o1:

net o1 = w5*out h1 + w6*out h2 + b2*1 → 0.4*0.593269992 + 0.45*0.596884378 + 0.6*1 = 1.105905967

Out o1 = $1/1+e^{-net\ o1}$ → $1/1+e^{-1.105905967}$ = 0.75136507

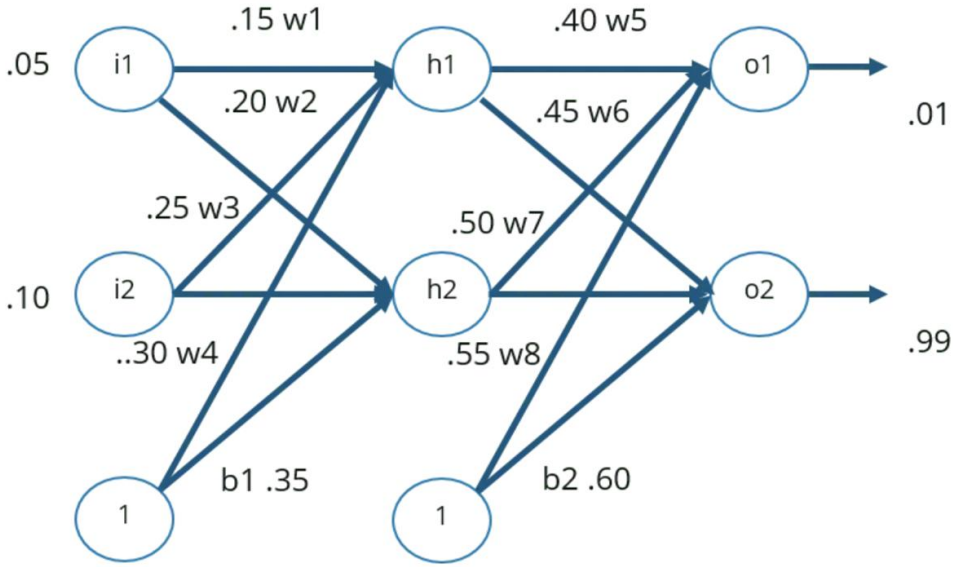
Output For o2:

Out o2 = 0.772928465

Backpropagation Example

Explained Using Neural Networks

Let's see what is the value of the error:



Error For o1:

$E_{o1} = \Sigma 1/2(target - output)^2$ → ½ (0.01 – 0.75136507)² = 0.274811083

Error For o2:

E o2 = 0.023560026

Total Error:

E_{total} = E o1 + E o2 → 0.274811083 + 0.023560026 = 0.298371109

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

MSE = mean squared error
n = number of data points
Y_i = observed values
$\hat{Y}_i$ = predicted values

Backpropagation Example

Explained Using Neural Networks

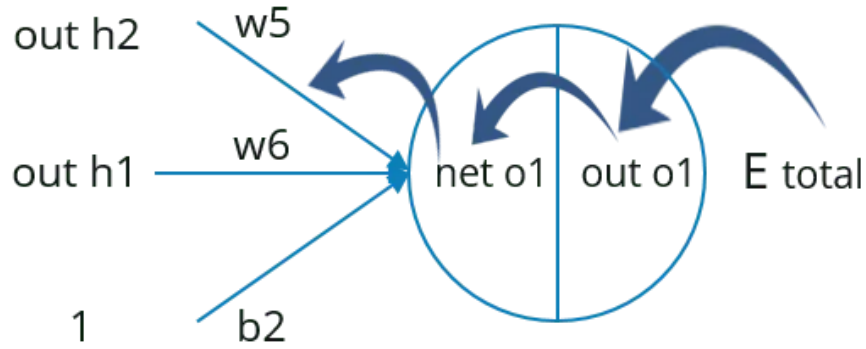
PHENIKAA UNIVERSITY
Faculty of Computer Science

Step — 2: Backward Propagation

- We will propagate backwards. This way we will try to reduce the error by changing the values of weights and biases.
- Consider W5, we will calculate the rate of change of error w.r.t change in weight W5
- But there is no term w5 present in the expression of Etotal.
  So we have to split it & apply the chain rule to partially differentiate it.

Chain rule

$$\frac{\delta Etotal}{\delta w5} = \frac{\delta Etotal}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w5}$$



Backpropagation Example

Explained Using Neural Networks

- Since we are propagating backwards, first thing we need to do is, calculate the change in total errors w.r.t the output o1 and o2.

$$E_{total} = 1/2(\text{target o1} - \text{out o1})^2 + 1/2(\text{target o2} - \text{out o2})^2$$

$$\frac{\delta Etotal}{\delta out\ o1} = -(\text{target o1} - \text{out o1}) = -(0.01 - 0.75136507) = 0.74136507$$

- We will propagate further backwards and calculate the change in output O1 w.r.t to its total net input.

$$\text{out o1} = 1/1+e^{-neto1}$$

$$\frac{\delta out\ o1}{\delta net\ o1} = \text{out o1 (1 - out o1)} = 0.75136507\ (1 - 0.75136507) = 0.186815602$$

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\sigma'(x) = \frac{d}{dx}\sigma(x) = \sigma(x)(1-\sigma(x))$$

Backpropagation Example

Explained Using Neural Networks

PHENIKAA UNIVERSITY
Faculty of Computer Science

- Let's see now how much does the total net input of o1 changes w.r.t W5?

$$\text{net o1} = w5 * \text{out h1} + w6 * \text{out h2} + b2 * 1$$

$$\frac{\delta net\ o1}{\delta w5} = 1 * \text{out h1}\ w5^{(1-1)} + 0 + 0 = 0.593269992$$

Step — 3: Putting all the values together and calculating the updated weight value

Now, let's put all the values together:

Chain rule

$$\frac{\delta Etotal}{\delta w5} = \frac{\delta Etotal}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w5}$$

0.082167041

Let's calculate the updated value of W5:

learning rate

Error at w5

$$w5^{+} = w5 - \lambda \frac{\delta Etotal}{\delta w5}$$

$$w5^{+} = 0.4 - 0.5 * 0.082167041$$

Updated w5

0.35891648

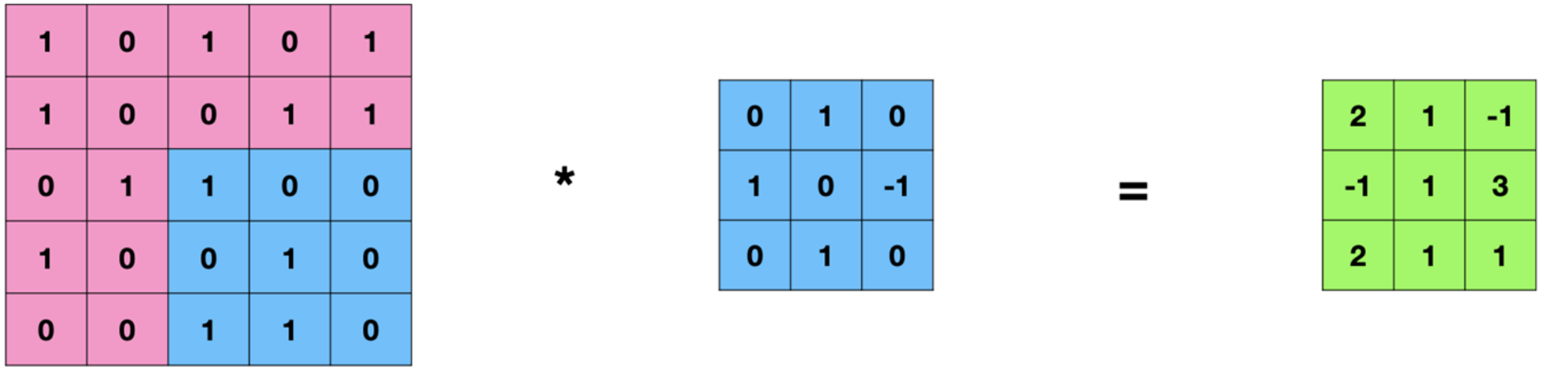
Backpropagation Example

Explained Using Neural Networks

PHENIKAA UNIVERSITY
Faculty of Computer Science

- We want to know how much changing W_5 changes the Total Error
- That is given by:
  - $\frac{dE_T}{dW_5}$
  - Where E_T is them sum of the error from outputs 1 and outputs 2
- Update $W_5 = -\lambda \times \frac{dE_T}{dW_5}$
- Note we introduced a new parameter: λ is our learning rate
- It controls how a big a jump (positive or negative) we take when updating W_5
- Large learning rates train faster, but can get stuck in a Global Minimum
- Small learning rates train more slowly

For a Convolutional Neural Network



- The values of our Filter/Kernel are the weights!

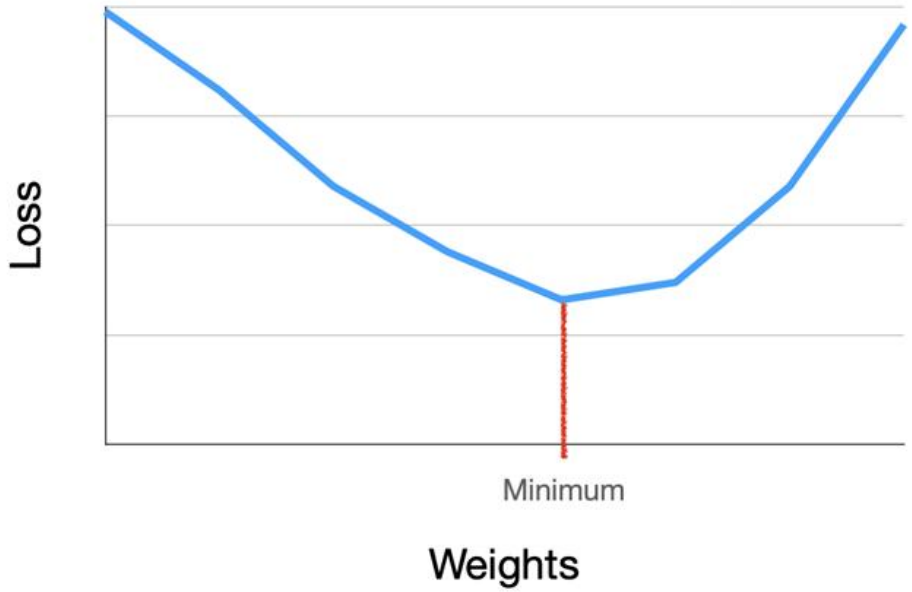
Gradient Descent

Finding the optimal weights

Loss Functions

How do we find the lowest loss?

- Back Propagation is the process we use to update the individual weights or gradients
  - $wx + b$
- Our goal is finding the right value of weights where the loss is lowest
- The method by which we achieve this goal (i.e. updating all weights to lower the total loss) is called **Gradient Descent**
- It's the point at which we find the **optimal weights** such that **loss is near lowest**



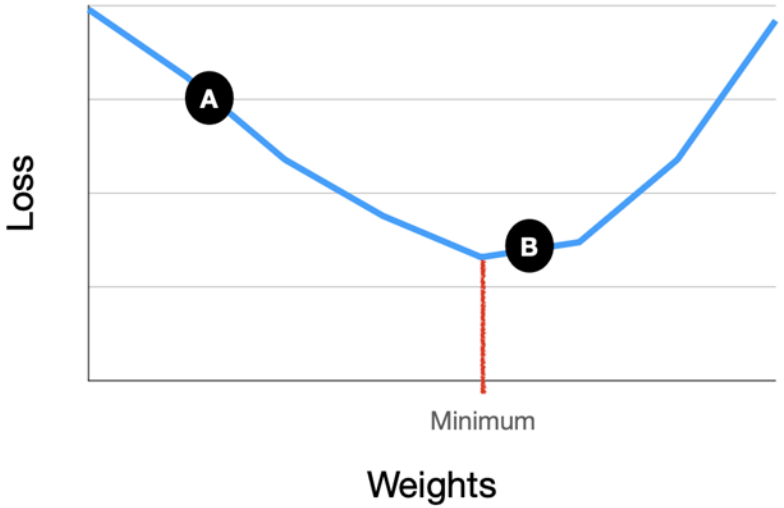
Gradients

Gradients are the derivative of a function

- It tells us the rate of change of one variable with respect to the other e.g.

  $Gradient = \frac{dE}{dw}$, where E is the Error or Loss and w is the weight

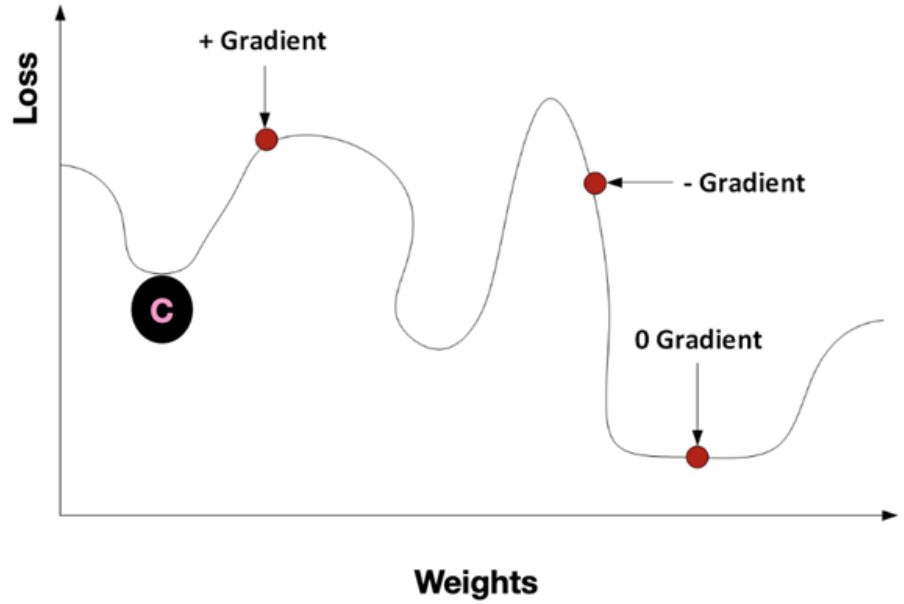
- A positive gradient means, loss increases if weights increase
- A negative gradient means, loss decreases if weights increase
- At point A, moving right increases our weights and decreases our loss, -ve
- A point B, moving right increases our weights and increases our loss, +ve
- Therefore, the **negative** of our gradient tells us the direction we should be moving



- If we have two functions $y = f(u)$ and $u = g(x)$ then the derivative of y is:
  - $\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx}$

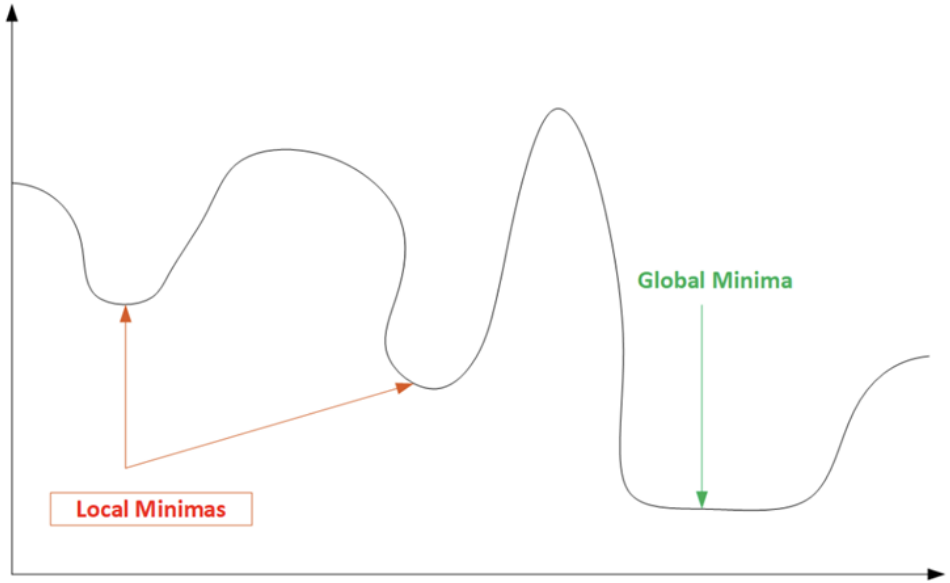
More on Gradients

- The point at which a Gradient is zero means that small changes to the left or right don't change the loss
- In training Neural Networks this is good and bad
- At point, C, very small changes to the left or right don't change the Loss
- This means, our network gets stuck during training.
- This is called getting **stuck in a Local Minima**



Local and Global Minimas

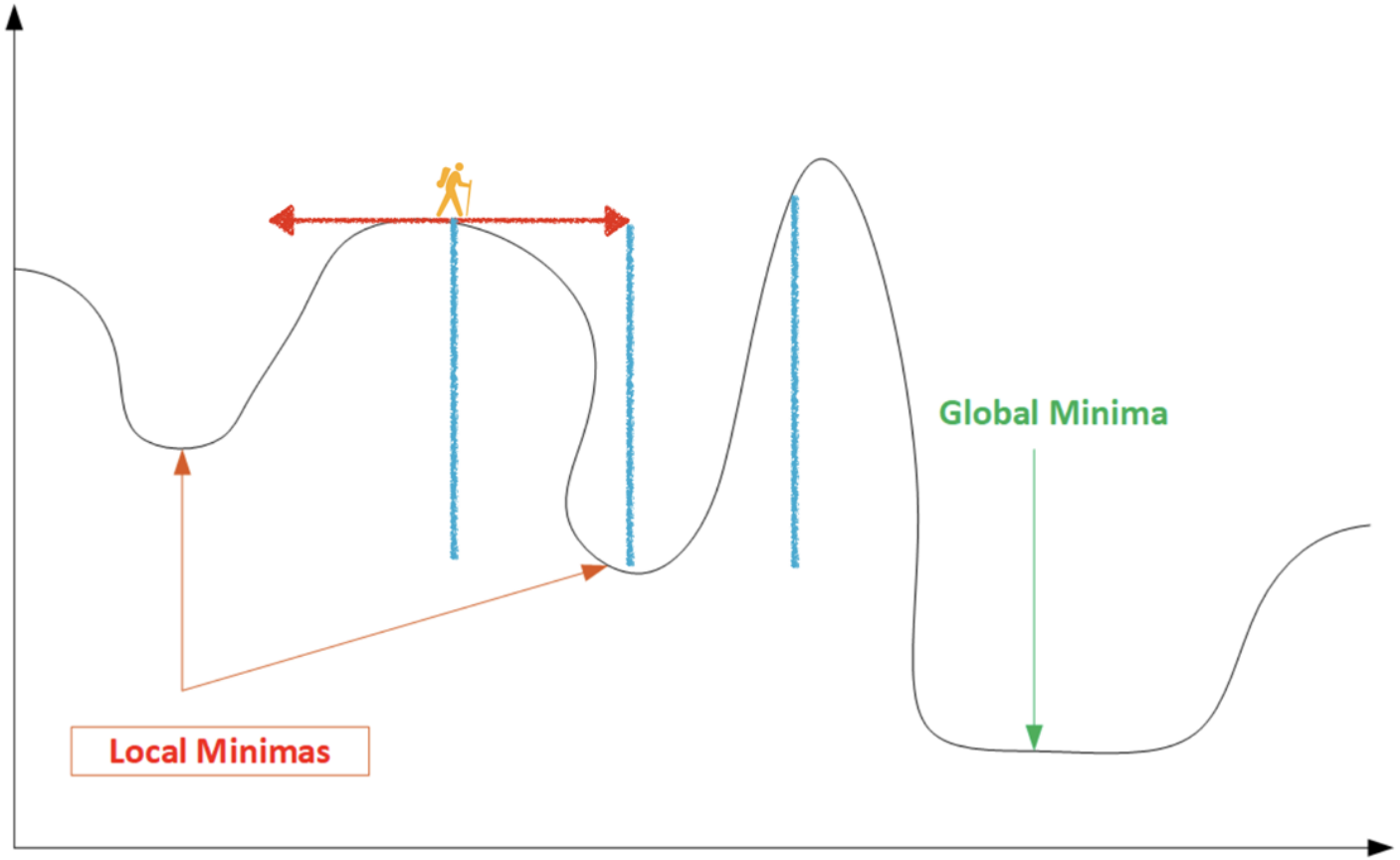
- We always want to find the Global Minima during training.
- That is the point where the combination of weights give the lowest loss



Gradient Descent

- Imagine being a really tiny person and you're traversing down the slope of this old, rough bowl.
- There'd be peaks, valleys, troughs etc.
- How do you know when you're truly at the bottom?
- Possibly take large steps so you don't get stuck in a valley
- But then you risk jumping over the Global Minima

Step Size is Important



Learning Rates

Recall our Back Propagation Weight Update Formula

- $W_5 = -\lambda \times \frac{dE_T}{dW_5}$
- λ is our learning rate
- Learning Rates allow us to adjust the magnitude of jumps in weights
- Finding an optimal value will avoid us finding Local Minimums and while preventing us from jumping over the Global Minimum.

Gradient Descent Methods

- **Naive Gradient Descent** - Passes the entire dataset through our network then updates the weights.
  - It is computationally expensive and slow.
- **Stochastic Gradient Descent (SGD)** - Updates weights after each data sample (image) is forward propagated through our network.
  - This leads to noisy fluctuating loss values and is also slow to train
- **Mini-Batch Gradient Descent** - Combines both methods, it takes a batch of data points (images) and forward propagates all, then updates the gradients.
  - This leads to faster training and convergence to the Global Minima
  - Batches are typically 8 to 256 in size

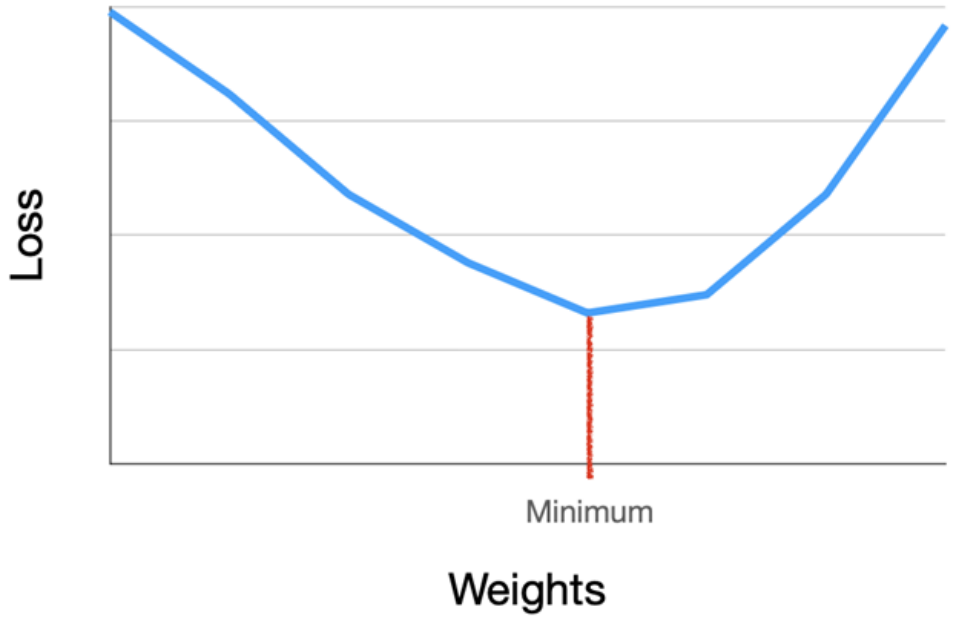
Optimisers & Learning Rate Schedules

Methods or Algorithms used in finding optimal weights

Optimisers

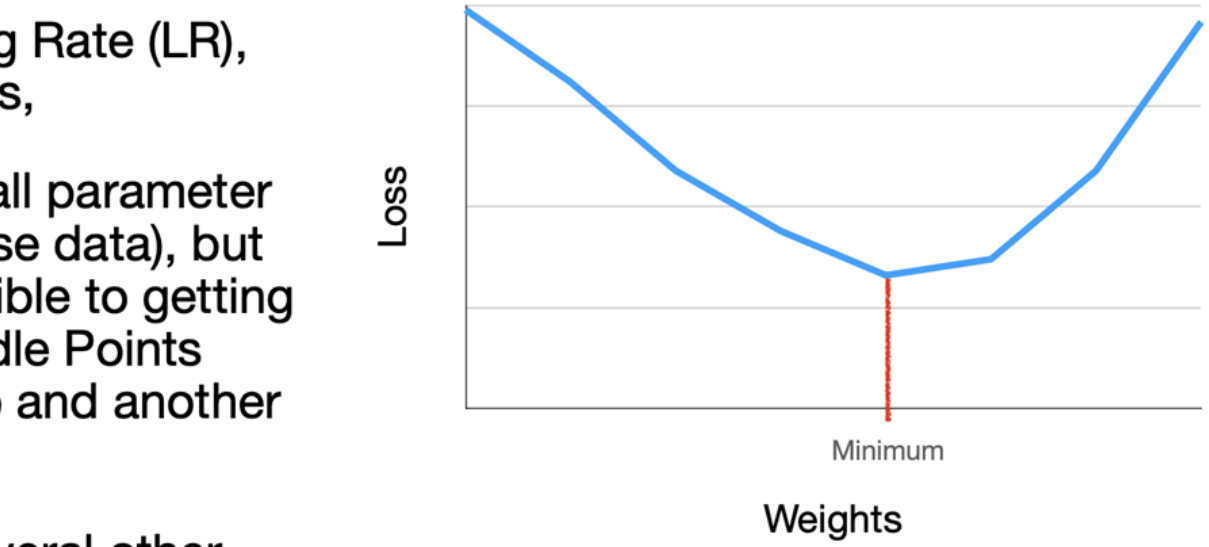
The algorithm we use to update the weights

- You have already been introduced to Gradient Descent which is an example of a first order optimisation algorithm.
- In this section we will explore some alternatives to Stochastic Gradient Descent and take a look at a few other optimisation algorithms.



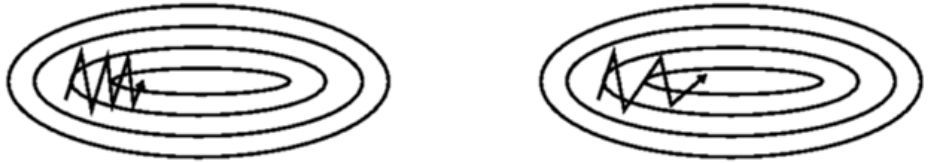
Problems with standard SGD

- Choosing an appropriate Learning Rate (LR), deciding Learning Rate Schedules,
- Using the same learning rate for all parameter updates (as is the case with sparse data), but most importantly SGD is susceptible to getting trapped in Local Minimas or Saddle Points (where one dimensions slopes up and another slopes down).
- To solve some of these issues several other algorithms have been developed including some extensions to SGD which include Momentum and Nestor's Acceleration.



Momentum

- One of the issues with SGD are areas where our hyper-plane is much steeper in one direction.
- This results in SGD oscillating around these slopes making little progress to the minimum point.
- Momentum increases the strength of the updates for dimensions whose gradients switch directions. It also dampens oscillations. Typically we use a Momentum value of 0.9.



(a) SGD without momentum (b) SGD with momentum

Figure 2: Source: Genevieve B. Orr

https://arxiv.org/pdf/1609.04747.pdf

Nesterov's Acceleration

- One problem introduced by Momentum is overshooting the local minimum.
- Nestoerov's Acceleration is effectively a corrective update to the momentum which lets us obtain an approximate idea of where our parameters will be after the update.
- Below we show the corrected Gradient updates (in green)

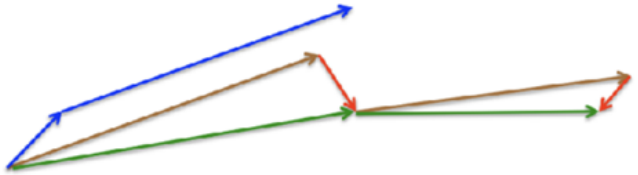
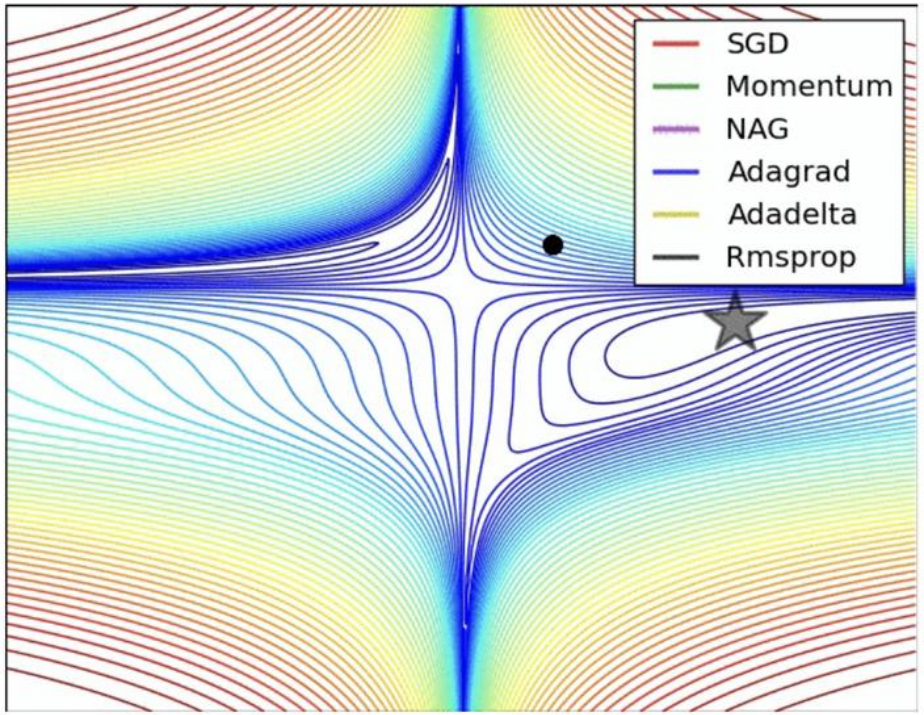


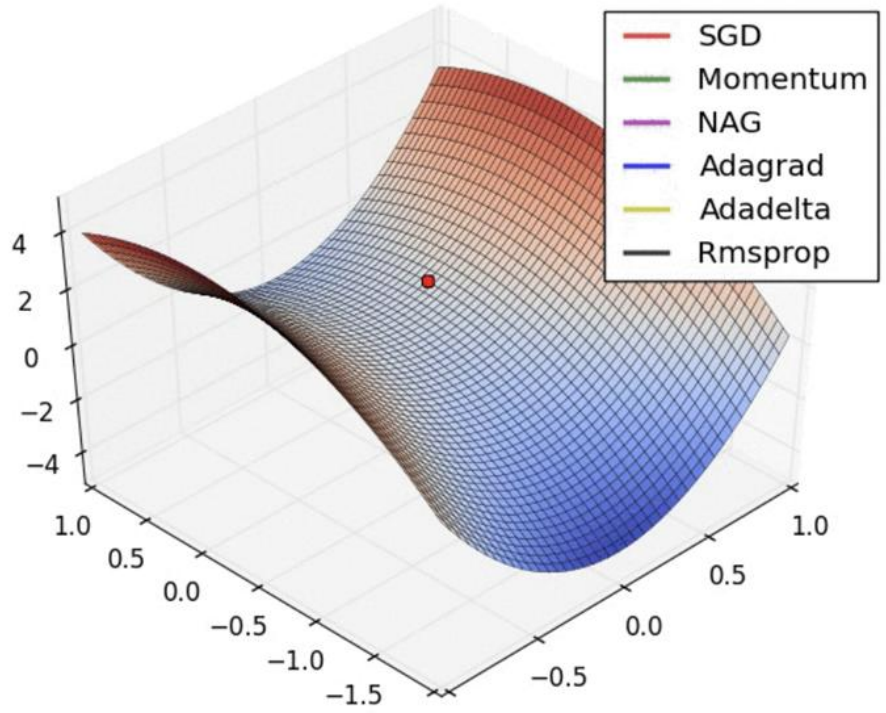
Figure 3: Nesterov update (Source: G. Hinton's lecture 6c)

Other Optimisers

- **Adagrad** - Good for Sparse data. It adapts the learning rate to the parameters, performing smaller updates (i.e. low learning rates) for parameters associated with frequently occurring features, and larger updates (i.e. high learning rates) for parameters associated with infrequent features.
- **Adadelta** - Extension of Adagrad that reduces its aggressive, monotonically decreasing learning rate.
- **Adam** - Adaptive Moment Estimation is a method that computes adaptive learning rates for each parameter. In addition to storing an exponentially decaying average of past squared gradients like Adadelta and RMSprop, Adam also keeps an exponentially decaying average of past gradients, similar to momentum.
- **RMSProp** - Similar to adadelta,
- **AdaMax** - It is a variant of Adam based on the infinity norm.
- **Nadam** - Nesterov accelerated gradient (NAG) is superior to vanilla momentum. Nadam (Nesterov-accelerated Adaptive Moment Estimation) thus combines Adam and NAG. In order to incorporate NAG into Adam, we need to modify its momentum term
- **AMSGrad** - AMSGrad is an extension to the Adam version of gradient descent that attempts to improve the convergence properties of the algorithm, avoiding large abrupt changes in the learning rate for each input variable

Visual Comparison of some Optimisers





http://louistiao.me/notes/visualizing-and-animating-optimization-algorithms-with-matplotlib/

Learning Rate Schedules

- A preset list of learning rates used for each epoch.
- Progressively reduce over time.
- We use LR Schedules because if our LR is too high, it can overshoot the minimum points (areas of lowest loss).
- Applying a progressively decreasing learning rate allows the network to take smaller steps (when gradients are updated) allowing our network to find the point of lowest loss instead of jumping over it.
- Early in the training process, we can afford to take big steps, however as the decrease in loss slows, it is often better to use smaller learning rates to avoid oscillations.
- Learning rate schedules are simply to implement with our Deep learning libraries (PyTorch or Keras/ Tensorflow) as they incorporate a decay parameter in optimisers that support LR schedules, such as SGD.

PHENIKAA UNIVERSITY

Faculty of Computer Science

END